ETAS RTA-BSW v12.7.0

RTA-Can Stack Reference Guide

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-Can Stack Reference Guide R01 EN – 05.2025

Contents

1 RTA-CAN . . . 5
1.1 Introduction . . . 5
2 CAN Interface . . . 7
2.1 Introduction . . . 7
2.2 Supported Features . . . 8
2.3 Extensions . . . 10
2.4 Limitations . . . 24
2.5 Exclusions . . . 26
2.6 Deviations . . . 27
2.7 API Definition . . . 33
2.8 Configuration Advice . . . 35
2.9 Integration Advice . . . 42
2.10 Hardware Requirements . . . 45
3 CAN Network Management . . . 46
3.1 Introduction . . . 46
3.2 Supported Features . . . 46
3.3 Extensions . . . 50
3.4 Limitations . . . 54
3.5 Exclusions . . . 55
3.6 Deviations . . . 55
3.7 API Definition . . . 56
3.8 Configuration Advice . . . 58
3.9 Integration Advice . . . 64
3.10 Hardware Requirements . . . 65
4 CAN State Manager . . . 66
4.1 Introduction . . . 66
4.2 Supported Features . . . 67
4.3 Extensions . . . 68
4.4 Limitations . . . 72
4.5 Exclusions . . . 72
4.6 Deviations . . . 72
4.7 API Definition . . . 73
4.8 Configuration Advice . . . 74

4.9 Integration Advice . . . 75
4.10 Hardware Requirements . . . 76
5 CAN Transport Layer . . . 77
5.1 Introduction . . . 77
5.2 Supported Features . . . 77
5.3 Extensions . . . 82
5.4 Limitations . . . 90
5.5 Exclusions . . . 90
5.6 Deviations . . . 90
5.7 API Definition . . . 91
5.8 Configuration Advice . . . 91
5.9 Integration Advice . . . 97
5.10 Hardware Requirements . . . 99
6 Time Synchronization over CAN . . . 100
6.1 Introduction . . . 100
6.2 Supported Features . . . 101
6.3 Extensions . . . 101
6.4 Limitations . . . 104
6.5 Exclusions . . . 105
6.6 Deviations . . . 105
6.7 API Definition . . . 110
6.8 Configuration Advice . . . 111
6.9 Integration Advice . . . 114
6.10 Hardware Requirements . . . 115
7 Glossary . . . 116

1 RTA-CAN

1.1 Introduction

The RTA-CAN stack encompasses modules from the AUTOSAR Communication Services layer (CanTp, CanNm, CanSM and CanTSyn) and the Communication Hardware layer of ECU Abstraction (CanIf). The RTA-CAN stack is designed specifically for vehicle network communication on a CAN system, and it supports Classic CAN communication (CAN 2.0) and CAN FD communication (if supported by the hardware).

1.1.1 RTA-CAN Modules

The RTA-CAN stack consists of the following modules, each of which is described in its own sub-section in this Reference Guide.

CAN Interface module (CanIf)

CanIf provides a unique interface to manage different CAN hardware device types, including CAN controllers and CAN transceivers. This allows multiple underlying internal and external CAN controllers/transceivers to be controlled by the CanIf module, based on a physical channel view of the network.

CAN Network Management module (CanNm)

CanNm is a hardware independent protocol that can only be used on CAN networks. It provides an adaptation between the Network Management Interface module (Nm) and the CAN Interface module (CanIf).

CAN State Manager (CanSM)

CanSM has interfaces to both the hardware abstraction layer (CanIf, CanNm) and the system service layer (ComM, EcuM, BswM, Dem). It coordinates with CanIf on the handling of network mode transitions of one or more underlying CAN Controllers and CAN Transceiver Drivers, and it uses an internal State Machine to manage the transitions between network modes.

CAN Transport Layer (CanTp)

CanTp is part of the Diagnosis Communication of the Basic Software architecture, and it is responsible for the communication between the peer protocol entities of the network layer in different nodes.

Time Synchronization over CAN (CanTSyn)

CanTSyn handles the distribution of time information over CAN busses.

1.1.2 RTA-CAN Architecture

The diagram below shows the position of the RTA-CAN modules in the Autosar layered architecture.

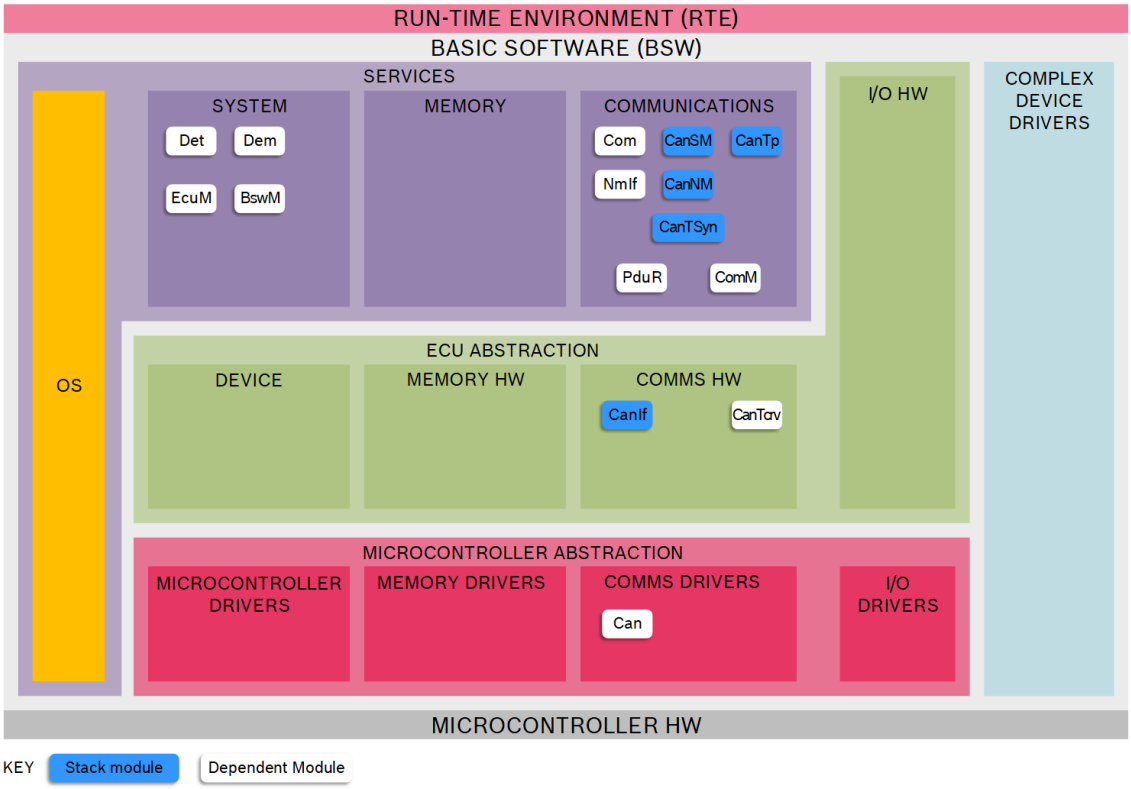


Figure 1.1.RTA-CAN Architecture

2 CAN Interface

Module name	CanIf
Package	RTA-CAN
AUTOSAR Module ID	60
AUTOSAR Version	22-11

2.1 Introduction

The CAN Interface module (CanIf) is located between the low-level CAN device and transceiver drivers (Can, CanTcrv) and the upper communication service layers (e.g. CanSM, CanNm, CanTp, PduR etc). The diagram in the Introduction to this Reference Guide shows the position of the module in the AUTOSAR layered architecture.

The CanIf module provides a unique interface to manage different CAN hardware device types like CAN controllers and CAN transceivers used by the defined ECU hardware layout. This allows multiple underlying internal and external CAN controllers/transceivers to be controlled by the CanIf module, based on a physical channel view of the network.

The main purpose of the CanIf module is to:

- Provide EcuM with a callable function to initialise the CanIf module.
- Implement all CAN hardware independent tasks which belong to the Can and CanTcrv drivers of the corresponding ECU.
- Look after the control flow and data flow requirements of PduR and other upper layer communication modules.
- Provide CAN communication-abstracted access to both the Can (driver) and CanTrcv services. This allows for control and supervision of the CAN network.
- Forward status change requests from CanSM to the lower layer Can and CanTrcv drivers, and notify the upper layers of Can/CanTrcv events.
- Process transmit requests.
- Propagate transmit confirmation, receive indication and error notification messages through the network.
- Start/stop a CAN CC (communication controller) and set the communication mode.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

2.2 Supported Features

2.2.1 Transmission

The CanIf module supports L-PDU transmission according to the AUTOSAR specification via the function call `CanIf_Transmit`. The following sequence of actions are performed when a transmission is requested:

- The initialisation status of the CanIf module is checked.
- The Can driver instance (if multiple CanDrvs are used) and controller ID are determined.
- The HTH (Hardware Transmit Handle) for access to the CAN hardware transmit object is determined.
- The Can driver function `Can_Write` is called to perform the transmit operation.
- If the function returns E_OK, the transmission is successfully completed.

If transmit buffering is enabled (through the CanIfBufferSize configuration parameter), transmitted L-PDUs are stored in a transmit buffer (CanIfTxBuffer) if they are rejected by the Can module following a transmit request, or if a transmit request was aborted. This buffer consists of a series of single element L-PDU buffers. Each L-PDU buffer is statically assigned to one transmit L-PDU handle and one CanId at configuration time. If the buffer of a specific L-PDU is full, CanIf overwrites the older L-PDU with the more recent L-PDU.

CanIf provides a callback function (CanIf_TxConfirmation), which is called by the Can module to notify CanIf if a transmit request completed successfully. CanIf identifies the upper communication layer linked to the successfully transmitted L-PDU and notifies it about the successful transmission by calling the TxConfirmation function of the relevant module.

> **NOTE**
>
> Some CAN controllers allow for cancellation of pending L-PDU transmit requests within their hardware transmit objects. This feature is used to prevent inner priority inversion, which may occur if the priority of an L-PDU requested for transmission is higher than the priority of the L-PDU waiting for transmission in the CAN hardware transmit object.

2.2.2 Reception

L-PDU reception is supported through the function call `CanIf_RxIndication` which is called by the Can module. CanIf then evaluates the L-PDU for acceptance and prepares it for later access by the upper communication layers. If the L-PDU is successfully detected and accepted for further processing, CanIf notifies upper layer modules of this (asynchronous) event using the function `User_RxIndication`.

The `CanIf_ReadRxPduData` function is a common interface for upper layer modules to read CAN L-PDUs recently received from the CAN network. The upper layer modules only initiate the receive request via the CanIf module (they do not have direct access to the Can module). This API makes it possible to read data without a direct dependency on the reception event.

2.2.3 PDU Channel Modes

CanIf supports PDU channel modes, which allow enabling/disabling of transmit and/or receive paths on a per-channel basis, as per the AUTOSAR specification. This feature is normally used for diagnostic purposes. The following modes are supported:

CANIF_OFFLINE	No transmission and reception in channel.
CANIF_RX_OFFLINE	Receive path of the channel is disabled.
CANIF_RX_ONLINE	Receive path of the corresponding channel is enabled.
CANIF_TX_OFFLINE	Transmit path of channel is disabled.
CANIF_TX_ONLINE	Transmit path of channel is enabled.
CANIF_ONLINE	Set to full operation mode.

2.2.4 Busoff Notification

CanIf supports the notification of Busoff events through the CanIf_ControllerBusOff callback function.

When a CAN controller signals the Busoff network event, it sets the relevant controller to STOPPED, and the CanIf function is invoked. This sets the internal controller mode state to CAN_CS_STOPPED and in turn notifies the required upper layer modules. For example, the CanSM module is notified of the state change by invoking the CanSm_ControllerBusOff(ControllerId) callback function. Other modules may be notified (e.g. EcuM, CDD) depending on the configuration of the upper layer interfaces.

If the channel mode is CANIF_TX_OFFLINE or CANIF_OFFLINE, no L-PDUs can be transmitted and thus the CAN controller is not able to go into Busoff mode. In this case the BusOff Notification is implicitly suppressed.

2.2.5 CAN FD (CAN with Flexible Data Rate)

CanIf supports the Flexible Data Rate feature (CAN FD), which is a Can protocol that allows the transmission and reception of L-PDUs using data rates higher than 1 MBit/s and payloads greater than 8 bytes per frame. Data up to 64 bytes can be communicated in a single frame. Compared to conventional CAN frames, CAN FD frames use two additional bits in the header to specify the frame format. The maximum payload of a CAN FD frame is 64 Bytes, and the allowed values are 0 to 8, 12, 16, 20, 24, 32, 48 and 64 Bytes.

CAN FD messages may only be transmitted in pure CAN FD networks. If non-CAN FD nodes actively participate in the communication, they will send Error Frames and destroy the CAN FD messages.

To use this feature, the CAN controller hardware must support CAN FD operations, and must be set to FD mode.

The current implementation of CAN FD supports only Node or Controller level selection of the FD feature.

See the Configuration Advice section for more details.

2.2.6 Multi-Channel Transceivers

Transceivers with more than one channel are supported in this release.

2.2.7 CanIf Calibration

CanIf Calibration support allows the changing of CanId, DLC and Node that are assigned to a CAN message during runtime. This is done via XCP/ETK.

Calibration is provided for Tx CanId (CanIfTxPduCanId), Rx CanId (CanIfRxPduCanId), Tx DLC (PduLength), Rx DLC (CanIfRxPduDlc) and Node assignment for both TxPdus and RxPdus from CanIf.

At present, the calibration for only FullCAN Tx and Rx Pdus and their DLCs is supported.

See the Configuration Advice for details on how to enable this optional feature.

2.2.8 Multiple Can Driver support

The Multiple Can Driver support is available in this release.

See the Configuration Advice for details on how to enable this optional feature.

2.2.9 Bus Mirroring

CanIf facilitates the use of Can to Ethernet Bus Mirroring, which replicates the traffic and state of internal (source) buses to an external (destination) bus. Therefore, a Tester connected to the external bus can monitor internal buses (which are not directly accessible) for debugging purposes.

Please see Section 2.8.8 for more details.

2.3 Extensions

2.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.1.1 CanIf/CanIfCtrlDrvCfg/CanIfCtrlCfg/CanIfCtrlTrcvRef

Short Name	CanIfCtrlTrcvRef

Description	This parameter contains a reference to the CanIf Transceiver to which the Controller is connected to. This parametre is mandatory if EcuM Wake-up is to be supported.The multiplicity is 0..1 This is required to have a mapping of Controller to Transceiver.Implementation Type: reference to CanIfTrcvCfg.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.2 CanIf/CanIfCtrlDrvCfg/CanIfRbCtrlDrvPrefix

Short Name	CanIfRbCtrlDrvPrefix
Description	This parameter is added to support third party CAN Drivers. It contains Vendor Id and Vendor API infix. The symbolic names of CanObjectIds are appended with Vendor Id and Vendor API infix. This parameter should be configured only when CAN Driver is required to append Vendor Id and Vendor API infix in the symoblic name of CanObjectId. If this parameter is configured by mistake and CAN Driver is not generating the symoblic name of CanObjectId without Vendor Id and Vendor API infix, a compiler error is thrown. If this parameter is configured, then HardwareObjectId symbolic names are generated as follows, <CanIfCtrlDrvPrefix>_<canIfHthIdSymRef shortName> If this parameter is not configured, then HardwareObjectId symbolic names are generated as follows, <canIfHthIdSymRef shortName>
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.3 CanIf/CanIfCtrlDrvCfg/CanIfRbForeignDrvSupport

Short Name	CanIfRbForeignDrvSupport
Description	Enable third party driver support.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.4 CanIf/CanIfDispatchCfg/CanIfDispatchErrorPassiveNotificationName

Short Name	CanIfDispatchErrorPassiveNotificationName

Description	This parameter defines the name of <User_ErrorPassiveNotification>.This parameter defines the function to be invoked when there is Error Passive notification by the underlying Can Driver.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

2.3.1.5 CanIf/CanIfInitCfg/CanIfInitHohCfg/CanIfHrhCfg/CanIfHrhListCfg

Short Name	CanIfHrhListCfg
Description	This container contains the configuration of CanId lists to be received
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.6 CanIf/CanIfInitCfg/CanIfInitHohCfg/CanIfHrhCfg/CanIfHrhListCfg/ CanIfHrhListCanId

Short Name	CanIfHrhListCanId
Description	This parameter contains the list of CanIds to be received via this HRH.
Multiplicity	1..*
Value Range	0..536870911
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.7 CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfRxPduType

Short Name	CanIfRxPduType
Description	Defines the type of each receive CAN L-PDU. 1). DYNAMIC - CanId of the Rx L-PDU can be reconfigured 2). STATIC - CanId of the Rx L-PDU cannot be reconfigured. The default value is STATIC (even if this parameter is left unconfigured). The API CanIf_Rb_ChangeRxPduCanId() is used to reconfigure the CanId of RxPdu if the CanIfRxPduType is configured as DYNAMIC.
Multiplicity	0..1
Default Value	-
Enumeration Literals	DYNAMIC, STATIC
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.8 CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfRbBusMirroringBufSize

Short Name	CanIfRbBusMirroringBufSize
Description	Size of the FIFO mapped to corresponding TxPdu. This is used to store Sdu Data.
Introduction	Range: 0..max. number of CantTxPduIds
Multiplicity	0..1
Value Range	0..10
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.9 CanIf/CanIfPrivateCfg/CanIfRbXCore

Short Name	CanIfRbXCore
Description	Module to define CanIf XCore parameters to support multi-core operation of the CanIf transmission, reception and transmission confirmation
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.10 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCoreCanCon-trollerCoreAffinity

Short Name	CanIfRbXCoreCanControllerCoreAffinity
Description	This container defines which CPU Core has “ownership” on which CAN Controller. This means that for each Can controller configured, an element must be present here to configure the destination core on which it will operate. The CAN Controller Interrupt and/or CAN Main function RX and TX will be scheduled on the owner CPU Core. Example: CAN Controller 0 will be owned by CPU Core 2. CAN Interrupt for Controller 0 MUST be routed to CPU Core 2
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.11 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCoreCanCon-trollerCoreAffinity/CanIfRbXCoreCanControllerRef

Short Name	CanIfRbXCoreCanControllerRef

Description	This parameter holds a reference to the Can controller that will operate on the core context specified by CanIf XCore DestCoreID
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.12 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCoreCanControllerCoreAffinity/CanIfRbXCoreCtrlDestCoreID

Short Name	CanIfRbXCoreCtrlDestCoreID
Description	Indicates the core ID on which context CanIfRbXCoreCanControllerRef will operate
Multiplicity	1..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.13 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCoreGeneral

Short Name	CanIfRbXCoreGeneral
Description	Container for general pre-compile parameters of the CanIf xcore.
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.14 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCoreGeneral/CanIfRbXCoreDebug

Short Name	CanIfRbXCoreDebug
Description	DEBUG_INSTRUMENTATION_ACTIVE ON or OFF.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.15 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe

Short Name	CanIfRbXCorePipe

Description	Configuration of the pipes used by CanIf XCore when a Tx confirmation, Tx request or Rx indication needs to be redirected from the current core to the destination core. For each user type of Tx confirmation and Rx indication a pipe will be defined from the Can controller to the core designated to handle that specific type of callback(specification per PDU of destination core is not possible, only per user type and callback type). For Tx, for each core to Can controller a pipe will be defined except for the core the Can controller runs on (a direct call is allowed).
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.16 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe/CanIfRbXCoreDestRecvId

Short Name	CanIfRbXCoreDestRecvId
Description	TaskID of the receiver on the destination core. Task name of the destination core should be as per the configured OS Task name.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.17 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe/CanIfRbXCorePipeDestCore

Short Name	CanIfRbXCorePipeDestCore
Description	Destination Core ID of this pipe
Multiplicity	1..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.18 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe/CanIfRbXCorePipeId

Short Name	CanIfRbXCorePipeId
Description	Unique ID of the pipe
Multiplicity	1..1
Value Range	0..65535
Default Value	-

Type	ECUC-INTEGER-PARAM-DEF

2.3.1.19 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe/CanIfRbXCorePipeProcessingMode

Short Name	CanIfRbXCorePipeProcessingMode
Description	Pipe Processing Mode as Polling, INTERRUPT, TASK
Multiplicity	1..1
Default Value	-
Enumeration Literals	POLLING, SWI, TASK
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.20 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe/CanIfRbXCorePipeProcessingType

Short Name	CanIfRbXCorePipeProcessingType
Description	Pipe Processing Type as Rx, Tx and TxConfirmation.
Multiplicity	1..1
Default Value	-
Enumeration Literals	RX, TX, TXCONF
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.21 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe/CanIfRbXCorePipeRamSize

Short Name	CanIfRbXCorePipeRamSize
Description	RAM_SIZE of this pipe
Multiplicity	1..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.22 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe/CanIfRbXCorePipeSrcCore

Short Name	CanIfRbXCorePipeSrcCore
Description	Source Core ID of this pipe
Multiplicity	1..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.23 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCorePipe/CanIfRbXCorePipeTaskPeriod

Short Name	CanIfRbXCorePipeTaskPeriod
Description	Pipe Task Period [ms]. when configured Task, calling period must be set
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.24 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCoreUserTypeCoreAffinity

Short Name	CanIfRbXCoreUserTypeCoreAffinity
Description	This container defines which CPU Core will receive the traffic for a specific CanIf_Cfg_UserType_ten. Eg: CAN NM traffic can be routed to Core1, and Can TP traffic can be routed to Core0
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.25 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCoreUserTypeCoreAffinity/CanIfRbUserType

Short Name	CanIfRbUserType
Description	Indicates UserType group of PDU's which Tx confirmation and Rx indication will be processed on the core indicated by CanIf XCore DestCoreID
Multiplicity	1..1
Default Value	-
Enumeration Literals	CAN_NM, J1939NM, CAN_TSYN, CDD, J1939TP, RBA_J1939FDTP, XCP
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.26 CanIf/CanIfPrivateCfg/CanIfRbXCore/CanIfRbXCoreUserTypeCoreAffinity/CanIfRbXCoreUserTypeDestCoreID

Short Name	CanIfRbXCoreUserTypeDestCoreID
Description	Indicates the core ID which will handle all the Tx confirmation and Rx indication for PDU's configured with CanIf UserType
Multiplicity	1..1

Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.27 CanIf/CanIfPublicCfg/CanIfPublicBusoffRecoveryfromRxIndication

Short Name	CanIfPublicBusoffRecoveryfromRxIndication
Description	Configuration parameter to make busoff recovery from Rx indication optional. By default, this parameter shall be FALSE as it is a RB specific parameter and this would impact the Busoff recovery in a way which is not intended by Autosar.. Then the busoff recovery would consider Rx indication of the Pdu transmitted to recover from busoff apart from getting Tx conformation for the same.If not, the busoff may be considered as recovered after a successful transmission.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.28 CanIf/CanIfPublicCfg/CanIfPublicCanIfLiteConfiguration

Short Name	CanIfPublicCanIfLiteConfiguration
Description	This parameter defines whether CanIf Lite configuration is required(TRUE) or not(FALSE).
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.29 CanIf/CanIfPublicCfg/CanIfPublicCancelTransmitSupport

Short Name	CanIfPublicCancelTransmitSupport
Description	Configuration parameter to enable/disable dummy API for upper layer modules which allows to request the cancellation of an I-PDU.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.30 CanIf/CanIfPublicCfg/CanIfPublicErrorPassiveSupport

Short Name	CanIfPublicErrorPassiveSupport

Description	This parameter defines whether Error Passive Support is configured(TRUE) or not(FALSE).
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.31 CanIf/CanIfPublicCfg/CanIfPublicIcomSupport

Short Name	CanIfPublicIcomSupport
Description	Selects support of Pretended Network features in CanIf. AUTOSAR_Dev-CanIfPublicIcomSupport is not supported currently hence the multiplicity is changed as 0.
Introduction	True: Enabled False: Disabled
Multiplicity	0..0
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.32 CanIf/CanIfPublicCfg/CanIfPublicUserDlcErrorNotificationName

Short Name	CanIfPublicUserDlcErrorNotificationName
Description	This parameter defines the function to be invoked when there is a error in the received DLC. This parameter depends on the parameter CanIfPrivateDlcCheck.If CanIfPrivateDlcCheck is true and if an RxPdu with invalid DLC is received. It is notified to <User_DlcErrorNotifaction>.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

2.3.1.33 CanIf/CanIfPublicCfg/CanIfPublicUserTxBufClearNotifCallBack

Short Name	CanIfPublicUserTxBufClearNotifCallBack
Description	This parameter defines the function to be invoked by CanIf when a PDU in the TxBuffer is cleared due to CAN driver couldn't accept the transmission request. This function will be called in Interrupt lock context. So the function has to be very small.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

2.3.1.34 CanIf/CanIfPublicCfg/CanIfPublicUserTxBufClearNotification

Short Name	CanIfPublicUserTxBufClearNotification
Description	This parameter enables/disables the user notification when a PDU in the TxBuffer is cleared due to CAN driver couldn't accept the transmission request.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.35 CanIf/CanIfPublicCfg/CanIfPublicUserTxInOfflineMode

Short Name	CanIfPublicUserTxInOfflineMode
Description	This parameter configures whether USER Pdus have to be transmit-ted(TRUE) or not(FALSE) when the TxPdu Mode is OFFLINE.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.36 CanIf/CanIfPublicCfg/CanIfRbCalibrationSupport

Short Name	CanIfRbCalibrationSupport
Description	This container contains the switch which has to be enabled/disabled for Calibration support
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.37 CanIf/CanIfPublicCfg/CanIfRbCalibrationSupport/CanIfRbCalibra-tion

Short Name	CanIfRbCalibration
Description	This parameter defines whether CanIf supports calibration (of Tx CanId, Rx CanId, Tx DLC, Rx DLC and assigning Node for the Pdus) or not.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.38 CanIf/CanIfPublicCfg/CanIfRbChangeRxPduIdApi

Short Name	CanIfRbChangeRxPduIdApi

Description	Enables and disables the API for reconfiguration of the CAN Identifier for each Receive L-PDU. True: Enabled False: Disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.39 CanIf/CanIfPublicCfg/CanIfRbForeignCanTrcvDrvSupport

Short Name	CanIfRbForeignCanTrcvDrvSupport
Description	Enable third party CanTransceiver driver support.
Introduction	When enabled, the callback APIs of CanIf will be named CanIf_Integn_{CallbackAPIName} and also re-routes driver API calls to integration code. When this configuration parameter is enabled, the template files CanIf_Integn.c_tpl and CanIf_Integ-n.h_tpl shall be used for call adaptation between CanIf and third party CanTrcv drivers.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.40 CanIf/CanIfPublicCfg/CanIfRbPublicDlcErrorNotifHeaderFile

Short Name	CanIfRbPublicDlcErrorNotifHeaderFile
Description	Defines header files for callback functions which shall be included in case CanIfPublicUserDlcErrorNotificationName parameter is configured.
Multiplicity	0..1
String Length Range	1..32
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.41 CanIf/CanIfPublicCfg/CanIfRbPublicUserHeaderFile

Short Name	CanIfRbPublicUserHeaderFile
Description	Defines header files for callback functions which shall be included in case of USER. Range of characters is 1.. 32
Multiplicity	0..*
String Length Range	1..32
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.42 CanIf/CanIfPublicCfg/CanIfRbVarDlcSupport

Short Name	CanIfRbVarDlcSupport
Description	Determines if or if not CanIf shall enable Variable DLC handling .
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.43 CanIf/CanIfPublicCfg/CanIfReadMsgIdApi

Short Name	CanIfReadMsgIdApi
Description	Enables / Disables the APIs CanIf_ReadTxmsgId() and CanIf_ReadRxmsgId() for reading configured CanId for configured hardware object (HOH). True: Enabled False: Disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.2 Additional Services

The following API functions are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.2.1 CanIf_Get_DirectHw_Info

Syntax	`CanIf_Get_DirectHw_Info (PduIdType CanTxPduId, rba_CanDirectHWInfo_o * hw_info_po)`
Parameters	`CanTxPduId` L-PDU handle of CAN L-PDU successfully transmitted.
	`hw_info_pco`
Description	Returns the information required for the HW direct access to CAN. Extract the PDU handle from CanIf configuration structure (variant dependent), then call rba_Can_Get_DirectHw_Info to fill further information in the info structure.

2.3.2.2 CanIf_DirectHw_Write

Syntax	`CanIf_DirectHw_Write(const uint32 * p_tx_data_buff_u32, uint32 data_len_u32, const rba_CanDirectHWInfo_o * hw_info_po)`
Parameters	`p_tx_data_buff_u32`
	`data_len_u32`
	`hw_info_pco`
Description	Initiates a request for direct HW write for CAN L-PDU specified by the CanTxPduId. The only check performed is to avoid sending data while CAN Drv not confirmed “running” (Device mode == STARTED)

2.3.2.3 CanIf_ResetDynamicTxId

Syntax	`CanIf_ResetDynamicTxId(PduIdType CanTxPduId)`
Parameters	`CanTxPduId` L-PDU handle of CAN L-PDU successfully transmitted.
Description	Reconfigures the corresponding Static CanId of the requested CAN L-PDU.

2.3.2.4 CanIf_Rb_ReadTxPduCanId

Syntax	`CanIf_Rb_ReadTxPduCanId(PduIdType CanIfTxPduId, uint32 * TxPduCanId)`
Parameters (in)	`CanIfTxPduId`
Parameters (out)	`*TxPduCanId`
Description	Returns CanId when the CanIfTRxPduId is given.

2.3.2.5 CanIf_Rb_ReadRxPduCanId

Syntax	`CanIf_Rb_ReadRxPduCanId(PduIdType CanIfRxPduId, uint32 *RxPduCanId)`
Parameters (in)	`CanIfRxPduId`
Parameters (out)	`*RxPduCanId`
Description	Returns CanId when the CanIfRxPduId is given.

2.3.2.6 CanIf_Rb_ChangeRxPduCanId

Syntax	`CanIf_Rb_ChangeRxPduCanId(PduIdType CanRxPduId, Can_IdType CanId)`
Parameters	`CanRxPduId`
	`CanId`
Description	Reconfigures the corresponding CAN identifier of the requested Receive L-PDU.

2.3.2.7 CanIf_ControllerErrorPassive

Syntax	`CanIf_ControllerErrorPassive`
Parameters	`Controller` The ID of the CAN controller to address.
Description	This function indicates a CAN controller passive error event for the specified CAN controller.
	This callback function is called by the Can driver module.

2.3.2.8 CanIf_ReturnTxPduId

Syntax	`CanIf_ReturnTxPduId`
Parameters	`CanIfTxTargetPduId` Tx Target PduId of a CDD Tx-Pdu.

	`CanIfTxPduIdPtr` Pointer provided by the upper layer where the Tx-PduId of the queried Target Id will be placed.
Description	Puts the Tx-PduId into CanIfTxPduIdPtr for the queried TxTargetPduId. Returns E_OK if proper match is found; E_NOT_OK for an invalid TxTargetP-duld or when CanIfTxPduIdPtr is passed as NULL.
	This function is only required for use with CDD modules.

2.3.2.9 CanIf_ReturnRxPduId

Syntax	`CanIf_ReturnRxPduId`
Parameters	`CanIfRxTargetPduId` Rx Target PduId of a CDD Rx-Pdu.
	`CanIfRxPduIdPtr` Pointer provided by the upper layer where the Rx-PduId of the queried Target Id will be placed.
Description	Puts the Rx-PduId into CanIfRxPduIdPtr for the queried RxTargetPduId. Returns E_OK if proper match is found; E_NOT_OK for an invalid RxTargetP-duld or when CanIfRxPduIdPtr is passed as NULL.
	This function is only required for use with CDD modules.

2.3.2.10 CanIf_ReadTxMsgId

Syntax	`CanIf_ReadTxMsgId`
Parameters	`msgHandleTx_u16` HOH (Hardware Object Handle) generated by Can driver module.
Description	Returns the CanId for the given HOH handle as configured in the Can driver module (transmitted message).

2.3.2.11 CanIf_ReadRxMsgId

Syntax	`CanIf_ReadRxMsgId`
Parameters	`msgHandleRx_u16` HOH (Hardware Object Handle) generated by Can driver module.
Description	Returns the CanId for the given HOH handle as configured in the Can driver module (received message).

2.3.2.12 CanIf_CancelTransmit

Syntax	`CanIf_CancelTransmit`
Parameters	`CanIfTxSduId`
Description	This is a dummy API introduced for interface compatibility.

2.4 Limitations

The following features are not supported for version v12.7.0:

2.4.1 General

- The maximum number of controllers supported by CanIf is 65,535.
- Wakeup is only supported for Can Transceivers and not for CAN Drivers.
- The XCP Direct Hardware access feature is not supported for TxPdus configured with CanIfTxPduType as DYNAMIC.
- The Pretended Networking feature is not supported (has been removed since AR 20-11)
- The TTCan feature [ECUC_CanIf_00675] is not supported.
- Multiple transceiver drivers are not supported, i.e. it is not possible to use two different CAN hardware devices that each require a separate transceiver driver concurrently. However, it is possible to use a single driver to control multiple transceivers.
- Wakeup by controller is not supported, only wakeup by transceiver is supported.
- The CanIf implementation for CanXL is not supported in Post Build Variant. Therefore CanXL should not be used if Post build variants are configured in CanIf.
- The CanXL does not support Partial Network features, Wake Up Validation features, read Rx PDU data and receive notification status features. CanXL does not support Dynamic update of CanId in Tx and Rx PDUs. BASIC CAN Rx PDU is not supported by CanXL.
- The MDS strategy is changed when generating the driver's symbolic name. For Xcan and Rscan drivers, the symbolic name will be generated based upon the `CanRbDriverInfix` parameter value. Also, irrespective of MDS ON/OFF, driver APIs will be called through a function pointer.

2.4.2 Software Filtering

- Filtering of only same type is supported - e.g. If RANGE_ID is configured in HRH, then it should also be configured in PDU level.

2.4.3 CanIf Calibration

- When the Calibration is ON, the DLC sent from the upper layer is not considered, instead the configured DLC is used. This introduces two limitations: (1) If the Tx Pdu DLC is configured, for example, as 8bytes, and the upper layer sends only 4bytes of data, the remaining 4bytes of data in the PDU will contain junk values. (2) If the Tx Pdu DLC is configured, for example, as

2bytes, and the upper layer sends 4bytes of data, a DET error will be reported.

- Since the PDUs other than COM will use SduLength provided by the upper layer for Transmission, the Tx DLC calibration parameter is provided only for COM PDUs.
- CanIf does not provide a DLC calibration parameter for the TxPdus configured with PduLength as '0'.
- CanIf does not provide a CanId calibration parameter for the TxPdus configured with CanIfTxPduType as DYNAMIC and for TxPdus not configured with CanIfTxPduCanId.
- CanIf does not provide a CanId calibration parameter for the RxPdus configured with CanIfRxPduType as DYNAMIC.
- CanIf does not provide a DLC calibration parameter for the RxPdus configured with CanIfRxPduDataLength as '0'.

2.4.4 Bus Mirroring

- When Multi-Driver Support is enabled, CanIf supports Bus Mirroring *only* with the MCAN driver.
- CanIf supports Bus Mirroring only for drivers of Tx processing type DEDICATED. Bus Mirroring is not therefore supported for drivers with the FIFO / QUEUE type of Tx buffer handling.

2.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `CanIf_GetControllerRxErrorCounter`

Callback functions not currently implemented:

- `CanIf_ConfirmCtrlPnAvailability`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- CanIf/CanIfCtrlDrvCfg/CanIfCtrlDrvInitHohConfigRef
- CanIf/CanIfInitCfg/CanIfInitCfgSet
- CanIf/CanIfInitCfg/CanIfMaxBufferSize
- CanIf/CanIfInitCfg/CanIfMaxRxPduCfg
- CanIf/CanIfInitCfg/CanIfMaxTxPduCfg
- CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfRxPduDataLengthCheck
- CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfTTRxFrameTriggering

- CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfTTRxFrameTriggering/CanIfTTRxHwObjectTriggerIdRef
- CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfTTRxFrameTriggering/CanTTRxJoblistTimeMark
- CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTTTxFrameTriggering
- CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTTTxFrameTriggering/CanIfTTTxHwObjectTriggerIdRef
- CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTTTxFrameTriggering/CanIfTTTxJoblistTimeMark
- CanIf/CanIfPrivateCfg/CanIfPrivateSoftwareFilterType
- CanIf/CanIfPrivateCfg/CanIfSupportTTCAN
- CanIf/CanIfPrivateCfg/CanIfTTGeneral
- CanIf/CanIfPrivateCfg/CanIfTTGeneral/CanIfTTDemEventParameterRefs
- CanIf/CanIfPrivateCfg/CanIfTTGeneral/CanIfTTDemEventParameterRefs/CANIF_TT_E_JLE_SYNC
- CanIf/CanIfPrivateCfg/CanIfTTGeneral/CanIfTTJoblist
- CanIf/CanIfPrivateCfg/CanIfTTGeneral/CanIfTTMaxIsrDelay
- CanIf/CanIfPublicCfg/CanIfPublicHandleTypeEnum
- CanIf/CanIfPublicCfg/CanIfSecurityEventRefs
- CanIf/CanIfPublicCfg/CanIfSecurityEventRefs/CANIF_SEV_ERRORSTATE_BUSOFF
- CanIf/CanIfPublicCfg/CanIfSecurityEventRefs/CANIF_SEV_ERRORSTATE_PASSIVE
- CanIf/CanIfPublicCfg/CanIfSecurityEventRefs/CANIF_SEV_RX_ERROR_DETECTED
- CanIf/CanIfPublicCfg/CanIfSecurityEventRefs/CANIF_SEV_TX_ERROR_DETECTED

2.6 Deviations

2.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

2.6.1.1 CanIf/CanIfCtrlDrvCfg/CanIfCtrlCfg

[RTA-BSW] Description	This container contains the configuration (parameters) of an adressed CAN controller by an underlying CAN Driver module. This container is configurable per CAN controller. AUTOSAR_Dev-PB variant Multiplicity is changed from false to true

[AUTOSAR] Description	This container contains the configuration (parameters) of an adressed CAN controller by an underlying CAN Driver module. This container is configurable per CAN controller.

2.6.1.2 CanIf/CanIfCtrlDrvCfg/CanIfCtrlCfg/CanIfCtrlWakeupSupport

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.3 CanIf/CanIfInitCfg/CanIfBufferCfg/CanIfBufferSize

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.4 CanIf/CanIfInitCfg/CanIfInitHohCfg/CanIfHrhCfg/CanIfHrhIdSymRef

[RTA-BSW] Introduction	CanIf receives the following information of the CanDrv module by this reference: - CanHandleType (only CAN 2.0 and CAN FD) - CanObjectId
[AUTOSAR] Introduction	CanIf receives the following information of the CanDrv module by this reference: * CanHandleType (only CAN 2.0 and CAN FD) * CanObjectId

2.6.1.5 CanIf/CanIfInitCfg/CanIfInitHohCfg/CanIfHrhCfg/CanIfHrhRangeCfg

[RTA-BSW] Upper Multiplicity	1
[AUTOSAR] Upper Multiplicity	*

2.6.1.6 CanIf/CanIfInitCfg/CanIfInitHohCfg/CanIfHrhCfg/CanIfHrhSoftwareFilter

[RTA-BSW] Description	Selects the hardware receive objects by using the HRH range/list from CAN Driver configuration to define, for which HRH a software filtering has to be performed at during receive processing. AUTOSAR_Dev- The multiplicity is changed to zero as this parameter is not needed for supporting any feature. And is having not added advantage
[AUTOSAR] Description	Selects the hardware receive objects by using the HRH range/list from CAN Driver configuration to define, for which HRH a software filtering has to be performed at during receive processing.
[RTA-BSW] Multiplicity	0..0

[AUTOSAR] Multiplicity	1..1

2.6.1.7 CanIf/CanIfInitCfg/CanIfInitHohCfg/CanIfHthCfg/CanIfHthIdSymRef

[RTA-BSW] Introduction	CanIf receives the following information of the CanDrv module by this reference: - CanHandleType (only for CAN 2.0 and CAN FD) - CanObjectId
[AUTOSAR] Introduction	CanIf receives the following information of the CanDrv module by this reference: * CanHandleType (only for CAN 2.0 and CAN FD) * CanObjectId

2.6.1.8 CanIf/CanIfInitCfg/CanIfRxPduCfg

[RTA-BSW] Introduction	The SHORT-NAME of "CanIfRxPduConfig" container itself represents the symolic name of Receive L-PDU. This L-SDU produces a meta data item of type CAN_ID_32.
[AUTOSAR] Introduction	The SHORT-NAME of "CanIfRxPduConfig" container itself represents the symolic name of Receive L-PDU. This L-SDU produces a meta data item of type CAN_ID_32, and, depending on CanIfRxPduXLParams, meta data items of type PRIORITYID_16, VLAN_16, SDUTYPE_8, and ACCEPTANCEFIELD_32.

2.6.1.9 CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfRxPduReadData

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.10 CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfRxPduReadNotifyStatus

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.11 CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfRxPduUserRxIndicationName

[RTA-BSW] Introduction	This parameter depends on the parameter CanIfRxPduUserRxIndicationUL. If CanIfRxPduUserRxIndicationUL equals CAN_NM, PDUR, XCP, CAN_TSYN, J1939NM, J1939TP or rba_J1939FdTp the name of the <User_RxIndication> is fixed. If CanIfRxPduUserRxIndicationUL equals CDD, the name of the <User_RxIndication> is selectable.

[AUTOSAR] Introduction	This parameter depends on the parameter CanIfRxPduUserRxIndicationUL. If CanIfRxPduUserRxIndicationUL equals CAN_TP, CAN_NM, PDUR, XCP, CAN_TSYN, J1939NM or J1939TP, the name of the ‹User_RxIndication› is fixed. If CanIfRxPduUserRxIndicationUL equals CDD, the name of the ‹User_RxIndication› is selectable.

2.6.1.12 CanIf/CanIfInitCfg/CanIfRxPduCfg/CanIfRxPduUserRxIndicationUL

[RTA-BSW] Enumeration Literals	CAN_NM, CAN_TSYN, CDD, J1939NM, J1939TP, RBA_J1939FDTP, PDUR, XCP
[AUTOSAR] Enumeration Literals	CAN_NM, CAN_TP, CAN_TSYN, CDD, J1939NM, J1939TP, PDUR, XCP

2.6.1.13 CanIf/CanIfInitCfg/CanIfTxPduCfg

[RTA-BSW] Introduction	The SHORT-NAME of “CanIfTxPduConfig” container represents the symolic name of Transmit L-PDU. This L-SDU consumes a meta data item of type CAN_ID_32 and depending on CanIfTxPduXLParams, meta data items of type PRIORITYID_16, VLAN_16, SDUTYPE_8 and ACCEPTANCE-FIELD_32.
[AUTOSAR] Introduction	The SHORT-NAME of “CanIfTxPduConfig” container represents the symolic name of Transmit L-PDU. This L-SDU consumes a meta data item of type CAN_ID_32, and, depending on CanIfTxPduXLParams, meta data items of type PRIORITYID_16, VLAN_16, SDUTYPE_8, and ACCEPTANCE-FIELD_32.

2.6.1.14 CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTxPduCanIdMask

[RTA-BSW] Maximum Value	536870911
[AUTOSAR] Maximum Value	3758096383
[AUTOSAR] Default Value	3758096383
[RTA-BSW] Default Value	536870911

2.6.1.15 CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTxPduReadNotifyStatus

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.16 CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTxPduTruncation

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.17 CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTxPduType

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	STATIC

2.6.1.18 CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTxPduUserTriggerTransmitName

[RTA-BSW] Description	This parameter defines the name of the <User_TriggerTransmit>. This parameter depends on the parameter CanIfTxPduUserTxConfirmationUL. If CanIfTxPduUserTxConfirmationUL equals CAN_NM, PDUR, XCP, CAN_TSYN, J1939NM, J1939TP or rba_J1939FdTp the name of the <User_TriggerTransmit> is fixed. If CanIfTxPduUserTxConfirmationUL equals CDD, the name of the <User_TxConfirmation> is selectable.
[AUTOSAR] Description	This parameter defines the name of the <User_TriggerTransmit>. This parameter depends on the parameter CanIfTxPduUserTxConfirmationUL. If CanIfTxPduUserTxConfirmationUL equals CAN_TP, CAN_NM, PDUR, XCP, CAN_TSYN, J1939NM or J1939TP, the name of the <User_TriggerTransmit> is fixed. If CanIfTxPduUserTxConfirmationUL equals CDD, the name of the <User_TxConfirmation> is selectable.

2.6.1.19 CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTxPduUserTxConfirmationName

[RTA-BSW] Introduction	This parameter depends on the parameter CanIfTxPduUserTxConfirmationUL. If CanIfTxPduUserTxConfirmationUL equals CAN_NM, PDUR, XCP, CAN_TSYN, J1939NM, J1939TP or rba_J1939FdTp, the name of the <User_TxConfirmation> is fixed. If CanIfTxPduUserTxConfirmationUL equals CDD, the name of the <User_TxConfirmation> is selectable.
[AUTOSAR] Introduction	This parameter depends on the parameter CanIfTxPduUserTxConfirmationUL. If CanIfTxPduUserTxConfirmationUL equals CAN_TP, CAN_NM, PDUR, XCP, CAN_TSYN, J1939NM or J1939TP, the name of the <User_TxConfirmation> is fixed. If CanIfTxPduUserTxConfirmationUL equals CDD, the name of the <User_TxConfirmation> is selectable.

2.6.1.20 CanIf/CanIfInitCfg/CanIfTxPduCfg/CanIfTxPduUserTxConfirmationUL

[RTA-BSW] Enumeration Literals	CAN_NM, CAN_TSYN, CDD, J1939NM, J1939TP, RBA_J1939FDTP, PDUR, XCP
[AUTOSAR] Enumeration Literals	CAN_NM, CAN_TP, CAN_TSYN, CDD, J1939NM, J1939TP, PDUR, XCP

2.6.1.21 CanIf/CanIfPrivateCfg/CanIfFixedBuffer

[RTA-BSW] Introduction	TRUE: Minimum buffer element length is fixed to 8 Bytes. FALSE: Buffer element length depends on the size of the referencing PDUs.
[AUTOSAR] Introduction	TRUE: Minimum buffer element length is fixed to 8 Bytes. FALSE: Buffer element length depends on the configured length of the referenced global PDUs (see ECUC_EcuC_00078).
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.22 CanIf/CanIfPublicCfg/CanIfDevErrorDetect

[AUTOSAR] Default Value	false
[RTA-BSW] Default Value	true

2.6.1.23 CanIf/CanIfPublicCfg/CanIfEnableSecurityEventReporting

[RTA-BSW] Description	Enables/Disables the IdsM callback APIs used when IdsM errors are to be reported.
[AUTOSAR] Description	Switches the reporting of security events to the IdsM: Tags: atp.Status=draft
[RTA-BSW] Introduction	True: Enabled False: Disabled
[AUTOSAR] Introduction	- true: reporting is enabled. - false: reporting is disabled.

2.6.1.24 CanIf/CanIfPublicCfg/CanIfGlobalTimeSupport

[RTA-BSW] Description	Enables/Disables the Global Time APIs used when hardware timestamping is supported.
[AUTOSAR] Description	Enables/Disables the Global Time APIs used when hardware timestamping is supported. Tags: atp.Status=draft
[ADDED] Introduction	True: Enabled False: Disabled
[ADDED] Default Value	false

2.6.1.25 CanIf/CanIfPublicCfg/CanIfPublicMultipleDrvSupport

[RTA-BSW] Description	Selects support for multiple CAN Drivers. AUTOSAR_Dev-Default value is updated to false. This is a deviation from AUTOSAR.
[AUTOSAR] Description	Selects support for multiple CAN Drivers.
[AUTOSAR] Default Value	true
[RTA-BSW] Default Value	false

2.6.1.26 CanIf/CanIfPublicCfg/CanIfVersionInfoApi

[AUTOSAR] Default Value	false
[RTA-BSW] Default Value	true

2.7 API Definition

List of supported API functions

Table 2.84.CanIf Supported API Functions

`CanIf_Init`	Initialises the CanIf module.
`CanIf_DeInit`	De-initialises the CanIf module.
`CanIf_SetControllerMode`	Initiates a transition to the requested mode for the given controller.
`CanIf_GetControllerMode`	Provides the current status of the requested controller.
`CanIf_GetControllerErrorState`	This service calls the corresponding CAN Driver service for obtaining the error state of the CAN controller.
`CanIf_Transmit`	Initiates a request for transmission of the L-PDU.
`CanIf_ReadRxPduData`	Returns the DLC and received data of the requested CanRxPduId to the calling upper layer.
`CanIf_ReadTxNotifStatus`	Returns the confirmation status of a specific static or dynamic Tx L-PDU for the requested CanTxPduId.
`CanIf_ReadRxNotifStatus`	Returns the indication status of a specific Rx L-PDU for the requested CanRxPduId.
`CanIf_SetPduMode`	Sets the current mode for the specified PDU channel.
`CanIf_GetPduMode`	Reports the current mode for a specified PDU channel.
`CanIf_GetVersionInfo`	Returns version information for the CanIf module.
`CanIf_SetDynamicTxId`	Reconfigures the corresponding CanId for the specified L-PDU.

`CanIf_SetTrcvMode`	Sets the operation mode of the specified transceiver.
`CanIf_GetTrcvMode`	Returns the current operating mode of the specified transceiver.
`CanIf_GetTrcvWakeupReason`	Returns the reason for the wakeup of the specified transceiver.
`CanIf_SetTrcvWakeupMode`	Sets the wakeup notification state for the specified transceiver.
`CanIf_CheckWakeup`	Checks if an underlying Can driver or transceiver has already signaled a wakeup.
`CanIf_CheckValidation`	Used for verifying the previously detected wakeup event.
`CanIf_ClearTrcvWufFlag`	Clears the WUF flag for the specified transceiver.
`CanIf_CheckTrcvWakeFlag`	Returns the WUF flag of the specified transceiver.
`CanIf_SetBaudrate`	Sets the baud rate configuration of the CAN controller.
`CanIf_GetControllerTxErrorCounter`	Calls the corresponding CAN Driver service for obtaining the Tx error counter of the CAN controller.
`CanIf_TriggerTransmit`	Upper layer (called) module checks whether the available data fits into the buffersize reported by PduInfoPtr > SduLength.
`CanIf_TxConfirmation`	Confirms the successfully-processed transmission of an L-PDU.
`CanIf_RxIndication`	Indicates successful reception of a received Can L-DPU after passing all filters and validation checks.
`CanIf_ControllerBusOff`	Indicates a controller bus off event for the specified controller.
`CanIf_ConfirmPnAvailability`	Indicates whether or not a transceiver is running in PN mode.
`CanIf_ClearTrcvWufFlagIndication`	Indicates that the transceiver has cleared the WufFlag referring to the corresponding CAN transceiver with the abstract CanIf TransceiverId.
`CanIf_CheckTrcvWakeFlagIndication`	Indicates that the check of the transceiver's wake-up flag has been finished by the corresponding CAN transceiver with the abstract CanIf TransceiverId.
`CanIf_ControllerModeIndication`	Indicates a controller state transition for the specified controller.
`CanIf_TrcvModeIndication`	Indicates a transceiver state transition for the specified transceiver.

`CanIf_GetTxConfirmationState`	Indicates if any TX confirmation has been done for the CAN controller since start.
`CanIf_EnableBusMirroring`	Enables or disables mirroring for a CAN controller.
`CanIf_GetCurrentTime`	Retrieves the current time value out of the HW registers.
`CanIf_EnableEgressTimeStamp`	Activates egress time-stamping on a dedicated message object.
`CanIf_GetEgressTimeStamp`	Reads back the egress time stamp on a dedicated message object.
`CanIf_GetIngressTimeStamp`	Reads back the ingress time stamp on a dedicated message object.
`CanIf_XLRxIndication`	Indicates a successful reception of a received CAN XL Rx L-PDU to the CanIf.
`CanIf_ControllerErrorStatePassive`	Callback that derives `ErrorCounterThreshold` from `RxErrorCounter/TxErrorCounter` values and reports it to IdsM as security event *SEV_CAN_ERRORSTATE_PASSIVE*.
`CanIf_ErrorNotification`	Callback that derives bus error source Rx or Tx from parameter `CanError` and reports it as security event *SEV_CAN_TX_ERROR_DETECTED* or *SEV_CAN_RX_ERROR_DETECTED*.

For Further information please refer to AUTOSAR_SWS_CANInterface.pdf

2.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

2.8.1 General Advice

- CanIfDispatchUserCtrlModeIndicationUL and/or CanIfDispatchUserCtrlModeIndicationName must always be configured.
- When an HRH is configured as BASIC_CAN (in CanIfHrhCfg), the CanIfHrhListCfg and CanIfHrhRangeCfg for the HRH must also be configured.
- When CanIfPublicWakeupCheckValidByNM is set to TRUE, at least one Rx L-PDU must be configured.
- If Transmit Buffering is enabled, although it supports both Interrupt and

Polling modes, the buffered messages should be configured as Interrupt messages in order to make the transmission of pending buffered messages as fast as possible.

- When integrating CanIf with CanDrv, for compliance with AUTOSAR 4.2.2, the CanId_t element (in CanIf_Config.HrhConfig_pcst) must be enabled in order to check against the CanId to obtain the Can_HwType.ControllerId value. For this reason, the CanIfReadMsgIdApi parameter (in CanIf/CanIfPublicCfg) must be set to TRUE.
- When two or more L-PDUs have the same CanId referring to the same controller, they must have a different CanIdType. And if two L-PDUs are connected to the same HRH, the CanIdType should not be STANDARD_CAN or EXTENDED_CAN.

2.8.2 Configuration for Rx multiplexing

A Can HOH can receive either a Range of CanIds or a List of CanIds.

1. Configure an HRH for the Rx-Pdu.
2. Then either…
    1. In the corresponding HRH, in the CanIfHrhListCfg container, configure it to receive a List of CanIds. You can add as many CanIds as required. Or…
    2. Configure the CanIfHrhRangeCfg container to configure the HRH to receive a Range of CanIds. The range can be specified by configuring LowerCanId and UpperCanId.
3. For the Basic Can HRH in CanIf, configure CanIfHrhSoftwareFilter as TRUE.
4. For the corresponding HOH in CanDrv, configure the CanHandleType as TRUE.

2.8.3 CanIf_Rb_ChangeRxPduCanId

This extension API can be called to reconfigure/change the Can Identifier (CanId) of the Receive CAN L-PDU (CanRxPduId). It copies the new CanId into the corresponding DynamicRxPduCanId array location of the requested CanRxPduId, and calls the rba_Can_ChangeRxCanId driver function to update the CanId of the corresponding receive Hardware object.

CanIf_Rb_ChangeRxPduCanId requires CanIfRxPduType to be configured as DYNAMIC, and CanIfRbChangeRxPduIdApi to be set to TRUE. It returns E_OK if the new CanId is updated successfully by the Can driver.

When CanIf_Rb_ChangeRxPduCanId is called, you should set the two MSB bits of the CanId (IDentifier Extension flag and CAN FD flag) based on the CanIfRxPduCanIdType of the CanRxPduId.

CanIf_Rb_ChangeRxPduCanId can only be called during ECU initialization, and it return E_NOT_OK if called during runtime.

2.8.4 CanIf Calibration

1. To enable the optional CanIf Calibration feature, configure CanIfRbCalibration (in CanIfPublicCfg/CanIfRbCalibrationSupport) as TRUE.
2. As CanIf has a dependency on CanDrv, calibration in CanIf does not work without also enabling calibration support in CanDrv (via CanRbCalibrationSupport).
3. This generates CANIF_RB_CALIBRATION as STD_ON.
4. The names of the calibration parameters are configurable - a prefix and suffix parameter is provided in the CanDrv paramdef to configure the names of each calibratable parameter.
5. Update the <Module>_BSWMD.arxml file with the static parameters (CanIds and their DLCs) under the "PrintConstantMemorysSection" section. Configure the CompuMethod, ImplementationDataType etc. for the calibratable and measurable parameters.
6. Two ARXML files are generated, one for the calibratable parameters, and one for the measurable parameters. Using these files, RTEGen generates the CanIds and DLCs as Macros in SchM_Can.h. These are memory mapped (Refer to MemMap.h).
7. In CanIf_PBcfg.c, in the Calibratable section (and other places where CanIds/DLCs are being used) are accessible via pointers to the location where they are actually defined. This may require changes in the datatypes being used for the calibratable parameters (i.e. via pointers).

Calibration parameters for CAN Node assignment for a CAN message will be provided by CanDrv module. CanIf will use the calibration element generated by can driver for supporting node calibration.

If CanIfMetaDataSupport (in CanIfPublicCfg) is set to True, the CanId calibration parameter will not be used for Transmission when the TxPdu is configured with a MetadataLength which is greater than or equal to the CanId size.

See the Supported Features and Limitations sections for more details on CanIf Calibration.

2.8.5 CAN FD (CAN with Flexible Data Rate)

The CAN FD feature is enabled via CanControllerFdBaudrateConfig in the Can Driver configuration.

The length of transmit and receive frames can be in the range of 0 to 64 bytes. Recommended values for DLC length are: 0 to 8, 12, 16, 20, 24, 32, 48, or 64 bytes.

The receive frames can be configured individually, you can decide which frames can be configured as FD frames or non-FD frames using the CanIfRxPduCanIdType parameter (in CanIf/CanIfInitCfg/CanIfRxPduCfg).

The transmit frames sent to the Can driver are configured on a per PDU basis,

using the CanIfTxPduCanIdType parameter (in CanIf/CanIfInitCfg/CanIfTxPduCfg).

Via the second most significant bit of the L-PDU indentifier passed to Can_Write, CanIf indicates to the Can module whether an L-SDU is transmitted as an FD frame or a conventional frame. A similar mechanism is used when CanIf_RxIndication is called, indicating that a received L-PDU should be treated as an FD frame.

See the Supported Features section for more information on CAN FD.

2.8.6 MetaData Feature

The MetaData feature is enabled by setting CanIfMetaDataSupport (in CanIfPublicCfg) to TRUE.

The MetaData feature uses the MetaDataLength parameter, which is referred from the Pdu in the EcuC container. The maximum value of the MetaDataLength is 4.

If CanIfMetaDataSupport is enabled, the CanId Calibration parameter will not be used for transmission when TxPdu is configured with a MetadataLength greater than or equal to the CanId size.

CanIfRxPduCanIdMask (in CanIfRxPduCfg) is used to filter incoming messages during reception, and CanIfTxPduCanIdMask (in CanIfTxPduCfg) is used for calculating CanId during transmission.

2.8.7 Multiple Can Driver support

1. RTA-BSW ConfGen does not support the generation of multiple Can Drivers for both the CanIf and Can Driver modules. However, ConfGen can be used to generate the configuration for most of the mailboxes, controllers, Pdus and PduRouting.
2. To enable this feature, the CanIfPublicMultipleDrvSupport parameter must be set to TRUE. By default, ConfGen will set this parameter to FALSE, but this can be overridden in the algo.properties file.
3. You can then configure each Can Driver, which must be configured on top of a separate paramdef.

Note: For each Can Driver that you configure:

- A unique `Can/CanGeneral/CanIndex` must be provided.
- `Can/CanConfigSet/CanController/CanControllerId` must be started from 0.
- `Can/CanConfigSet/CanHardwareObject/CanHwObjectCount` must be started from 0.

4. For each Can Driver, a corresponding `CanIf/CanIfCtrlDrvCfg` and `CanIf/CanIfInitCfg/CanIfInitHohCfg` must be configured.
5. And for each Can Driver that requires a VendorID and a Vendor Specific API name as specified in [SWS_Can_00284], the relevant BswImplementation (usually in the BSWMD arxml file) must be configured with following values:

- VendorId
- VendorApiInfix
- VendorSpecificModuleDefs: reference to relevant Can Driver paramdef instance

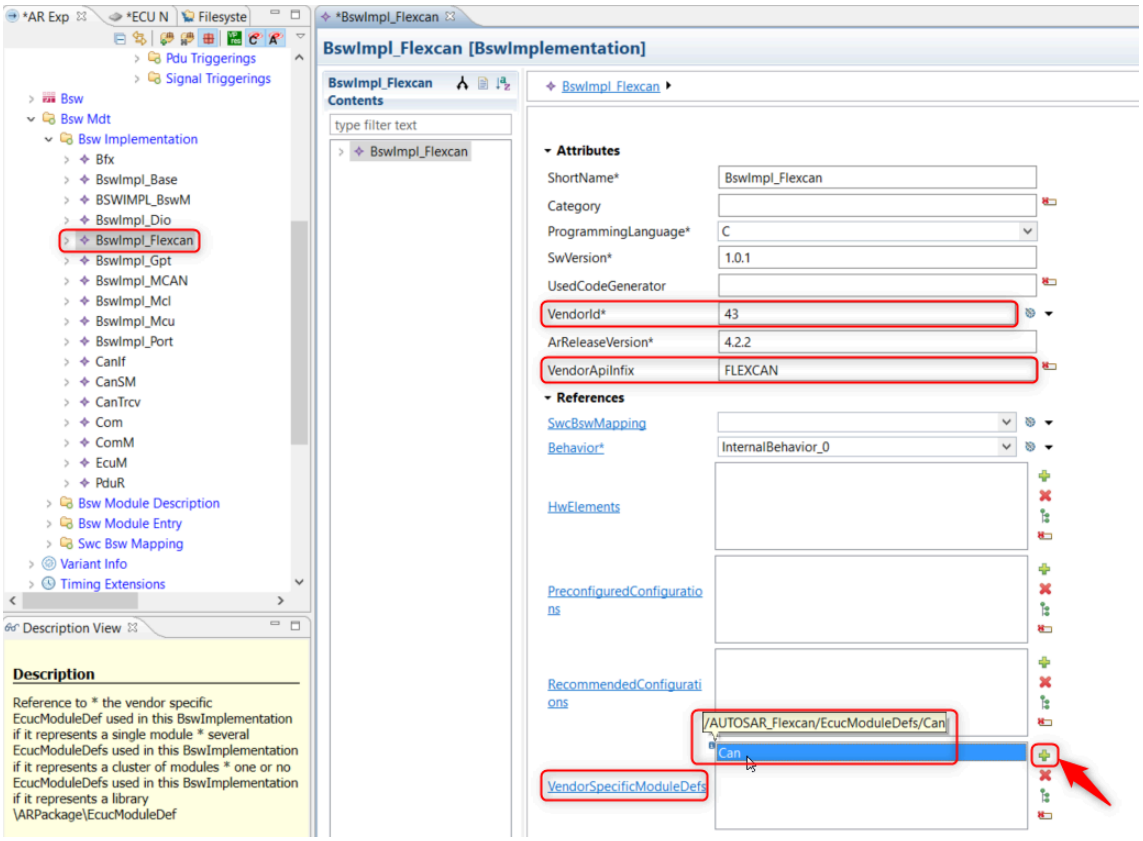


Figure 2.1.Vendor Specific Configuration

This configuration gives RTA-BSW CodeGen (Can Driver interfaces, macros, header file inclusion for CanIf) and RTEGen (Can Driver exclusive area name, Main Function name) the Vendor Specific configuration for generating code.

Note: If the VendorID and Vendor Specific API name are not required by one of the Can Drivers, do not configure VendorSpecificModuleDefs.

6. The required integration functions below:

   - `Can_Integn_<VendorName>_SetControllerMode()`
   - `Can_Integn_<VendorName>_GetControllerMode()`
   - `Can_Integn_<VendorName>_Write()`

need to be implemented in CanIf integration code, in CanIf/integration/CanIf_Integn.c and CanIf/integration/CanIf_Integn.h

Sample implementation for those functions can be found in `Can_Integn_Write()` and `Can_Integn_SetControllerMode()` in the same files.

Note: These required functions are specified as function pointers in CanIf_CtrlDrvCfgStruct CanIf_DrvCtrlGen, CanIf_PBcfg.c generated source file.

2.8.8 Bus Mirroring

The User can enable Bus Mirroring in the **ECU Navigator** by double-clicking **‹Project Name› › Bsw Modules › Com Stack › Can Modules › CanIf** to open the **BSW Editor**.

Then select the container **CanIf › CanIfPublicCfg** and, in the **Attributes** section, set **CanIfBusMirroringSupport** to *TRUE*:

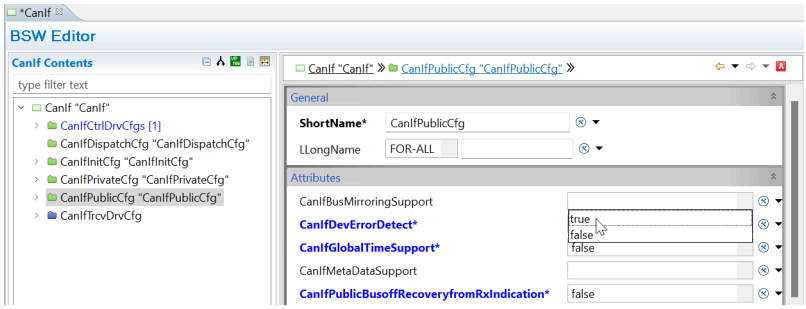


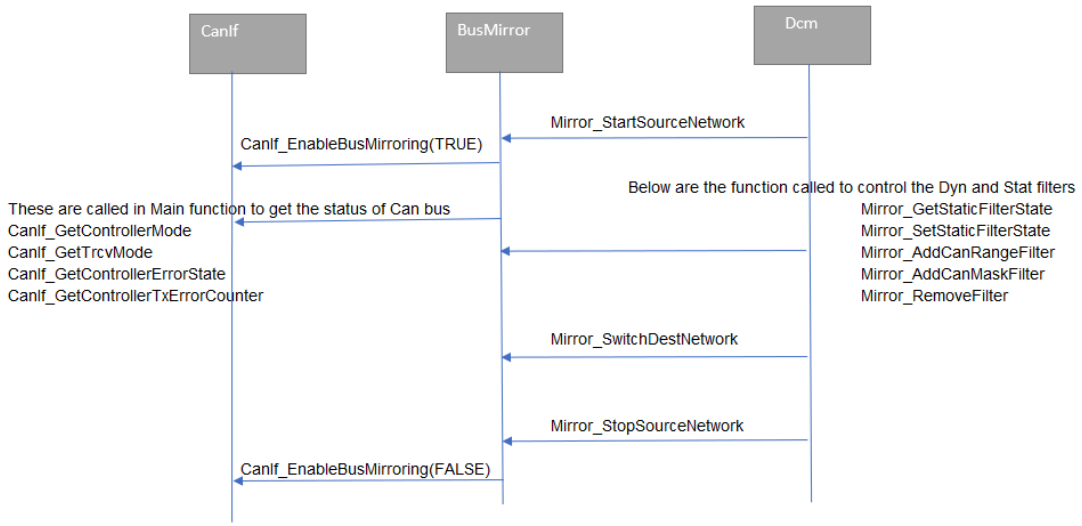
Figure 2.2.Set Bus Mirroring Support

When `Mirror_StartSourceNetwork` is called to start a CAN source bus, the Bus Mirroring module shall call `CanIf_EnableBusMirroring`, with `MirroringActive` set to *TRUE* to start reporting of received and transmitted CAN frames from the corresponding CAN controller.

`Mirror_StartSourceNetwork` receives a *ComMChannelId* as `network`, while `CanIf_EnableBusMirroring` expects a `CanIfCtrlId` as `ControllerId`. The translation of one to the other can be determined at generation time by following the references from the `ComMChannelId` to the `CanIfCtrlId` through the ECU configuration.

When `Mirror_StopSourceNetwork` is called to stop a CAN source bus, the Bus Mirroring module shall call `CanIf_EnableBusMirroring`, with `MirroringActive` set to *FALSE* to stop reporting of received and transmitted CAN frames from the corresponding CAN controller.

When mirroring, the source bus is identified by a *network ID*, but `Mirror_ReportCanFrame` reports the `controllerId`. The translation of the one to the other can be determined at generation time by following the references from the `CanIfCtrlId` to the `MirrorNetworkId` through the ECU configuration via `MirrorComMNetworkHandleRef`.



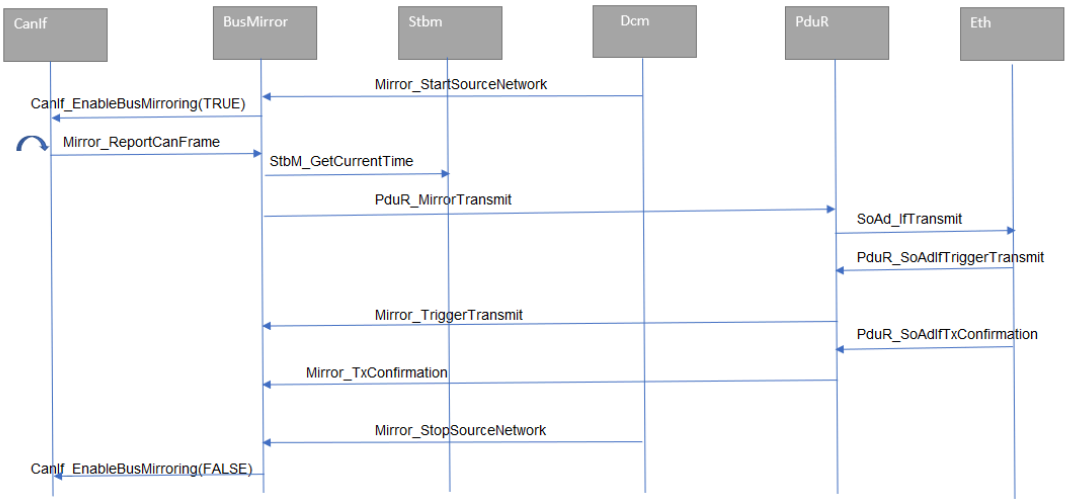


Figure 2.3.Set Bus Mirroring Support

Please see the RTA-Com Stack Reference Guide for more details on Bus Mirroring.

2.8.9 CanXL

RTA-BSW ConfGen supports the generation of the CanXL configuration. The `performMailboxMapping` flag must be set to *false* in order to run this feature:

- manprop_performMailboxMapping=false

Because of the ConfGen limitation for CanXL, the following workaround steps are required on top of the generated output:

- There are 2 `Can/CanConfigSet/CanController/CanControllerBaudrateConfig` containers generated for the CanXL CanController. Remove the 2nd duplicated one.
- There are 2 `Nm/NmChannelConfig` containers generated for the CanXL channel. Remove the 2nd duplicated one.
- There are 2 `CanNm/CanNmGlobalConfig/CanNmChannelConfig` containers generated for the CanXL channel. Remove the 2nd duplicated one.
- The `Nm/NmGlobalConfig/NmGlobalConstants/NmNumberOfChannels` value needs to be reduced by 1.
- If all of the Eth messages are expected to be sent/received by CanXL, and the project does not expect to have Eth and EthTrcv modules generated by ConfGen, then remove the Eth and EthTrcv module configurations that have been generated.

2.9 Integration Advice

2.9.1 Init Functions

The EcuM module calls CanIf_Init. During the initialization process, all global variables and data structures are initialized, including flags and buffers. After the initialization process has been completed, CanIf expects that the CAN controller remains in STOPPED mode (as after a power-on reset). In this mode, CanIf and CanDrv are neither able to transmit nor receive CAN L-PDUs.

EcuM executes the initialization of CanDrvs and CanTrcvs separately by calling their corresponding initialization services.

If a full re-initialization of all CAN modules is required during runtime, EcuM will invoke CanSm to initiate the required state transitions of the CAN controller via a call to CanIf_SetControllerMode. CanIf maps these calls from CanSm to calls of the respective CanDrvs.

CanIf_Init must be called after Calibrating node assignment to the HTH/HRHs, otherwise some features (e.g. Partial Networking) will not work correctly.

2.9.2 DeInit Functions

CanIf_DeInit is required to de-initialise all the RAM areas of the previously active configset. Note that this function is used only when Post-build is enabled.

2.9.3 Scheduled Functions

The CanIf module has no periodic functions that require scheduling.

2.9.4 Integration with 3rd Party Can Driver

In RTA-BSW v12.7.0, all Can drivers are considered to be “third party”.

CanIf supports third party Can drivers via the integration files `CanIf_Integn.c` and `CanIf_Integn.h`. These files must be modified to implement an interface between the Can drivers in the MCAL and CanIf.

The following parameters must be configured in order to enable the `CanIf_Integn.c` functionality:

- CanIf/CanIfCtrlDrvCfg/CanIfRbForeignDrvSupport = TRUE

2.9.5 API Selection

The API used for the MCAL is not configurable and the user will need to write their own code if targeting a different version. The default is the AR v22-11 API but other API versions can be used by updating integration files as below:

The following CanIf callback functions in `CanIf_Integn.c` will require modifications to translate calls to the AR v22-11 API used by CanIf:

- The function `CanIf_RxIndication` (API changed from 4.1.x)
- The function `CanIf_ControllerModeIndication` (API changed from 4.3.x)

The implementation of the following functions in `CanIf_Integn.c` are stubs and need to be updated to translate calls to older AUTOSAR APIs if required:

- Calls to `Can_Write` are redirected via `Can_Integn_Write`
- Calls to `Can_SetControllerMode` are redirected via `Can_Integn_SetControllerMode`
- Calls to `Can_GetControllerMode` are redirected via `Can_Integn_GetControllerMode` This Can API function does not exist in older AUTOSAR versions < = 4.2.x so calls always return `E_NOT_OK`

`Can_GeneralTypes.h` will need to be updated to include the following definitions as per older AUTOSAR versions:

- define `Can_StateTransitionType`
- define `CanIf_ControllerModeType`
- define internal controller mode state definitions `CANIF_CS_UNINIT`, `CANIF_CS_SLEEP`, `CANIF_CS_STARTED` and `CANIF_CS_STOPPED`

2.9.6 CanIdValue

As it is no longer part of the AUTOSAR standard, RTA-BSW ConfGen does not generate the CanIdValue MCAL configuration parameter, however some MCALs using API version 4.0.x may still require it. For this reason, if API version 4.0.x is used, it may be necessary to manually configure CanIdValue for each receiving CanHardwareObject.

After running ConfGen to generate the other parameters, open “Parameter Definitions/Can/CanConfigSet/CanHardwareObject” in the ECU Navigator, select **New Child** > **Parameter** > **Parameters | Ecuc Integer Param Def** and set the new parameter’s ShortName to CanIdValue. Then, again using ECU Navigator, find and open each CanHardwareObject in “Bsw Modules/Com Stack/Can Modules/Can/-CanConfigSet” and set an appropriate CanIdValue (e.g. by copying the CanObjectId assigned by ConfGen).

2.9.7 Transceiver Integration

- If using hardwired CAN transceivers to detect wakeup/validation, the CanTrcvChannels parameter must be configured for each transceiver.
- The configuration data structure under CanIf/CanIfTrcvDrvCfg/CanIfTrcvCfg must contain a unique reference to a channel ID configured in the CanTrcv module (CanIfTrcvCanTrcvRef).
- When wakeup/validation is enabled, CanIfCtrlTrcvRef for all CanIfCtrlCfg must be configured. Ensure that each CanIfCtrlCfg is referred to the same CAN transceiver as in CanSM.
- If a hardwired transceiver is required to only validate a wakeup, OR detect-and-validate a wakeup, configure the CanTrcWakeupSourceRef and CanIfCtrlTrcvRef parameters. The CanIfPublicWakeupCheckValidSupport parameter should be set to TRUE to enable wakeup validation, and CanIfTrcvWakeupSupport set to TRUE for the corresponding transceiver to enable wakeup detection.
- When CAN Transceivers are configured, the CanIfDispatchUserTrcvModeIndicationUL and/or CanIfDispatchUserTrcvModeIndicationName parameters must be configured, the Can Transceiver driver should call the CanIf_TrcvModeIndication callback function, and the Can controller driver should call the CanIf_ControllerModeIndication callback function.
- When CanIfTrcvWakeupSupport is set to TRUE, the CanIfDispatchUserValidateWakeupEventUL and/or CanIfDispatchUserValidateWakeupEventName must be configured.
- If CanIfPublicPnSupport is set to TRUE, ensure that the Can Transceiver driver calls the CanIf_ClearTrcvWufFlagIndication, CanIf_CheckTrcvWakeFlagIndication and CanIf_ConfirmPnAvailability callback functions.

2.9.8 PduIdTypeEnum (EcuC)

PduIdTypeEnum (in EcuC/EcucPduCollection/PduIdTypeEnum) should be set correctly as either UINT8 or UINT16. The size of this global type depends on the maximum number of PDUs used within one software module. If no software module deals with more than 256 PDUs, this type can be set to uint8. If at least one software module handles more than 256 PDUs, this type must be set to uint16.

2.9.9 Can_IdType Type Support

RTA-BSW only supports the Can_IdType of uint32, as per v22-11 [SWS_Can_00416].

2.9.10 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to

disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The CanIf module uses the following OS resources:

- CanIf_TxBufAccessNoNest
- CanIf_RxBufAccessNoNest
- CanIf_ControllerState
- CanIf_NewMsgRxWkpEvtNoNest
- CanIf_TxRxNotifNoNest
- CanIf_DynIdNoNest
- CanIf_PNPduStatusNoNest
- CanIf_TxMuxing
- CanIf_TxMirrorBufAccess
- CanIf_TxBufAccessNoNest
- CanIf_RxBufAccessNoNest
- CanIf_ControllerState
- CanIf_NewMsgRxWkpEvtNoNest
- CanIf_TxRxNotifNoNest
- CanIf_DynIdNoNest
- CanIf_PNPduStatusNoNest
- CanIf_TxMuxing
- CanIf_TxMirrorBufAccess

2.10 Hardware Requirements

Table 2.85.CanIf Hardware Requirements

Support	Usage
FLASH	17895 bytes
RAM	2820 bytes

Usage calculated based on a sample configuration.

3 CAN Network Management

Module name	CanNm
Package	RTA-CAN
AUTOSAR Module ID	31
AUTOSAR Version	22-11

3.1 Introduction

The CAN Network Management module (CanNm) is located inside the Communication Services layer. It is a hardware independent protocol that can only be used on CAN networks. The diagram in the Introduction to this Reference Guide shows the position of the module in the AUTOSAR layered architecture.

CanNm provides an adaptation between the Network Management Interface module (Nm) and the CAN Interface module (CanIf).

The purpose of the CanNm is:

1. To coordinate the transitions on the network between Normal Operation and Bus Sleep mode.
2. To provide an Rx PDU Filtering algorithm (the core functionality required to support Partial Networking).
3. In addition to the core functionality, several configurable features are provided, including a service to detect all present nodes and a service to detect if all other nodes are ready to sleep.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

3.2 Supported Features

3.2.1 Operational Modes

CanNm supports three operational modes:

- Network Mode
- Prepare Bus-Sleep Mode
- Bus-Sleep Mode

3.2.2 Network States

CanNm supports the following Network States:

- Requested
- Released

3.2.3 CanNm State Machine Handling

The CanNm State Machine is used to coordinate transitions between Normal Operation and the Bus Sleep state. This mechanism is used to reduce the ECU's power consumption.

If a node wants to keep the network awake, it will broadcast Network Management (NM) messages periodically to all other nodes.

When Bus communication is released and there are no NM PDUs received for a given period of time, the CanNm State Machine will begin the transition to Bus Sleep mode.

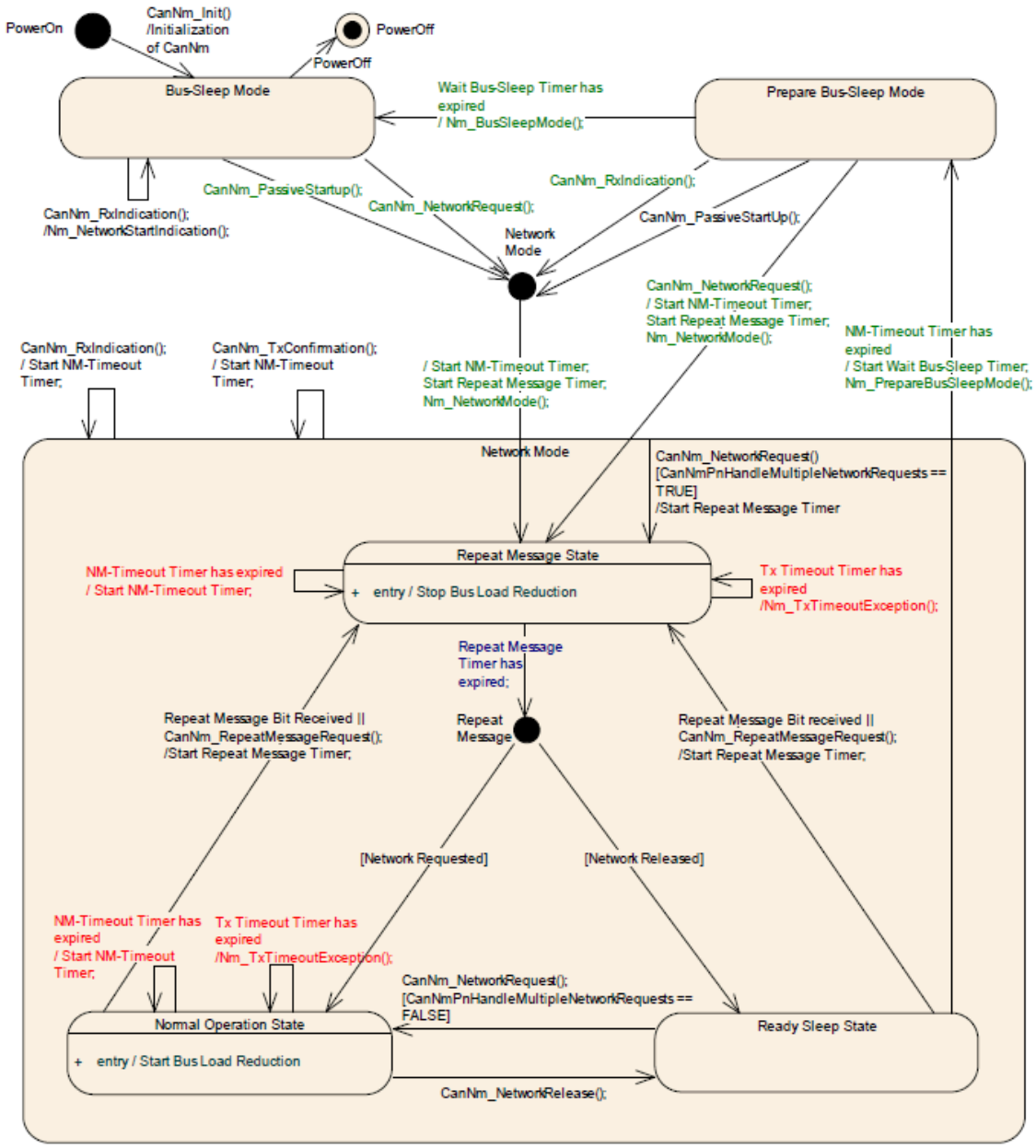


Figure 3.1.CanNm State Transition Diagram

3.2.4 CanNm States

CanNm can enter the following states:

State	Numerical value	Description
NM_STATE_UNINIT	0	Uninitialized State
NM_STATE_BUS_SLEEP	1	Bus Sleep State
NM_STATE_PREPARE_BUS_SLEEP	2	Prepare-Bus State

NM_STATE_READY_SLEEP	3	Ready Sleep State
NM_STATE_NORMAL_OPERATION	4	Normal Operation State
NM_STATE_REPEAT_MESSAGE	5	Repeat Message State
NM_STATE_SYNCHRONIZE	6	Synchronize State

NOTE

NM_STATE_UNINIT is the default state of CanNm, and it lasts until CanNm initialization is complete.

NOTE

When CanNm enters Network Mode, the state is NM_STATE_REPEAT_MESSAGE by default.

3.2.5 Transmit and Receive NM Messages

When in Network Mode and in the state of NM_STATE_REPEAT_MESSAGE, CanNm will start/re-start the transmission of Network Management PDUs. These messages are used to inform other nodes in the network that the ECU's communication is still required in order to keep the CAN bus awake.

- The Transmission of NM PDUs is triggered by a call to CanIf_Transmit from CanNm_MainFunctionTx.
- When a NM message is received, CanNm_RxIndication is called by CanIf_RxIndication to process the received PDU.
- Whenever a NM message is received or transmitted in Network Mode, the Nm-Timeout timer is restarted.

3.2.6 Partial Networking

In order to reduce the power consumption used for CAN communication, CanNm supports the Partial Networking feature to switch off the communication stack of single CAN ECUs during active bus communication. This support includes the NM PDU Filtering algorithm, which drops all received NM PDUs that are not relevant for the ECU. See the Configuration Advice for more information.

3.2.7 Spontaneous Transmission of NM PDUs

CanNm supports the spontaneous transmission of NM PDUs. See the Configuration Advice for more details.

3.2.8 Remote Sleep Indication

Remote Sleep Indication signals a situation where a node detects that all other nodes are ready to sleep, but the node where the indication occurs is still keeping the bus awake. CanNm will notify Nm (through a call to Nm_RemoteSleepIndica-

tion) that all nodes in the cluster are ready to sleep.

3.2.9 COM User Data

CanNm can facilitate transmission of NM User Data passed by the SWC via COM. It calls PduR_CanNmTriggerTransmit to collect the User Data from the referenced NM I-PDU, which it combines with other NM PDU's bytes before requesting the transmission of the NM PDU. See the Configuration Advice for details of validation errors/warnings that are generated during the configuration for Com User Data.

3.3 Extensions

3.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.1.1 CanNm/CanNmGlobalConfig/CanNmChannelConfig/CanNmPnEraCalcEnabled

Short Name	CanNmPnEraCalcEnabled
Description	Specifies if CanNm calculates the PN request information for external requests. (ERA)
Introduction	false: PN request are not calculated true: PN request are calculated
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.2 CanNm/CanNmGlobalConfig/CanNmChannelConfig/CanNmPnEraRxNSduRef

Short Name	CanNmPnEraRxNSduRef
Description	Reference to a Pdu in the COM-Stack. The SduRef is required for every CanNm Channel, because ERA is reported per channel.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.3 CanNm/CanNmGlobalConfig/CanNmPnEiraCalcEnabled

Short Name	CanNmPnEiraCalcEnabled
Description	Specifies if CanNm calculates the PN request information for internal an external requests. (EIRA)
Introduction	true: PN request are calculated false: PN request are not calculated
Multiplicity	0..1

Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.4 CanNm/CanNmGlobalConfig/CanNmPnEiraRxNSduRef

Short Name	CanNmPnEiraRxNSduRef
Description	Reference to a Pdu in the COM-Stack.
Introduction	Only one SduRef is required for CanNm because the EIRA is the aggregation over all Can Channels.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.5 CanNm/CanNmGlobalConfig/CanNmPnInfo

Short Name	CanNmPnInfo
Description	PN information configuration
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.6 CanNm/CanNmGlobalConfig/CanNmPnInfo/CanNmMaxNumberOf-PnsConfigurableAtPostBuildTime

Short Name	CanNmMaxNumberOfPnsConfigurableAtPostBuildTime
Description	Specifies the maximum number of the PNs can be configured during post build time.This parameter should be configured during pre compile time itself.
Multiplicity	0..1
Value Range	0..56
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.7 CanNm/CanNmGlobalConfig/CanNmPnInfo/CanNmPnFilter-MaskByte

Short Name	CanNmPnFilterMaskByte
Description	PN information configuration
Multiplicity	0..7
Default Value	-

Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.8 CanNm/CanNmGlobalConfig/CanNmPnInfo/CanNmPnFilterMaskByte/CanNmPnFilterMaskByteIndex

Short Name	CanNmPnFilterMaskByteIndex
Description	Index of the filter mask byte. Specifies the position within the filter mask byte array.
Multiplicity	1..1
Value Range	0..6
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.9 CanNm/CanNmGlobalConfig/CanNmPnInfo/CanNmPnFilterMaskByte/CanNmPnFilterMaskByteValue

Short Name	CanNmPnFilterMaskByteValue
Description	Parameter to configure the filter mask byte.
Multiplicity	1..1
Value Range	0..255
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.10 CanNm/CanNmGlobalConfig/CanNmPnInfo/CanNmPnInfoLength

Short Name	CanNmPnInfoLength
Description	Specifies the length of the PN request information in the NM PDU.
Multiplicity	1..1
Value Range	1..7
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.11 CanNm/CanNmGlobalConfig/CanNmPnInfo/CanNmPnInfoOffset

Short Name	CanNmPnInfoOffset
Description	Specifies the offset of the PN request information in the NM PDU.
Multiplicity	1..1
Value Range	1..7
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.12 CanNm/CanNmGlobalConfig/CanNmPnResetTime

Short Name	CanNmPnResetTime
Description	Specifies the runtime of the reset timer in seconds. This reset time is valid for the reset of PN requests in the EIRA and in the ERA. The value shall be the same for every channel. Thus it is a global config parameter.
Multiplicity	0..1
Value Range	0.001..65.535
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

3.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.2.1 CanNm_SetPnState

Syntax	`void CanNm_SetPnState( const NetworkHandleType nmChannelHandle, boolean state )`
Parameters	`nmChannelHandle` Identification of the NM-channel `state` state of PN Enable sent through test application.
Description	Sets the PN state in the configuration structure.

3.3.2.2 CanNm_SetAllMessageKeepAwake

Syntax	`void CanNm_SetAllMessageKeepAwake(const NetworkHandleType nmChannelHandle,boolean state)`
Parameters	`nmChannelHandle` Identification of the NM-channel `state` state of PN Enable sent through test application.
Description	Sets AllNmMessagesKeepAwake_b in the configuration structure.

3.3.2.3 CanNm_SetPnHandleMultipleNetworkReqState

Syntax	`void CanNm_SetPnHandleMultipleNetworkReqState(const NetworkHandleType nmChannelHandle,boolean state)`
Parameters	`nmChannelHandle` Identification of the NM-channel `state` state of PN Enable sent through test application.
Description	Sets the multiple network request in the configuration structure.

3.3.2.4 CanNm_RequestSynchronizedPncShutdown

Syntax	`void CanNm_RequestSynchronizedPncShutdown ( NetworkHandleType nmChannelHandle, PNCHandleType pncId)`
Parameters(In)	`nmChannelHandle` Identifier of the NM-Channel where the given PNC (pncId) is assigned to. `pncId` Identifier of the PNC which is requested for a synchronized shutdown across the PN topology.
Description	Requests transmission of a NM-PDU with PNSR bit set to 1 (PN shutdown message).

3.4 Limitations

The following features are not supported for version v12.7.0:

- Post Build Selectable is not supported for the CanNmTxConfirmationPduId and CanNmRxPduId parameters (bug in Autosar).

3.4.1 Incorrect generation of CanNm ERA enabled channel offset in CanNmChannelConfig when CanNmPnEraCalcEnabled is true

WARNING

When CanNmPnEraCalcEnabled is set to true, there is a risk of a critical issue causing an array index overflow (See Defect-633519 in the Release Notes) that can lead to an unexpected memory overwrite. Please see below for a workaround.

Workaround:

Use a linear mapping between CanNm-ComM channels: configure all ComM-Channels that are referred to by CanNmComMNetworkHandleRef to be the first ComMChannels (i.e., re-order the ComMChannels so that the one referred by CanNm have ComMChannelIds as 0-N, where N is the number of CanNM channels).

3.4.1.1 Example of linear CanNm-ComM channel mapping:

Linear mapping: NetworkHandle in Configuration structure of CanNm channels (CanNm_ChannelConfigType) should be consecutive numbers, starting from 0 till the number of channels in CanNm.

CanNm channel	ID	ComM channel referred in CanNm	ID
CanNm_ChannelA	0	ComM_ChannelA	0
CanNm_ChannelB	1	ComM_ChannelB	1
CanNm_ChannelC	2	ComM_ChannelC	2

3.4.1.2 Example of non-linear CanNm-ComM channel mapping:

Non-Linear: NetworkHandle in Configuration structure of CanNm channels (CanNm_ChannelConfigType) are not consecutive numbers starting from 0.

CanNm channel	ID	ComM channel referred in CanNm	ID
CanNm_ChannelA	0	ComM_ChannelA	0
CanNm_ChannelB	1	ComM_ChannelB	2
CanNm_ChannelC	2	ComM_ChannelC	3

3.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `CanNm_DeInit`
- `CanNm_PnLearningRequest`
- `CanNm_ActivateTxPnShutdownMsg`
- `CanNm_DeactivateTxPnShutdownMsg`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- CanNm/CanNmGlobalConfig/CanNmChannelConfig/CanNmDynamicPncToChannelMappingEnabled
- CanNm/CanNmGlobalConfig/CanNmChannelConfig/CanNmStayInPbsEnabled
- CanNm/CanNmGlobalConfig/CanNmDynamicPncToChannelMappingSupport

3.6 Deviations

3.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

3.6.1.1 CanNm/CanNmGlobalConfig/CanNmChannelConfig/CanNmMsgTimeoutTime

[RTA-BSW] Description	Transmission Timeout of NM PDU. If there is no transmission confirmation by the CAN Interface within this timeout, the CANNM module shall give an error notification.

[AUTOSAR] Description	When using Partial Network and this timeout is defined then CanNm monitors that a NM-PDU is transmitted successfully within this Transmission Timeout Time and provides an error notification otherwise.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

3.6.1.2 CanNm/CanNmGlobalConfig/CanNmChannelConfig/CanNmSynchronizedPncShutdownEnabled

[RTA-BSW] Description	Specifies if CanNm handle PN shutdown messages to support a synchronized PNC shutdown across a PN topology. This is only used for ECUs in the role of a top-level PNC coordinator or intermediate PNC coordinator. Thus, the PNC gateway functionality is enabled and therefore ERA calculation is used. Tags: atp.Status=draft
[AUTOSAR] Description	Specifies if CanNm handle PN shutdown messages to support a synchronized PNC shutdown across a PN topology. This is only used for ECUs in the role of a top-level PNC coordinator or intermediate PNC coordinator. Thus, the PNC gateway functionality is enabled and therefore ERA calculation is used.

3.6.1.3 CanNm/CanNmGlobalConfig/CanNmChannelConfig/CanNmWaitBusSleepTime

[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

3.7 API Definition

List of supported API functions

`CanNm_Init()`	Initializes the CanNm module (all channels activated at configuration time are initialized).
`CanNm_CheckRemoteSleepIndication()`	Checks if remote sleep indication takes place or not.
`CanNm_ConfirmPnAvailability()`	Enables the PN filter functionality on the indicated NM channel.
`CanNm_DisableCommunication()`	Disables the NM PDU transmission ability due to a ISO14229 Communication Control (28hex) service.
`CanNm_EnableCommunication()`	Enables the NM PDU transmission ability due to a ISO14229 Communication Control (28hex) service.

`CanNm_GetLocalNodeIdentifier()`	Gets the node identifier configured for the local node.
`CanNm_GetNodeIdentifier()`	Gets the node identifier out of the most recently received NM PDU.
`CanNm_GetPduData()`	Gets the whole PDU data out of the most recently received NM message.
`CanNm_GetState()`	Returns the state and the mode of the network management.
`CanNm_GetUserData()`	Gets user data from the most recently received NM message.
`CanNm_GetVersionInfo()`	Gets the version information of the CanNm module.
`CanNm_NetworkRelease()`	Releases the network as the ECU doesn't have to communicate on the bus.
`CanNm_NetworkRequest()`	Requests the network as the ECU needs to communicate on the bus.
`CanNm_PassiveStartUp()`	Triggers the transition from Bus Sleep Mode or Prepare Bus Sleep Mode to the Network Mode in Repeat Message State.
`CanNm_RepeatMessageRequest()`	Sets the Repeat Message Request Bit for NM messages transmitted next on the bus.
`CanNm_RequestBusSynchronization()`	Requests bus synchronization.
`CanNm_RxIndication()`	Indicates a successful reception of a received NM message to the UdpNm after passing all filters and validation checks.
`CanNm_SetSleepReadyBit ()`	Called by Nm to set the NM SleepReady bit in Control Bit Vector for messages transmitted next on the bus.
`CanNm_SetUserData()`	Sets user data for all NM messages transmitted on the bus after this function has returned without error.
`CanNm_Transmit()`	Implemented as an empty function and always returns E_NOT_OK. Only available if the configuration switch CanNmComUserDataSupport is enabled.
`CanNm_TxConfirmation()`	Confirms a previous successfully processed transmit request.
`CanNm_TriggerTransmit()`	Upper layer module shall check whether the available data fits into the buffer size reported by PduInfoPtr→SduLength
`CanNm_MainFunction()`	Main function of the CanNm which processes the algorithm describes in that document

For Further information please refer to AUTOSAR_SWS_CANNetworkManagement.pdf

3.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

3.8.1 Configuration of NmConfig

The RTA-BSW Configuration Generation (ConfGen) supports generation for all basic configurations in `CanNmGlobalConfig` and `CanNmChannelConfig`. The input for this generation is the `NmConfig` in the System Template as shown below.

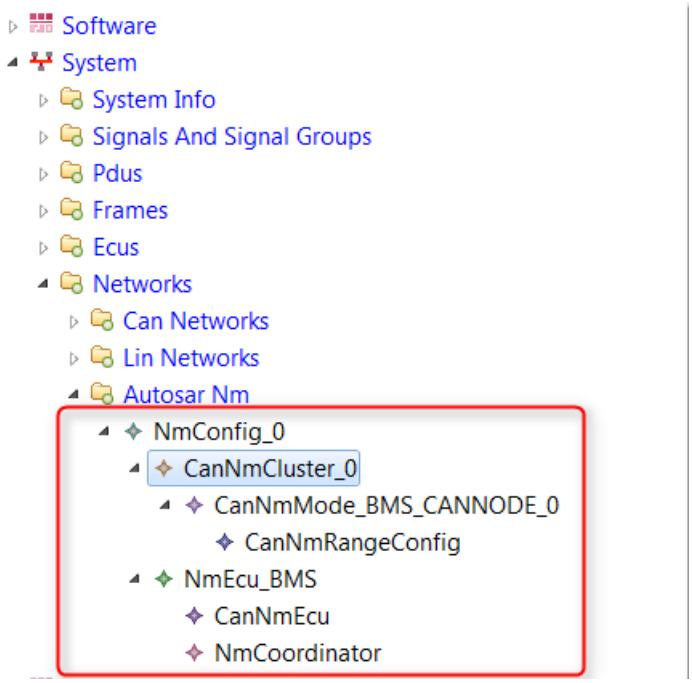


Figure 3.2.NmConfig SystemTemplate

However, the attributes in `NmConfig` are not automatically generated by the DBC importer script provided in ISOLAR-AB. Therefore, in order to generate a full CanNm module configuration using RTA-BSW Configuration Generation, `NmConfig` must be configured manually.

3.8.2 Partial Networking

CanNm supports the Partial Networking feature, with the following considerations for Rx and Tx handling of Nm PDUs.

3.8.2.1 Rx Handling of NM PDUs

If CanNmPnEnabled is FALSE, CanNm will perform the normal Rx Indication handling and the Partial Networking extensions will be disabled.

If CanNmPnEnabled is TRUE, and if the PNI bit in the received NM PDU is 0, and if CanNmAllNmMessagesKeepAwake is TRUE, CanNm will perform the normal Rx Indication handling, omitting the extensions for Partial Networking.

If CanNmPnEnabled is TRUE, and if the PNI bit in the received NM PDU is 0, and if CanNmAllNmMessagesKeepAwake is FALSE, CanNm will ignore the received NM PDU.

If CanNmPnEnabled is TRUE, and if the PNI bit in the received NM PDU is 1, CanNm will process the Partial Networking Information of the NM PDU as per the AUTOSAR specification.

3.8.2.2 Tx Handling of Nm PDUs

If CanNmPnEnabled is TRUE, CanNm will set the value of the transmitted PNI bit to 1, otherwise the value of the transmitted PNI bit will always be set to 0.

3.8.3 Rx PDU Filtering

Rx PDU Filtering can be overridden by setting CanNmAllNmMessagesKeepAwake to TRUE, which will enable the Gateway to stay awake on any kind of NM PDU. CanNm will not apply Rx PDU filtering and will process the received NM PDU as normal.

If Rx PDU Filtering is to be used, the following two containers provide the core of the Rx Filter Algorithm:

1. CanNmPnInfo: This container is used to configure the position of the PN Info in the NM PDU. CanNmPnInfoLength specifies the length of the PN request information in the NM message, and CanNmPnInfoOffset specifies the offset.
2. CanNmPnFilterMaskByte: This container is used to configure individual bytes of the filter mask. It has an upper multiplicity of 7 and there should be as many containers as the value configured for CanNmPnInfoLength. CanNmPnFilterMaskByteIndex specifies the offset of the PN request information in the NM message, and CanNmPnFilterMaskByteValue configures the filter mask byte.

3.8.4 Spontaneous Transmission of NM PDUs

CanNm supports the Spontaneous Transmission of NM PDUs.

- CanNmPnEnabled must to be set to TRUE for the channel which is used.
- CanNmPnHandleMultipleNetworkRequests must be set to TRUE for CanNm to use the Spontaneous Transmission feature.
- The Immediate Transmission feature is mandatory (i.e. CanNmImmediateNmTransmissions is set to 1), and CanNmImmediateRestartEnabled must be

set to FALSE.

- The CanNmImmediateNmCycleTime container is used to configure the immediate NM PDU cycle time (in seconds), which is used for Immediate Transmission of NM PDUs. The result of CanNmImmediateNmTransmissions and CanNmImmediateNmCycleTime should be less than CanNmRepeatMessageTime.

3.8.5 Validation errors/warnings for Com User Data configuration

Note the following errors/warnings that can be generated during the validation of the configuration for Com user data.

- Error if `CanNmComUserDataSupport` is `TRUE` and `CanNmPassiveModeEnabled` is `TRUE`.
- Error if `CanNmComUserDataSupport` is `TRUE` and `CanNmUserDataEnabled` is `FALSE`.
- Error if `CanNmComUserDataSupport` is `TRUE` and `NmComUserDataEnabled` is `FALSE`.
- Error if `CanNmComUserDataSupport` is `TRUE` and `CanNmUserDataTxPdu` is not configured.
- Warning if `CanNmComUserDataSupport` is `FALSE` and `CanNmUserDataTxPdu` is configured.
- Error if `CanNmComUserDataSupport` is `TRUE` and `CanNmTxUserDataPduRef` is not configured.
- Error if `CanNmComUserDataSupport` is `TRUE` and a corresponding routing path is not configured in PduR.
- Error if `CanNmComUserDataSupport` is `TRUE` and `CanNmTxUserDataPduId` is not unique for each configured channel.
- Error if the length of the lpdu referred to by `CanNmTxUserDataPduRef` does not match the length of the `UserData` specified by `UserDataLength_u8`.
- Error if `CanNmComUserDataSupport` is `TRUE` and `PduR_CanNm_Support` is `FALSE`.

3.8.6 Code Generation based on configuration class

CanNm supports 2 kinds of configuration class -

- Pre-compile
- Post-build Loadable

If the Post-Build Loadable option is to be used for CanNm, the following two configuration steps must first be completed.

Step 1 Select the Configuration Class

Go to the CanNm module configuration, and then the “Properties” tab.

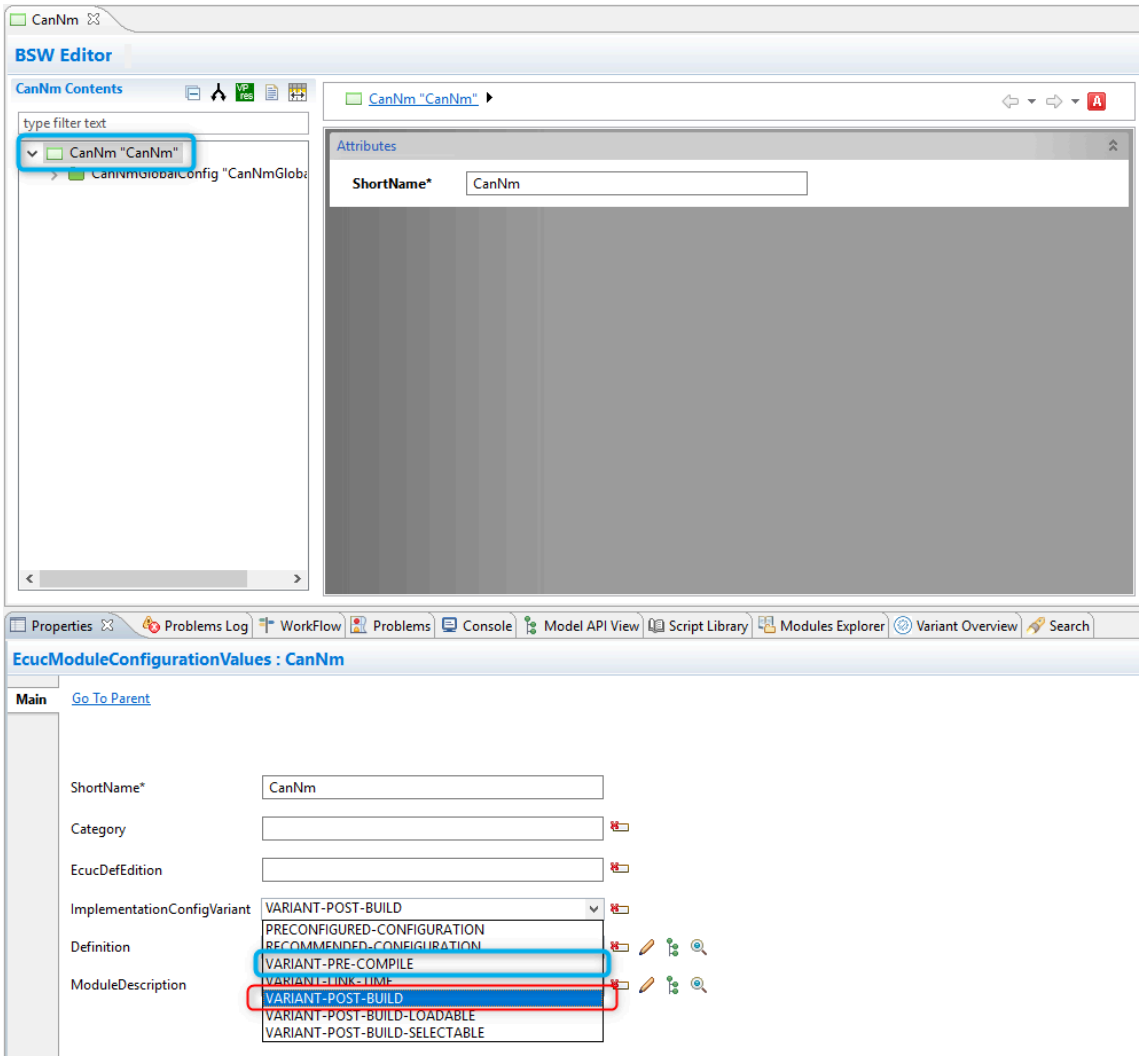


Figure 3.3.Select which configuration class of CanNm

ImplementationConfigVariant will be set to “variant-pre-compile” by default, but if Post-Build Loadable will be used for CanNm, select the “variant-post-build” option.

Step 2 EcuM Configuration

If ImplementationConfigVariant is set to “variant-post-build”, the EcuM configuration for CanNm must be configured with POSTBUILD_PTR.

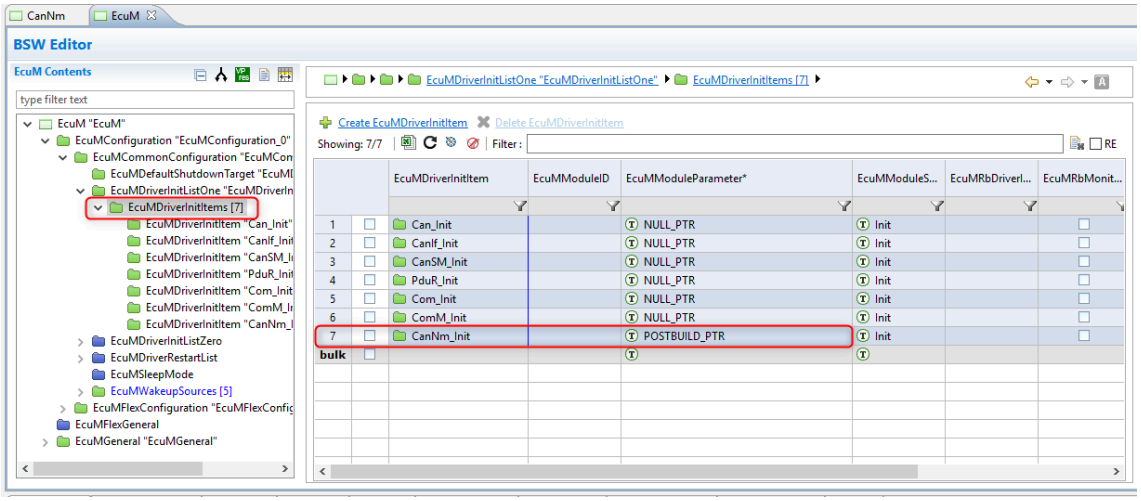


Figure 3.4.EcuM configuration

3.8.6.1 Code Generation Options

Depending on the above setting of ImplementationConfigVariant, the following options can be deployed at Code Generation.

Pre-Compile

If this option is chosen, all ECUs using the generated BSW will have the same configuration and there is no way to change this configuration without re-generating and re-compiling the whole of the ECU's software.

As shown below, select the "CanNm_PreCompile_and_PB_Variant" option for the CanNm module. The other options on the right can be set at your discretion.

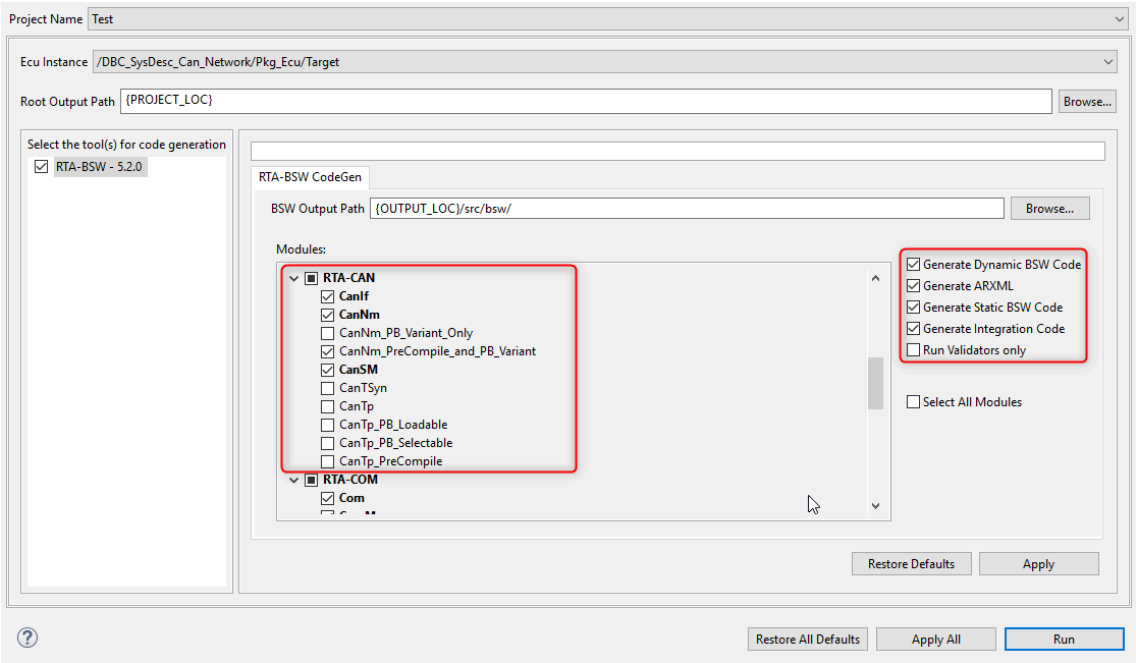


Figure 3.5.Generate CanNm with CanNm_PreCompile_and_PB_Variant

Post-Build Loadable

This option results in the generation of only the configuration for the CanNm module, and it does not generate any of the other code.

Post-Build Loadable is only available after the CanNm module's main code has initially been fully generated in a Post-Build configurable form, and this is achieved with a one-time Code Generation using the CanNm_PreCompile_and_PB_Variant option, as shown above.

Then, if the module's configuration is subsequently changed, as shown below, the CanNm_PB_Variant_Only option can be used so that only the configuration files are re-generated (the main CanNm code will not be re-generated).

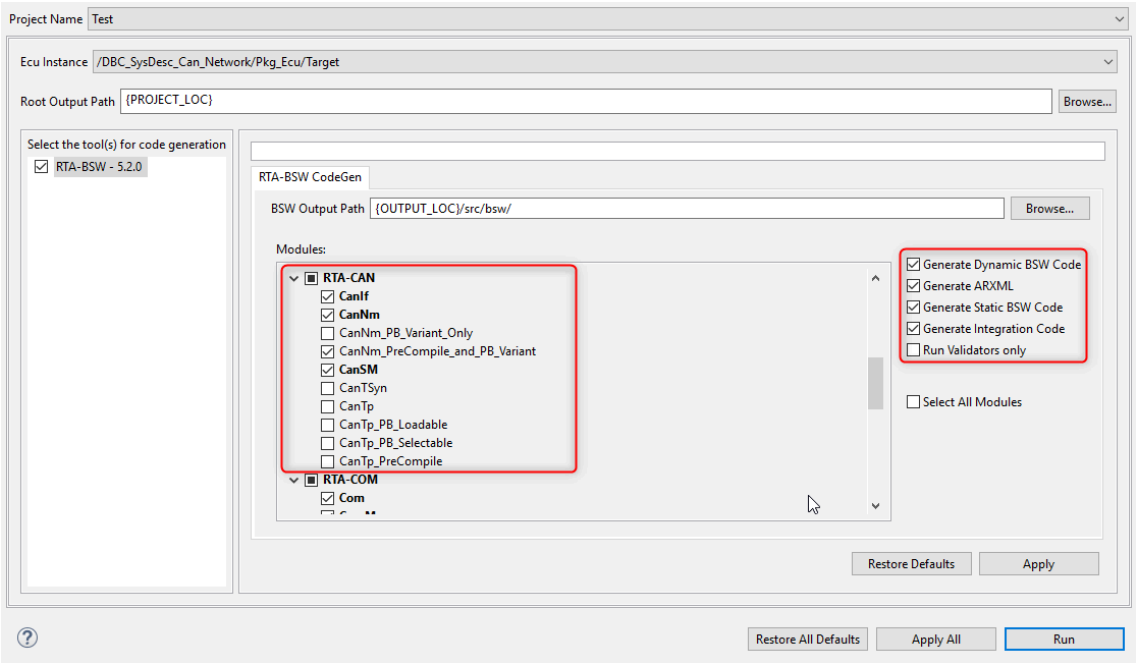


Figure 3.6.Full Code Generation with Post-Build

Note that the "Delete Existing Output Folder" option must be un-ticked. This results in the loadable parts being allocated into a different folder from the full generation that was previously created by the initial full build.

3.8.7 Configuration of System Extract for ConfGen

Multiple CanNmNodes in one network is not supported by ConfGen. This conforms to the AUTOSAR limitation where one channel of CanNm is associated with only one Network Management cluster in one network, and one Network Management cluster can have only one channel of CanNm in one node.

ConfGen supports the use case of an ECU supporting multiple Clusters/NmEcus. The NmGlobalConfig and CanNmGlobalConfig parameters will be generated based on the first NmEcu's configuration in the System Template.

3.9 Integration Advice

3.9.1 Init Functions

CanNm is initialized via a call to CanNm_Init by EcuM during the system initialization. CanNm_Init must be called after the initialization of CanIf.

3.9.2 DeInit Functions

No specific handling necessary.

3.9.3 Scheduled Functions

The OS calls CanNm_MainFunction, which calls the channel-specific main

processes internally.

CanNm_BSWMD.arxml is used by RTEGen to specify the requirements for scheduling the required functions with OS tasks.

3.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The CanNm module uses the following OS resources:

- CanNm_GetStateNoNest
- CanNm_GetPduDataNoNest
- CanNm_ChangeStateNoNest
- CanNm_GetUserDataNoNest
- CanNm_SetUserDataNoNest
- CanNm_MainFunctionNoNest
- CanNm_RxIndicationNoNest
- CanNm_UpdateTxPduDataNoNest
- CanNm_EnableCommunicationNoNest
- CanNm_TxConfirmationNoNest
- CanNm_RequestSynchronizedPncShutdownNoNest
- CanNm_TriggerTransmitNoNest

3.10 Hardware Requirements

Table 3.25.CanNm Hardware Requirements

Support	Usage
FLASH	2774 bytes
RAM	188 bytes

Usage calculated based on a sample configuration.

4 CAN State Manager

Module name	CanSM
Package	RTA-CAN
AUTOSAR Module ID	140
AUTOSAR Version	22-11

4.1 Introduction

The CAN State Manager (CanSM) is a member of the Communication service layer. It has interfaces to both the hardware abstraction layer (CanIf, CanNm) and the system service layer (ComM, EcuM, BswM, Dem). The diagram in the Introduction to this Reference Guide shows the position of the module in the AUTOSAR layered architecture.

CanSM coordinates with CanIf on the handling of network mode transitions of one or more underlying CAN Controllers and CAN Transceiver Drivers, and it supports multiple controllers for each network.

CanSM uses an internal State Machine to manage the transitions between network modes. These network modes are:

- CANSM_BSM_S_NOCOM
- CANSM_BSM_S_SILENTCOM
- CANSM_BSM_S_SILENTCOM_BOR
- CANSM_BSM_S_FULLCOM
- CANSM_BSM_S_NOT_INITIALIZED
- CANSM_BSM_S_PRE_NOCOM
- CANSM_BSM_S_PRE_FULLCOM
- CANSM_BSM_S_SET_BAUDRATE
- CANSM_BSM_S_TX_TIMEOUT_EXCEPTION
- CANSM_BSM_WUVALIDATION

The primary roles of CanSM are:

1. To manage Communication network modes.
2. To manage the Bus-Off Recovery state machine (Full and Silent Communication)
3. To manage the Baudrate state machine.
4. To manage the WakeUpValidation state machine
5. To manage the TxTimeOutException state machine.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

4.2 Supported Features

4.2.1 Manage state for each CAN network

The CanSM_RequestComMode API initiates the transitions between network modes, and that transition is processed by a periodic call to CanSM_MainFunction (see Integration Advice for more information).

4.2.2 Bus-Off Recovery

When CanIf detects the Bus-Off state, it will call CanSm_ControllerBusOff. CanSM uses a sub-state machine to ensure that BswM and ComM are notified of the state change, and CanIf_SetControllerMode is called to recover the network communication.

These are the Bus-Off recovery states:

CANSM_BOR_IDLE	0	Idle state
CANSM_S_BUS_OFF_CHECK	1	Initial bus-off check at beginning of full-communication
CANSM_S_NO_BUS_OFF	2	Regular state during full-communication without detected bus-off events
CANSM_S_BUS_OFF_RECOVERY_L1	3	Bus-off recovery level 1 (Short Recovery), TX disabled
CANSM_S_RESTART_CC	4	Restarting the controllers after Bus-off
CANSM_S_RESTART_CC_WAIT	5	Request for STARTED has returned E_OK, but mode indications not received
CANSM_S_BUS_OFF_RECOVERY_L2	6	Bus-off recovery level 2 (Long Recovery), TX disabled

The RTA-BSW implementation of CanSM also includes additional non-Autosar APIs to stop the Bus-Off Recovery (CanSm_Rb_DisableBusOffRecovery) and return the current BusOff-state of a given CanController (CanSM_GetBusoff_-Substate).

4.2.3 Changing the Baudrate

CanSM controls the Can Controller and the Pdu Modes for a network when a change to the Baud Rate is requested. If the current network state is CANSM_BSM_S_FULLCOM, then the baud rate can be changed by a call to CanSM_SetBaudrate.

Note: The previously supplied CanSM_CheckBaudrate API has been deprecated.

4.2.4 Wakeup Validation

A wake-up source can be started/stopped with CanSM_StartWakeupSource and CanSM_StopWakeupSource. CanSM also provides services to check and clear

WUF flag and Wake Flag from the Can Transceiver using the CanSM_ClearTrcvWufFlagIndication and CanSM_CheckTrcvWakeFlagIndication functions.

4.2.5 Transmission Timeout

CanNM will call CanSM_TxTimeoutException to notify CanSM when a transmission timeout has occurred. This will be handled by the respective network State Machine of the CanSM module.

4.2.6 Partial Networking

CanSM provides a service to notify CanNm when the CAN Transceiver is running in Partial Network communication mode. CanSM_ConfirmPnAvailability is called, which in turn notifies CanNM by calling CanNm_ConfirmPnAvailability.

4.2.7 Controller Mode Indication

Controller Mode Indication (via the CanSm_ControllerModeIndication callback) is used to indicate to CanSM any change in the Controller Mode. CanSM supports following Controller modes: START, STOP, SLEEP and WAKE.

4.2.8 Transceiver Mode Indication

Transceiver Mode Indication (via the CanSm_TransceiverModeIndication callback) is used to indicate to CanSM any change in the Transceiver mode. CanSM monitors two modes of the Transceiver: Normal and Standby.

4.3 Extensions

4.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.1.1 CanSM/CanSMConfiguration/CanSMManagerNetwork/CanSMDemEventParameterRefs/CanSMRbDynCreate_CANSM_E_BUS_OFF

Short Name	CanSMRbDynCreate_CANSM_E_BUS_OFF
Description	Selects the way how the CanSM DemEvents are created. If not set or set to “Disabled”, no such DemEvent is automatically created. If the dynamic creation of this DemEvent is enabled, the reference to this DemEvent (in parameter CANSM_E_BUS_OFF) is also automatically created and shall therefore not be set manually.
Introduction	ImplementationType: Disabled, Standard
Multiplicity	0..1
Default Value	Disabled
Enumeration Literals	Disabled, Standard
Type	ECUC-ENUMERATION-PARAM-DEF

4.3.1.2 CanSM/CanSMConfiguration/CanSMManagerNetwork/CanSMNetworkIndex

Short Name	CanSMNetworkIndex
Description	This parameter holds the unique channel index value. The value shall be the same as the ComMChannelId of the ComMChannel referenced by CanSMComMNetworkHandleRef. Implementation Type: NetworkHandleType
Multiplicity	1..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.3 CanSM/CanSMGeneral/CanSMCalibrationSupport

Short Name	CanSMCalibrationSupport
Description	This container contains the switch which has to be enabled/disabled for Calibration support
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

4.3.1.4 CanSM/CanSMGeneral/CanSMCalibrationSupport/CanSMCalibration

Short Name	CanSMCalibration
Description	This parameter defines whether CanSM supports calibration(of CanSMBorRTimeL1 and CanSMBorRTimeL2) or not.
Introduction	true: Enabled false: Disabled
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.5 CanSM/CanSMGeneral/CanSMCalibrationSupport/CanSMCalibrationNamePrefixCanSMBorTimeL1

Short Name	CanSMCalibrationNamePrefixCanSMBorTimeL1
Description	This container provides an option of giving the desired prefix to the calibratable CanSMBorTimeL1 parameter.
Multiplicity	0..1

Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.6 CanSM/CanSMGeneral/CanSMCalibrationSupport/CanSMCalibrationNamePrefixCanSMBorTimeL2

Short Name	CanSMCalibrationNamePrefixCanSMBorTimeL2
Description	This container provides an option of giving the desired prefix to the calibratable CanSMBorTimeL2 parameter.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.7 CanSM/CanSMGeneral/CanSMCalibrationSupport/CanSMCalibrationNameSuffixCanSMBorTimeL1

Short Name	CanSMCalibrationNameSuffixCanSMBorTimeL1
Description	This container provides an option of giving the desired suffix to the calibratable CanSMBorTimeL1 parameter.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.8 CanSM/CanSMGeneral/CanSMCalibrationSupport/CanSMCalibrationNameSuffixCanSMBorTimeL2

Short Name	CanSMCalibrationNameSuffixCanSMBorTimeL2
Description	This container provides an option of giving the desired suffix to the calibratable CanSMBorTimeL2 parameter.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.9 CanSM/CanSMGeneral/CanSMDSMReInitEnabled

Short Name	CanSMDSMReInitEnabled
Description	Enables or disables the reporting of DEM events to DEM after the Re-initialisation of DEM has occured.The status of latest DEM event is reported in the next MainFunction cycle
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.10 CanSM/CanSMGeneral/CanSMIcomSupport

Short Name	CanSMIcomSupport
Description	Selects support of Pretended Network features in CanSM. Autosar_Dev - Feature is not supported. hence parameter multiplicity is changed to 0
Introduction	True: Enabled False: Disabled
Multiplicity	0..0
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.11 CanSM/CanSMGeneral/CanSMNoPreNoCom

Short Name	CanSMNoPreNoCom
Description	This parameter is used to enable/disable the CANSM_BSM_S_PRE_NOCOM substate during startup. It is not recommended to set this value to True.
Multiplicity	0..0
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.2.1 CanSm_Rb_DisableBusOffRecovery

Syntax	`CanSM_Rb_DisableBusOffRecovery(NetworkHandleType network)`
Parameters	`network` Index of the Network Configuration structure.
Description	Called by Upper layer, updates a global array. The api does the following operations in Main_Function(): 1. Stops the busoff recovery 2. As busoff is not recovered within the intended time, stop the CAN controllers: This is to make sure that after the API is called there is no further communication (Tx or Rx by this ECU on the affected network). 3. Put all the TX and Rx pdu to OFFLINE mode for the intended CAN controller mapped to the CanSM network.

4.3.2.2 CanSM_GetNetworkMode_Substate

Syntax	`CanSM_GetNetworkMode_Substate(NetworkHandleType network, CanSM_NetworkModeStateType_ten * NetworkMode_SubstatePtr)`
Parameters	`network` Index of the Network Configuration structure. `NetworkMode_SubstatePtr` Data pointer to store the Current Network Mode Substate.

Description	Provides the current Network Mode Substate of the given CAN Network

4.3.2.3 CanSM_GetBusoff_Substate

Syntax	`CanSM_GetBusoff_Substate(uint8 ControllerId, CanSM_BusOffRecoveryStateType_ten* BORStatePtr)`
Parameters	`ControllerId` Id of the controller. `BORStatePtr` Data pointer to store the BusoffStatus.
Description	Provides the current BusOff-state of a given Can Controller.

4.4 Limitations

The following features are not supported for version v12.7.0

- Pretended Networking function is not supported, this feature has been removed since AR 20-11

4.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `CanSM_DeInit`

Callback functions not currently implemented:

- `CanSM_ConfirmCtrlPnAvailability`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- CanSM/CanSMConfiguration/CanSMManagerNetwork/CanSMDemEventParameterRefs/CANSM_E_MODE_REQUEST_TIMEOUT

4.6 Deviations

4.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

4.6.1.1 CanSM/CanSMConfiguration/CanSMManagerNetwork

[RTA-BSW] Description	This container contains the CAN network specific parameters of each CAN network. Autosar_Dev- Postbuild multiplicity is changed from false to true
[AUTOSAR] Description	This container contains the CAN network specific parameters of each CAN network

4.6.1.2 CanSM/CanSMConfiguration/CanSMManagerNetwork/CanSMDemEventParameterRefs

[RTA-BSW] Description	Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId symbolic value. The standardized errors are provided in this container and can be extended by vendor-specific error references.Autosar_Dev - Lower multiplicity is changed from 0 to 1
[AUTOSAR] Description	Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId symbolic value. The standardized errors are provided in this container and can be extended by vendor-specific error references.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

4.7 API Definition

List of supported API

`CanSM_Init()`	Called by EcuM to initialize the CanSm module.
`CanSM_RequestComMode()`	Sets the baud rate of a specific CAN controller.
`CanSM_GetCurrentComMode()`	Used to get the current communication mode of the Can network.
`CanSM_StartWakeupSource`	Called by EcuM when a wakeup source shall be started.
`CanSM_StopWakeupSource`	Called by EcuM when a wakeup source shall be stopped.
`CanSM_GetVersionInfo()`	Returns the version information of the called CanSM module.
`CanSM_SetBaudrate()`	Used to change the baud rate for the configured CAN controllers of a certain CAN network.
`CanSM_SetEcuPassive()`	Can be used to set all CanSM channels of the ECU to a receive only mode.
`CanSM_MainFunction()`	Scheduled function of the CanSM.

`CanSM_ControllerBusOff()`	Informs the CanSM about a bus-off event on a certain CAN controller.
`CanSM_ControllerModeIndication()`	Informs CanSM about a CAN controller mode change.
`CanSM_TransceiverModeIndication()`	Informs CanSM about a CAN transceiver mode change.
`CanSM_TxTimeoutException()`	CanNM calls this function to notify CanSM about transmission timeout exceptions.
`CanSM_ClearTrcvWufFlagIndication()`	Called by CanIf to indicate the end of the CheckTransceiverWakeFlag API process for the specified CAN Transceiver.
`CanSM_CheckTransceiverWakeFlagIndication()`	Called by CanIf to indicate CanSM about wakeup flag.
`CanSM_ConfirmPnAvailability()`	This callback function indicates that the transceiver is running in Partial Networking communication mode.

For Further information please refer to AUTOSAR_SWS_CANStateManager.pdf

4.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

4.8.1 General Configuration

- If the Can Transceivers are configured, ensure that the CanSM_-TransceiverModeIndication function is called.
- Ensure that the controller mode callback function CanSM_ControllerModeIndication is called.
- If Bus-off recovery is to be based on tx-confirmation, then set CanSMBorTxConfirmationPolling (in CanSMConfiguration/CanSMManagerNetwork) as TRUE.
- When using the CanSM_GetBusoff_Substate API, the enum CanSM_BusOffRecoveryStateType_ten must first be checked.
- The header file name along with the extension should be provided in the configuration of the CanSMGetBusOffDelayHeader parameter.

- CanSM does not support Bus-off processing via Interrupt, so CanBusoffProcessing should be set to POLLING.

4.8.2 Partial Networking (PN)

Partial Network support in CanSM is enabled by setting CanSMPncSupport (in CanSMGeneral) as TRUE, and you must also ensure that parameters that relate to Partial Networking in CanIf, ComM, CanNm and CanTrcv are configured as TRUE.

If the ComMPncSupport parameter in ComM is configured as FALSE, then CanSMPncSupport in CanSM must also be configured as FALSE.

Also ensure that the CanSM_ClearTrcvWufFlagIndication and CanSM_CheckTransceiverWakeFlagIndication callback functions are called.

4.9 Integration Advice

4.9.1 Init Functions

CanSm_Init is called by EcuM during the system initialization.

Once CanSM is initialised, the CanSM network is initialised based on whether or not it is mapped to a CAN Transceiver, and, if it is, whether the CAN Transceiver is configured to support the PN feature. CanSM supports following initialisation for any network:

CANSM_BSM_DeInitPnSupported	If the CanSM network is configured to a transceiver and the transceiver supports PN feature.
CANSM_BSM_DeInitPnNotSupported	If the CanSM network is configured to a transceiver and the transceiver does not supports PN feature, or if the CanSM network is not configured to a transceiver.

A successful initialisation of the CANSM network ensures the state of the Transceiver is in STANDBY, the controller is in SLEEP state, and the CanSM network state is NO_COMM. A request for FULL_COMM can be made after CanSM is initialized.

To ascertain whether CanSM initialized or not, call CanSM_GetCurrentComMode. If CanSM is initialized, CanSM_GetCurrentComMode will return E_OK. If CanSm is not initialized, it will return E_NOT_OK.

4.9.2 DeInit Functions

CanSm_DeInit de-initialises the RAM contents of all the Channels of the given configset to default value. This API is required when Post-build configuration is enabled in CanIf.

4.9.3 Scheduled Functions

CanSM_MainFunction should be called cyclically (the cycle time is define by CanSMMainFunctionTimePeriod, the default value is 10ms).

4.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The CanSM module uses the following OS resources:

- CanSM_BOR_Nw_ModesNoNest
- CanSM_BaudModesNoNest
- CanSM_GetMutexNoNest

4.10 Hardware Requirements

Table 4.21.CanSM Hardware Requirements

Support	Usage
FLASH	9169 bytes
RAM	408 bytes

Usage calculated based on a sample configuration.

5 CAN Transport Layer

Module name	CanTp
Package	RTA-CAN
AUTOSAR Module ID	35
AUTOSAR Version	22-11

5.1 Introduction

The CAN Transport Layer (CanTp) is part of the Communication Stack, and it sits between the PDU Router (PduR) and the CAN Interface (CanIf). CanTp is part of the Diagnosis Communication of the Basic Software architecture. The diagram in the Introduction to this Reference Guide shows the position of the module in the AUTOSAR layered architecture.

CanTp handles transport protocol frames. It segments and reassembles CAN I-PDUs longer than 8 bytes or longer than 64 bytes in case of CAN FD.

CanTp is responsible for the communication between the peer protocol entities of the network layer in different nodes by exchanging N_PDUs. In general, all N_PDUs consist of three (3) fields as given in below table.

Address information	Protocol control information	Data field
N_AI	N_PCI	N_Data

Figure 5.1.N_PDU format

CanTp provides services for:

1. Segmentation of data in transmit direction
2. Reassembling of data in receive direction
3. Control and monitor of data flow
4. Detection of errors in segmentation sessions
5. Transmit cancellation

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

5.2 Supported Features

5.2.1 Segmentation and reassembling of data

CanTp accepts asynchronous transmit requests for MultiFrame/SingleFrame transmission from PduR through the `CanTp_Transmit()` function.

The decision on which type of frame to be formed is based on CAN FD

enabled/disabled use cases and Sdu length.

The N_PDU contains three parts: address information, protocol control information (N_PCI) and data field. The N_PCI field identifies four types of N_PDU: SingleFrame, FirstFrame, ConsecutiveFrame, and FlowControl.

The figures below show an example of an unsegmented message transmission and an example of a segmented message transmission.

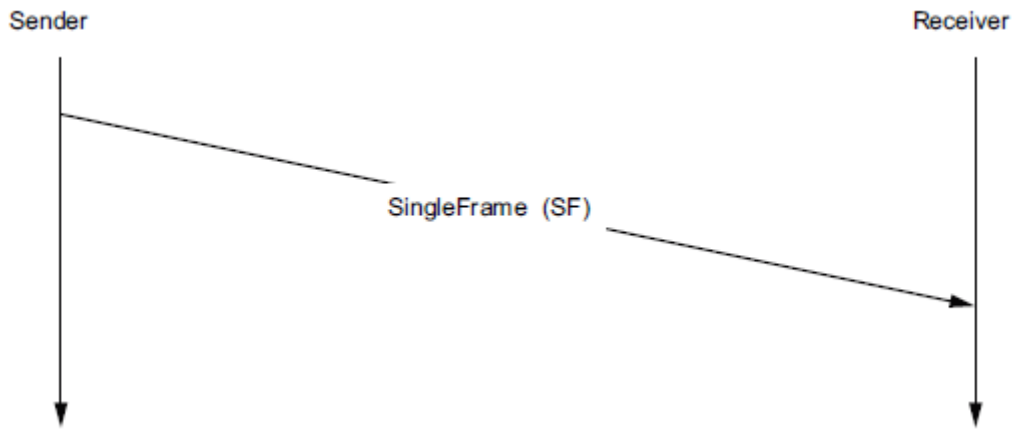


Figure 5.2.Single Frame message

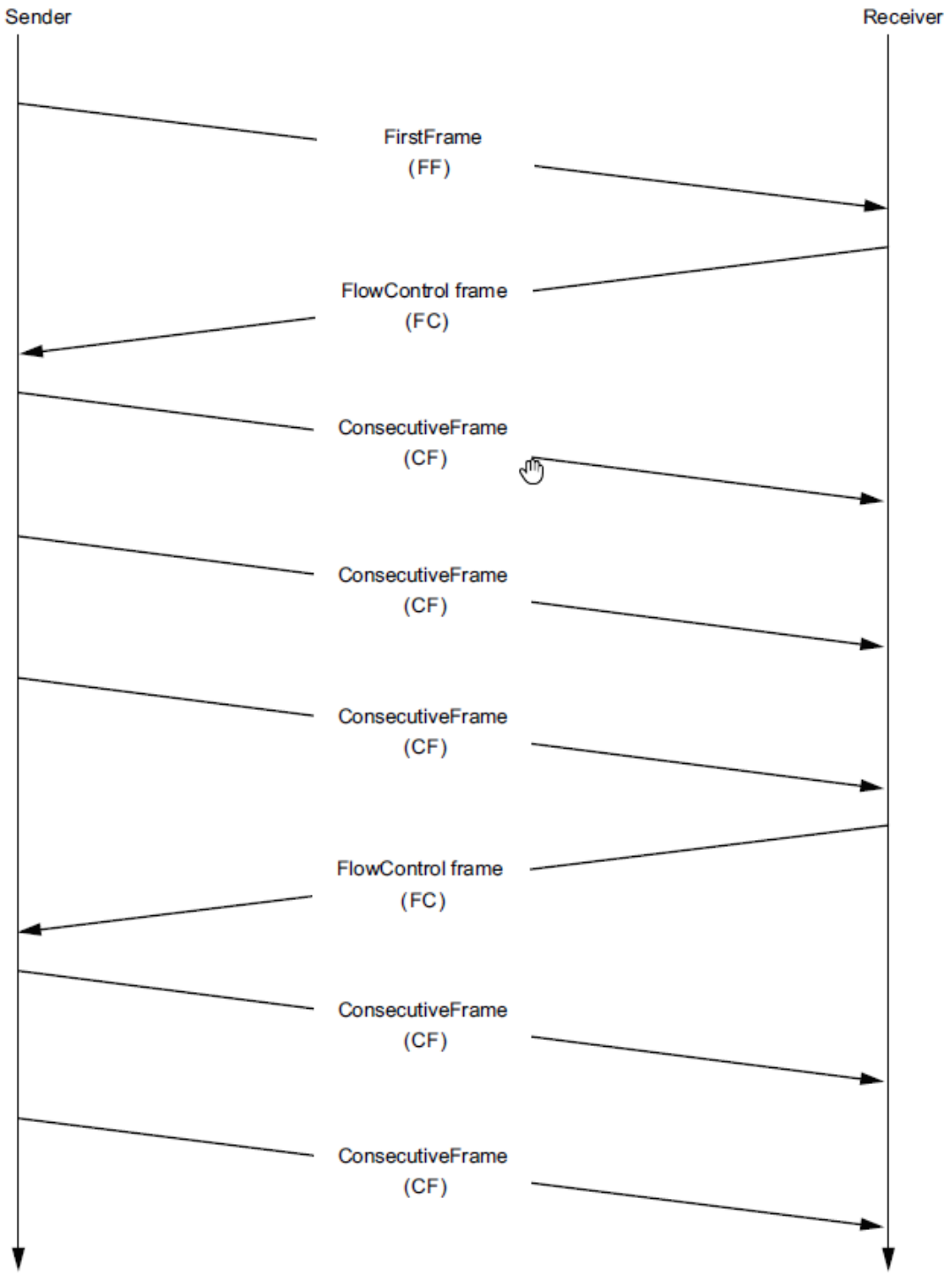


Figure 5.3.Multiple Frame message

In a MultiFrame sequence, there are three types of frames: first-frame (FF N_PDU), consecutive-frame (CF N_PDU) and flow-control (FC N_PDU).

5.2.2 Addressing format

The exchange of network layer data is supported by five addressing formats: `CANTP_EXTENDED`, `CANTP_MIXED`, `CANTP_MIXED29BIT`, `CANTP_NORMALFIXED`, `CANTP_STANDARD`. Each addressing format requires a different number of CAN frame data bytes to encapsulate the addressing information associated with the data to be exchanged.

The CAN frame data field and CAN Id format are dependent on the combination of the network target address type and N_TAtype (physical/functional).

5.2.3 Control and monitor of data flow

CanTp monitors timeout and states of Transmission/Reception messages. When a timeout occurs, CanTp stops the process and notifies the upper layer of this failure by calling the `PduR_CanTpTxConfirmation` callback function with the result `E_NOT_OK`.

The definition of each timer is illustrated below.

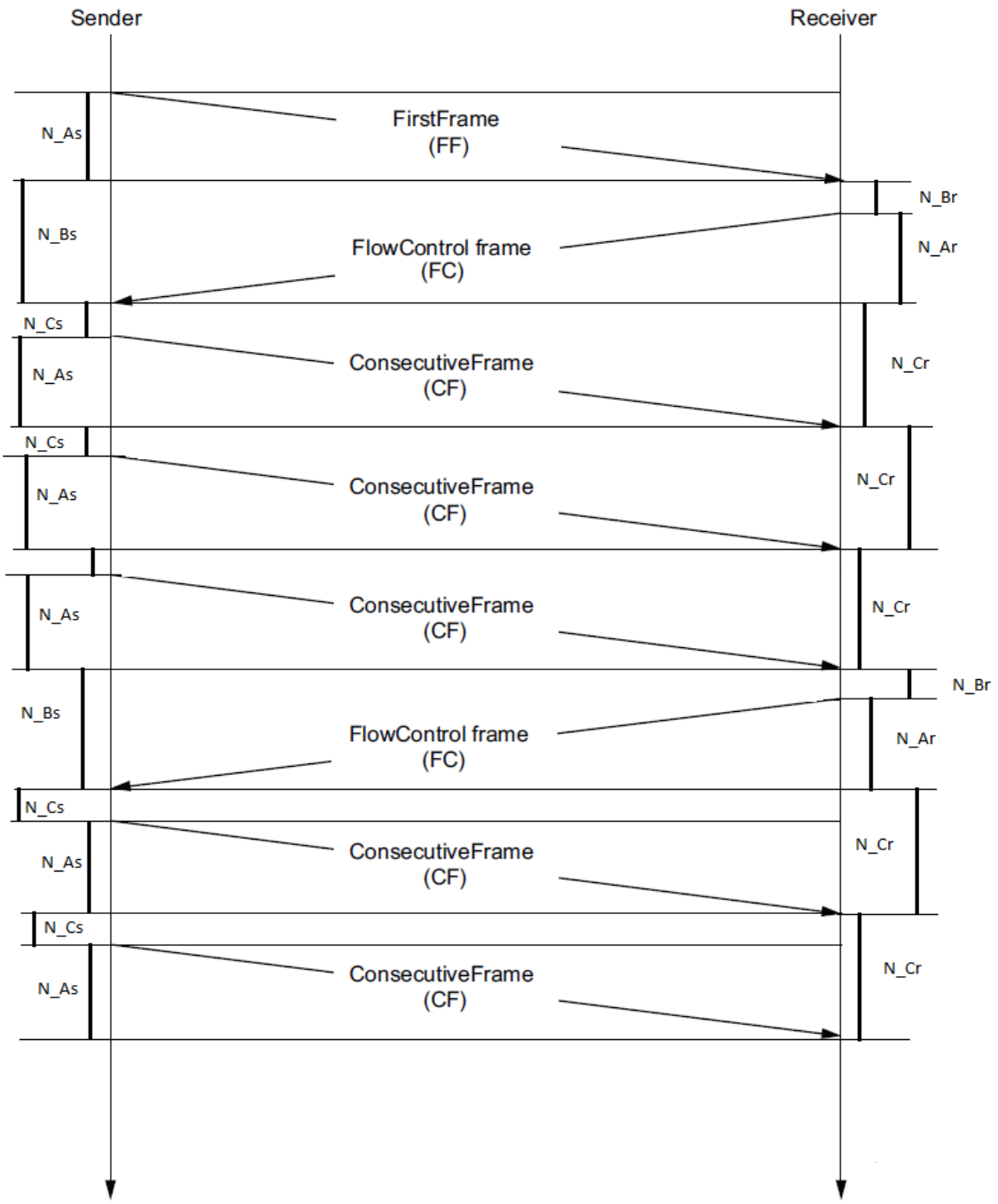


Figure 5.4.Detail of timer in sequence diagram

5.2.4 Cancel transmission

CanTp supports the cancellation of the transfer of a pending CAN N-SDU via the `CanTp_CancelTransmit()` function.

5.2.5 Concurrent connection

CanTp supports the concurrent connection feature while transmitting or receiving MultiFrame. This is controlled by the CanTpChannelMode parameter, which can be set to either CANTP_MODE_FULL_DUPLEX or CANTP_MODE_HALF_DUPLEX, which have the following effects:

1. Half-duplex, point-to-point communication between two nodes is only possible in one direction at a time.
2. Full-duplex, point-to-point communication between two nodes is possible in both directions at once.

5.3 Extensions

5.3.1 Additional features available in version v12.7.0

- CanTpLite Support: Lite mode is achieved by reduction in configuration. Configure only the Channels and Connections required. Disable the unused Apis in `CanTpGeneral` container. Refer to the description of configuration parameter `CanTpRbLiteSupport` in Configuration Extensions section.
- BurstMode: (This is implemented as pure Autosar solution with StMin = 0). Refer to the description of configuration parameter `CanTpRbBurstTaskRef` in Configuration Extensions section.
- Disable FC support: This is implemented on connection level. Refer to the description of configuration parameters `CanTpRbRxFcDisabled` and `CanTpRbTxFcDisabled` in Configuration Extensions section.
- Strict DLC check: Refer to the description of configuration parameter `CanTpRbStrictDlcCheck` in Configuration Extensions section.
- Strict BsCr timeout check: Refer to the description of configuration parameter `CanTpRbStrictBsCrTimeoutCheck` in Configuration Extensions section.
- Shared FC Pdu support: Refer to the description of configuration parameter `CanTpRbLegacySharedFcEnabled` in Configuration Extensions section.

5.3.2 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

5.3.2.1 CanTp/CanTpConfig/CanTpChannel/CanTpRbHalfDuplexEnable

Short Name	CanTpRbHalfDuplexEnable

Description	Preprocessor switch for enabling the Half Duplex funtionality for this CanTp Channel. If configured as TRUE, this CanTp Channel will function as Half Duplex. If configured as FALSE or not configured at all(by default), this CanTp Channel will function as Full Duplex.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.2 CanTp/CanTpConfig/CanTpChannel/CanTpRxNSdu/CanTpRbRxFcDisabled

Short Name	CanTpRbRxFcDisabled
Description	Vendor specific extension defines whether Flow Control Frame is supported in this connection. For an ISO 15765-2 Specific Implementation its value should be false. false: The connection will send Flow Control Frames during segmented reception. true - No Flow Control Frame is sent during segmented reception.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.3 CanTp/CanTpConfig/CanTpChannel/CanTpTxNSdu/CanTpRbTxFcDisabled

Short Name	CanTpRbTxFcDisabled
Description	Vendor specific extension defines whether Flow Control Frame is supported in this connection. For an ISO 15765-2 Specific Implementation. Its value should be false. false: The connection will Receive and process the Flow Control Frames during segmented transmission. true - No Flow Control Frame is expected during segmented transmission.The Consecutive frames are transmitted one after the other without waiting for a flow control frame.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.4 CanTp/CanTpGeneral/CanTpRbBurstQsize

Short Name	CanTpRbBurstQsize

Description	Size of the burst Queue. If this value is configured, Burst Mode gets enabled. It gives maximum possible burst simultaniously.
Multiplicity	0..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.2.5 CanTp/CanTpGeneral/CanTpRbBurstTaskRef

Short Name	CanTpRbBurstTaskRef
Description	Absolute Reference to OsTask in which context the CanTp_MainFunction is called for Burst Mode. To enable Burst Mode this Reference configuration is a must. This task is activated in case of additional execution of Burstmode activity. Depending on the value of STmin (received through FC) is zero for a particular connection, the transmission will happen in Burst mode, else if the STmin is a non zero valid value, then the CFs shall be sent normally. Other modules MainFunction should not be mapped to this task. Dependency: CanTpRbBurstQsize. Note: Priority of this Task should be same as the Task in which Normal cyclic Timing Event is mapped w.r.t. CanTp_MainFunction. For more details on different type of events supported by CanTp please refer CanTp_Cfg_BSWMD.arxml generated.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.2.6 CanTp/CanTpGeneral/CanTpRbDynamicFlowControlSupport

Short Name	CanTpRbDynamicFlowControlSupport
Description	Enables Dynamic BS/STmin values in subsequent FlowControl frames. Iso Document gives possibility to have dynamic values for STMin and BS values in single Reception process. Autosar has strategy to have only fixed Values. This parameter provides a means to override the Autosar behaviour. For more details refer section 9.6.5.6 of Iso document. Note: This parameter shall be set True only after considering that corresponding Client Server communication is compatible in this respect for all the RxConnections as this parameter enables this dynamic behaviour globally.

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.7 CanTp/CanTpGeneral/CanTpRbEnableDoipV4Support

Short Name	CanTpRbEnableDoipV4Support
Description	Vendor specific extension defines if doip v4 support feature is enabled or not. false: Doip v4 feature is disabled. true - Doip v4 feature is enabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.8 CanTp/CanTpGeneral/CanTpRbExternalCounterSecondsPerTick

Short Name	CanTpRbExternalCounterSecondsPerTick
Description	This parameter is required if High resolution Timer (i.e. Autosar Os based or Project specific) is used. This gives number of seconds in one Tick of the used Timer. If this parameter is configured then Integrator shall provide the Call Out implementation for CanTp_GetElapsedValue. Its Signature: void CanTp_GetElapsedValue(TickType *Value, TickType *ElapsedValue). If this parameter is not configured then internal Cyclic counter based on CanTp_MainFunction period is used, in which case CallOut implementation shall not be provided by integrator.
Multiplicity	0..1
Value Range	0..Inf
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

5.3.2.9 CanTp/CanTpGeneral/CanTpRbFdSupportInfoCallback

Short Name	CanTpRbFdSupportInfoCallback
Description	When user configures this parameter CanTp gives callback using this function. With this function CanTp passes ID, Data pointer and FlexibleDataRateSupport whether enabled for relevant connection.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

5.3.2.10 CanTp/CanTpGeneral/CanTpRbImmediateFfProcessing

Short Name	CanTpRbImmediateFfProcessing
Description	Vendor specific extension defines whether First Frame shall be processed Immediately after state changes tx accepted. false: Normal operation . true - First frame will be processed immediately.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.11 CanTp/CanTpGeneral/CanTpRbLegacySharedFcEnabled

Short Name	CanTpRbLegacySharedFcEnabled
Description	This is vendor specific parameter. TRUE: single Flow Control Pdu frame can be shared between different connection. FALSE: single Flow Control Pdu frame cannot be shared between different connection. Note: This parameter is relevant only for sharing of FC NPDU with standard Addressing format. Limitation: Only such mechanism of sharing for FC NPDU works if one Connection is relavant for OBD and other for UDS Protocol of Dcm, and hence the disambiguation for the reception is possible due to overall PduLength difference created using padding bytes. For Extended Addressing Format relevant FC NPDU sharing is supported by usual method. This is only for legacy projects. For new Projects this is not recomended.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.12 CanTp/CanTpGeneral/CanTpRbLiteSupport

Short Name	CanTpRbLiteSupport
Description	This parameter helps Integrator to configure CanTp in such a way to have Lite footprint. For an ISO 15765-2 Specific and Autosar compliant Implementation, this parameter shall not be configured. If this parameter is TRUE then following features are not to be configured: CanTpChangeParameterApi, CanTpReadParameterApi, CanTpVersionInfoApi. Also CanTp_TriggerTransmit and CanTp_CancelTransmit will not be supported.

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.13 CanTp/CanTpGeneral/CanTpRbMaxRxConnectionCnt

Short Name	CanTpRbMaxRxConnectionCnt
Description	Maximum possible value of number of Rx Connection. If this value is to be updated then source code has to be recompiled. This decides number of different Stmin/Bs combination can be stored in RAM for CanTp_ChangeParameter Api. CanTpRbMaxRx ConnectionCnt and CanTpRbMaxTxConnectionCnt decides the size for CanTp_ConnectionIdType.
Multiplicity	0..1
Value Range	1..1.84467440737095E+19
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.2.14 CanTp/CanTpGeneral/CanTpRbMaxTxConnectionCnt

Short Name	CanTpRbMaxTxConnectionCnt
Description	Maximum possible value of number of Tx Connection. If this value is to be updated then source code has to be recompiled. CanTpRbMaxTxConnectionCnt and CanTpRbMaxRxConnectionCnt decides the size for CanTp_ConnectionIdType.
Multiplicity	0..1
Value Range	1..1.84467440737095E+19
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.2.15 CanTp/CanTpGeneral/CanTpRbMaxTxNpduCnt

Short Name	CanTpRbMaxTxNpduCnt
Description	Maximum possible value of number of uinque CanIf referred Pdus. If this value is to be updated then source code has to be recompiled. This value decides the maximum value for CanTpTxFcNPdu ConfirmationPduId or CanTpTxNPduConfirmation-PduId.
Multiplicity	0..1

Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.2.16 CanTp/CanTpGeneral/CanTpRbNonCanIfLowerLayer

Short Name	CanTpRbNonCanIfLowerLayer
Description	This parameter is to enable the CanTp to be used when other lower layer is configured other than CanIf. True - Lower layers other than CanIf. False - CanIf
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.17 CanTp/CanTpGeneral/CanTpRbPageConfirmation

Short Name	CanTpRbPageConfirmation
Description	This parameter is to be set to True if Dcm needs a trigger from lower layer to refill its buffer when Paged Buffer feature is enabled. It is set to False if Dcm has the feature to refill its buffer on its own without any trigger.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.18 CanTp/CanTpGeneral/CanTpRbSTmin

Short Name	CanTpRbSTmin
Description	This vender specific paramter is to support FC deactivation. This value is used for STmin. Sets the duration of the minimum time the CanTp sender shall wait between the transmissions of two CF N-PDUs. Also if this parameter is configured, it means that CanTp will not send the FC for the FF recieved.
Multiplicity	0..1
Value Range	0..0.127
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

5.3.2.19 CanTp/CanTpGeneral/CanTpRbStrictBsCrTimeoutCheck

Short Name	CanTpRbStrictBsCrTimeoutCheck
Description	FC and CF frames are handled within event calls in CanTp_RxIndication; thus it is possible to accurately check for the currenttime within the CanTp_RxIndication before accepting that FC/CF frame. It is also known that this event call can also be triggered from the interrupt context and many projects may not intend to call OS functions within interrupt context. Thus, it is provided as a option to users to choose if TimeOut handling needs to be done before processing a received CF/FC frame within CanTp_RxIndication. If configured TRUE, This timeout check is done in CanTp_RxIndication also, if FALSE, the timeouts are handled only from CanTp_MainFunction. Dependency: CanTpRbExternalCounterSecondsPerTick.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2.20 CanTp/CanTpGeneral/CanTpRbStrictDlcCheck

Short Name	CanTpRbStrictDlcCheck
Description	If this parameter is configured TRUE, the received frames are checked strictly for the exact DLC expected in the frame. Configuring this parameter as FALSE provides some leniency in the check for received DLC, where any DLC greater than the expected length of the frame (but not greater than 8) are accepted. For behaviour according to ISO 15765-2, please configure this parameter as FALSE. NOTE: The configuration of this parameter will not have any effect if the parameter CanTpRxPaddingActivation or CanTpTxPaddingActivation are configured as CANTP_ON
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.3 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

Table 5.22.CanTp API Extensions

`CanTp_TriggerTransmit()`	Can Driver calls this Api for the data to be transmitted corresponding to the request in CanIf_Transmit.

5.4 Limitations

The following features are not supported for version v12.7.0

- 16-bit Micro controller architecture is not supported as the Compiler abstraction is not implemented.

- CanTp does not support dynamic ID handling via N-PDU MetaData. The `CanTpDynIdSupport` and `CanTpGenericConnectionSupport` configuration parameters are not supported.

- CanTp does not support partial configurations for metadata implementation. The current implementation of metadata is being taken up only for diagnostic purposes.

- In CanTp PostBuild Variant, values are not supported for `CanTpRxNSduRef`, `CanTpTxFcNPduRef`, `CanTpTxNSduRef` and `CanTpRxFcNPduRef`.

5.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

5.6 Deviations

5.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

5.6.1.1 CanTp/CanTpConfig/CanTpChannel/CanTpRxNSdu/CanTpRxPaddingActivation

[RTA-BSW] Description	Defines if the receive frame uses padding or not. This parameter is restricted to 8 byte N-PDUs.
[AUTOSAR] Description	Defines if the received frame uses padding or not. This parameter is restricted to 8 bytes N-PDUs.

[RTA-BSW] Introduction	Definition of enumeration values: CanTpOn: The N-PDU received uses padding for SF, FC and the last CF. (N-PDU length is always >= 8 bytes in case of CAN 2.0) CanTpOff: The N-PDU received does not use padding for SF, CF and the last CF. (N-PDU length is dynamic - any valid DLC value). Note: The mandatory mapping to the next higher valid DLC value for N-PDUs with a length > 8 bytes is not affected by this parameter.
[AUTOSAR] Introduction	Definition of enumeration values: CanTpOn: The N-PDU received uses padding for SF, FC and the last CF. (N-PDU length is always >= 8 bytes in case of CAN 2.0) CanTpOff: The N-PDU received does not use padding for SF, FC and the last CF. (N-PDU length is dynamic - any valid DLC value). Note: The mandatory mapping to the next higher valid DLC value for N-PDUs with a length > 8 bytes is not affected by this parameter.

5.7 API Definition

List of supported API functions

`CanTp_Init()`	Called by EcuM. Initializes the CanTp module
`CanTp_GetVersionInfo()`	Returns the version information of the CanTp module.
`CanTp_MainFunction()`	The main function for scheduling the CAN TP.
`CanTp_Shutdown()`	This function is called to shutdown the CanTp module.
`CanTp_Transmit()`	Asyncrounus Transmission request Api for upper layer module
`CanTp_CancelTransmit()`	This service primitive is used to cancel the transfer of a pending CAN N-SDU.
`CanTp_CancelReceive()`	Cancels the reception of an ongoing N-SDU.
`CanTp_RxIndication()`	Indication of a received I-PDU from a lower layer communication interface module (CanIf module)
`CanTp_TxConfirmation()`	Indication of a transmit confirmation from a lower layer communication interface module (CanIf module)
`CanTp_ChangeParameter()`	This API is used to change the current value of reception parameters BS and STmin for a specified RxNSdu.
`CanTp_ReadParameter()`	This API is used to read the current value of reception parameters BS and STmin for a specified N-SDU

For Further information please refer to AUTOSAR_SWS_CANTransportLayer.pdf

5.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

5.8.1 Code Generation based on configuration class

CanTp supports 3 kinds of configuration class -

- Pre-compile
- Post-build Loadable
- Post-build Selectable

If either of the two Post-Build options is to be used for CanTp, the following two configuration steps must first be completed.

Step 1 Select the Configuration Class

Go to the CanTp module configuration, and then the “properties” tab.

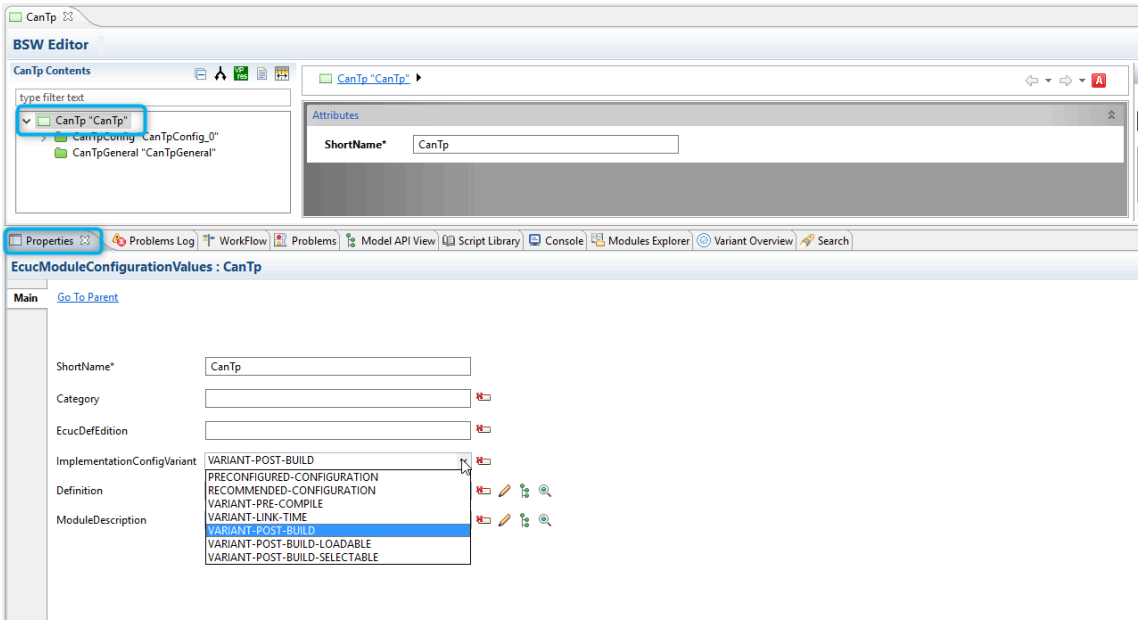


Figure 5.5.Select which configuration class of CanTp

ImplementationConfigVariant will be set to “variant-pre-compile” by default, but if either of the two Post-Build options will be used for CanTp, you must select the generic “variant-post-build” option (and not the specific “variant-post-build-loadable” or “variant-post-build-selectable” options).

Step 2 EcuM Configuration

If ImplementationConfigVariant is set to “variant-post-build”, the EcuM configuration for CanTp and all its dependent modules must be configured with POST-BUILD_PTR.

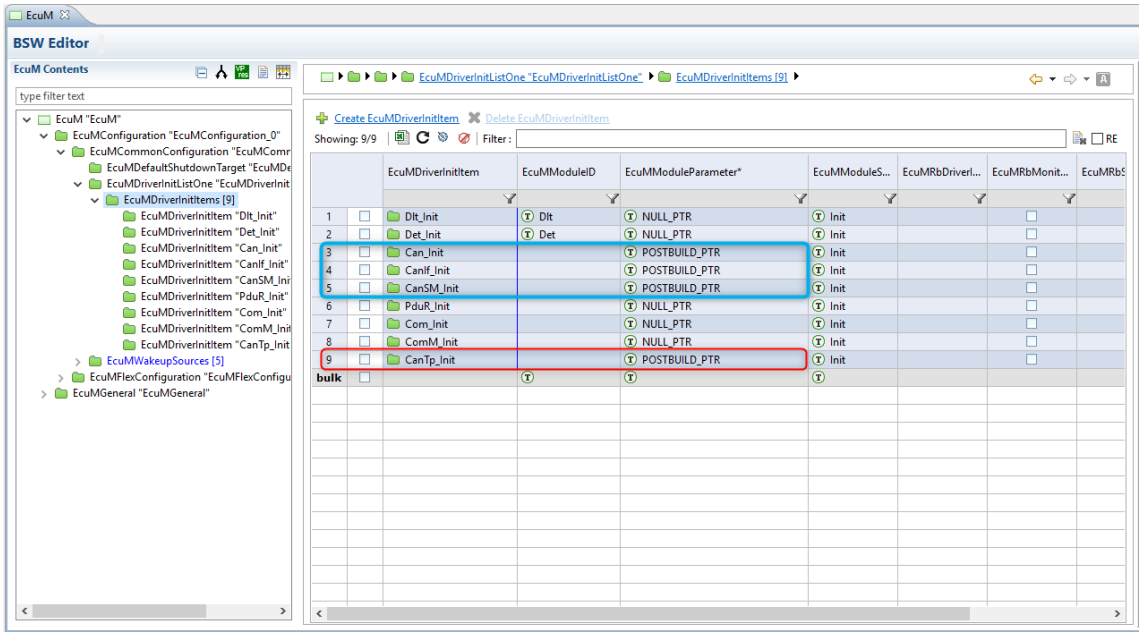


Figure 5.6.EcuM configuration

5.8.1.1 Code Generation Options

Depending on the above setting of ImplementationConfigVariant, the following options can be deployed at Code Generation.

Pre-Compile

If this option is chosen, all ECUs using the generated BSW will have the same configuration and there is no way to change this configuration without re-generating and re-compiling the whole of the ECU's software.

As shown below, select the "CanTp_PC" option for the CanTp module. The other options on the right can be set at your discretion.

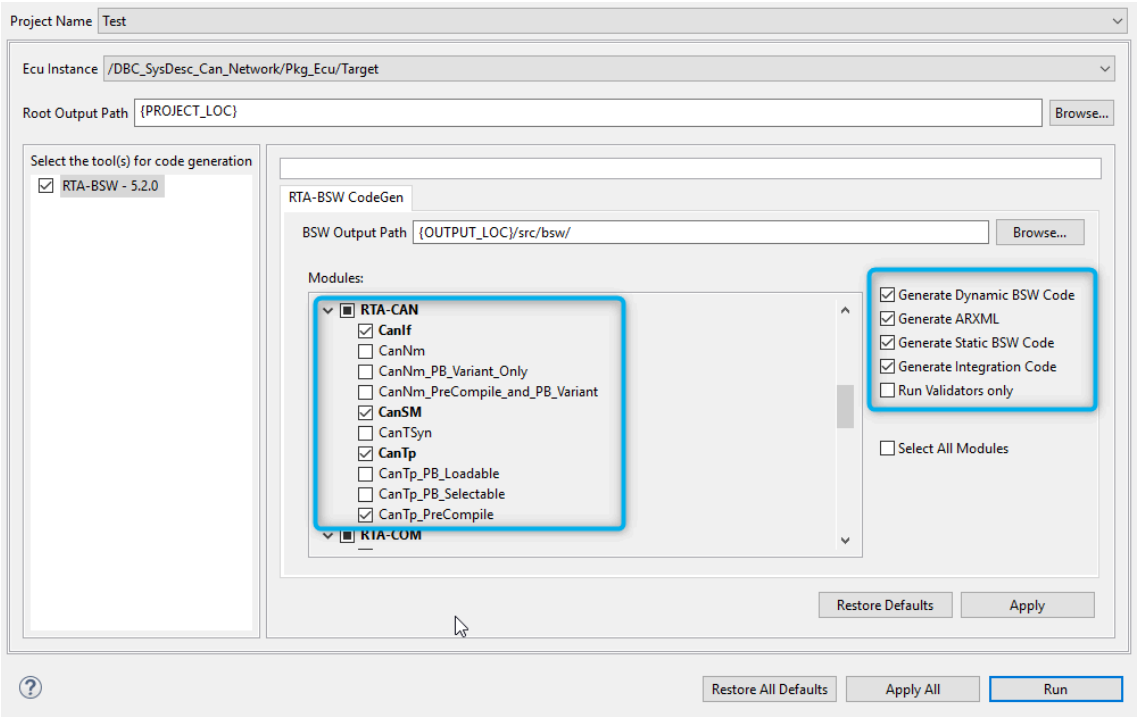


Figure 5.7.Generate CanTp with Configuration Class as Pre-compile

Post-Build Loadable

This option results in the generation of only the configuration for the CanTp module, and it does not generate any of the other code.

Post-Build Loadable is only available after the CanTp module's main code has initially been fully generated in a Post-Build configurable form, and this is achieved with a one-time Code Generation using Post-Build Selectable, as shown here.

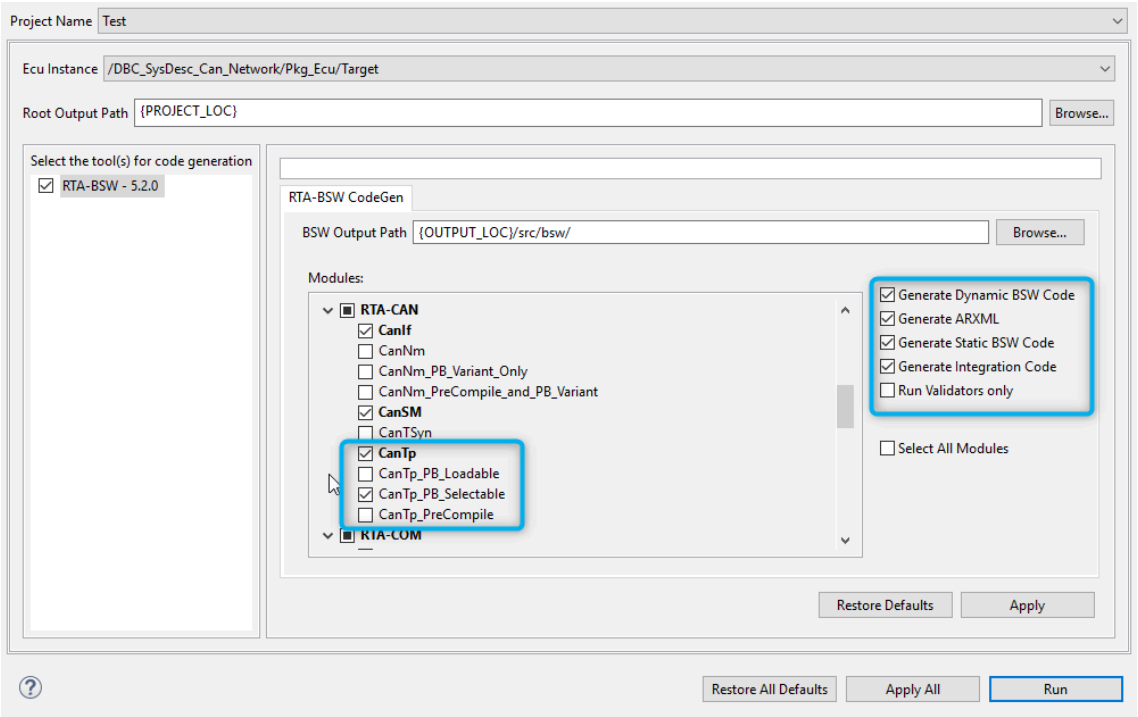


Figure 5.8.Full Code Generation with Post-Build

Then, if the module's configuration is subsequently changed, as shown below the Post-Build Loadable option can be used so that only the configuration files are re-generated (the main CanTp code will not be re-generated).

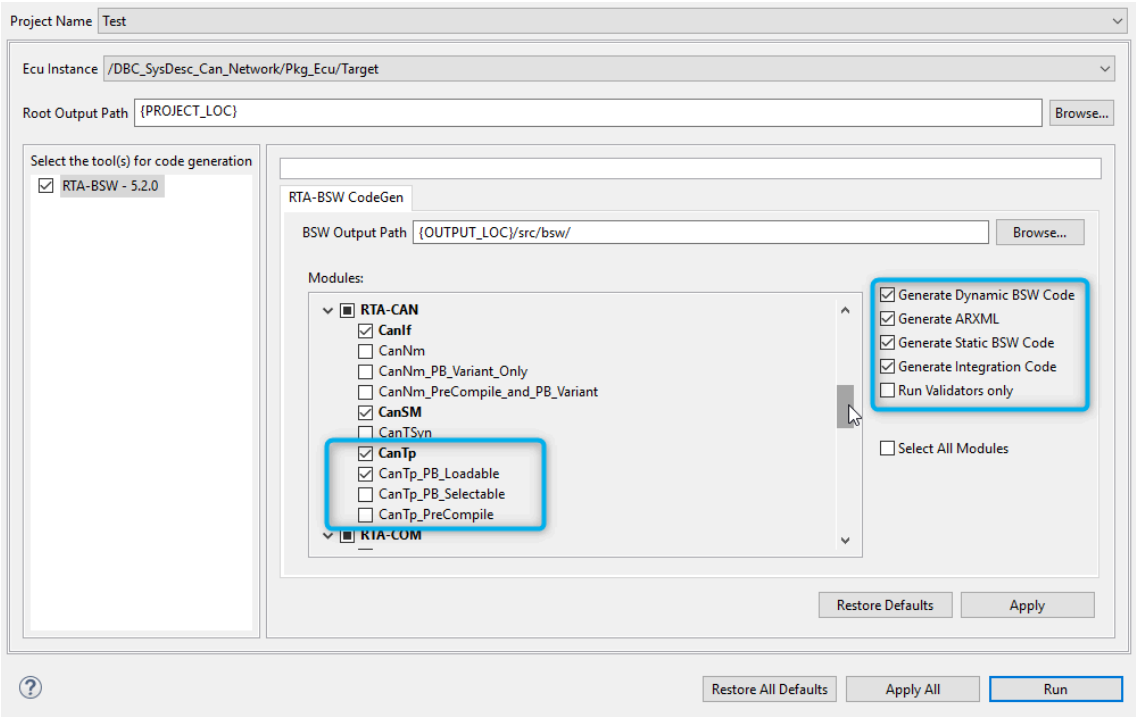


Figure 5.9.Regeneration for Post-Build

Note that the “Delete Existing Output Folder” option must be un-ticked when using Post-Build Loadable. This results in the loadable parts being allocated into a different folder from the full generation that was previously created by the initial Post-Build Selectable.

Post-Build Selectable

This option is used when multiple configurations for CanTp will be created, and the required configuration can be selected at run time.

As shown below, the CanTp_PBS option is ticked.

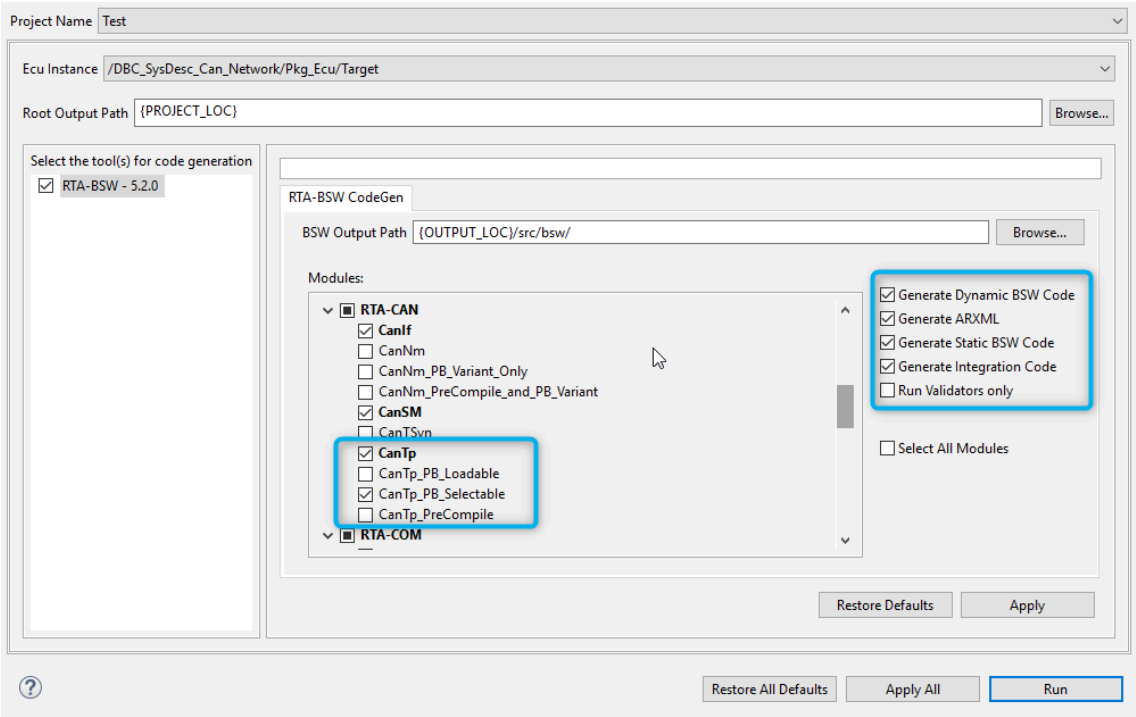


Figure 5.10.Code Generation with Selectable Post-Build

5.8.2 Miscellaneous

- The configuration of the CanTpTxAddressingFormat and CanTpTxTaType parameters will influence the location of target Address and PCI in the CAN data message.
- The configuration of CanTpMainFunctionPeriod must always be configured in seconds, as a float value in the range 0 to 0.255.
- All configurable timers in CanTpTxChannel and CanTpRxChannel must be integer multiples of the CanTpMainfunction period.
- As per AR v22-11, the parameter CanTpChannelMode has been removed with the intention of having all CanTp channels work on full duplex mode by default. For backward compatibility new Rb parameter CanTpRbHalfDuplex-Enable has been introduced at the channel level to provide the same flexibility when choosing channel mode.

5.9 Integration Advice

5.9.1 Init Functions

`CanTp_Init` shall be called by EcuM or BswM during the system initialization. It shall be called before calling Init Functions of other Diagnostic Stack modules.

5.9.2 DeInit Functions

`CanTp_Shutdown` function is required to de-initialize all the RAM areas of the previously active config set. This function is used only when CanTpLite mode is disabled.

5.9.3 Scheduled Functions

`CanTp_MainFunction()` should be called cyclically from a task that has a periodicity equal to the value configured in `CanTpMainFunctionPeriod`.

5.9.4 CanTp_Cfg_BSWMD.arxml

CanTp supports the generation of a CanTp_Cfg_BSWMD.arxml file. This file will contain CanTP timing events such as `CanTp_MainFunction`, this can be used to map the timing events to an OS task using Entity to Task Mapping in ISOLAR-AB. The file can also be used as an input to RTE generation.

5.9.5 RTE generation

During RTE Generation the option `--exclusive-area-optimization=disable` shall be added, otherwise the resource will be optimized.

5.9.6 Dependency to TxConfirmation and RxIndication order

In order to be compliant to UDS transport protocol, the BSW modules in charge of the CanTp implementation requires that the Tx confirmation is received before the Rx indication. To satisfy this requirement it is necessary to configure the system as following:

- When using CAN communication in polling mode `Can_MainFunction_Write()` must be scheduled before the `Can_MainFunction_Read()`.
- When using CAN communication in interrupt mode, the Tx confirmation interrupt must have an higher priority than the Rx indication interrupt.

5.9.7 Post Build Selectable

When Post-build Selectable is used with multiple variants, target variant can be selected by using user defined API - EcuM_DeterminePbConfiguration from EcuM integration. Please refer relevant integration advice from ECU State Management for more detail information.

5.9.8 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The CanTp module uses the following OS resources:

- CanTp_EXCLUSIVE_AREA

5.10 Hardware Requirements

Table 5.25.CanTp Hardware Requirements

Support	Usage
FLASH	7908 bytes
RAM	267 bytes

Usage calculated based on a sample configuration.

6 Time Synchronization over CAN

Module name	CanTSyn
Package	RTA-CAN
AUTOSAR Module ID	161
AUTOSAR Version	22-11

6.1 Introduction

There is no time information in the normal CAN frame. The Time Synchronization over CAN module (CanTSyn), which is a member of the Communication Service layer, handles the distribution of time information over CAN busses. The diagram in the Introduction to this Reference Guide shows the position of the module in the AUTOSAR layered architecture.

The idea of a common time base is based on the concept of a hierarchical clock distribution, with clocks acting as master or slave. An ECU can be a Time Master, Time Slave or Time Gateway.

- The Time Master is the source of the time base, propagates this Time Base to a set of Time Slaves.
- If a Time Master is also the owner of the Global Time Base, the Time Base from which all further Time Bases derived from, then it is the Global Time Master.
- A Time Gateway typically consists of one Time Master port which is connected to one or more Time Slaves.
- A Time Slave is an entity, which is the recipient for a certain Time Base within a certain segment of a communication network, being a consumer for this time base.

A time master sets the time for the ECUs (which are acting as time slaves) connected to it through a communication system. The time slave ECUs get the time from the time master and adjust their own clock to that received time.

This diagram shows CanTSyn's place in the overall Communications architecture.

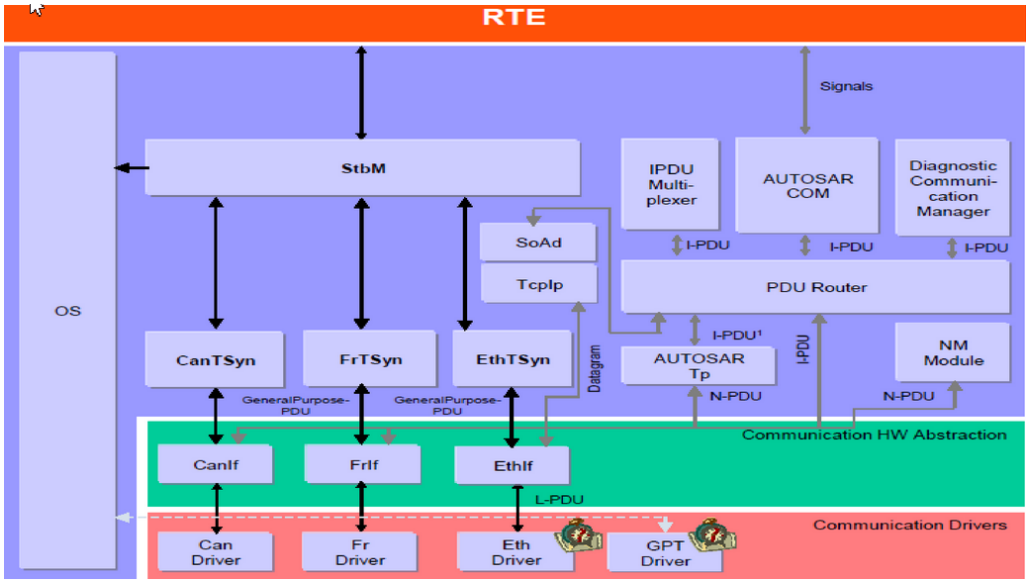


Figure 6.1.CanTSyn Architecture

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

6.2 Supported Features

- Handles the distribution of time information over CAN busses.

6.3 Extensions

6.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

6.3.1.1 CanTSyn/CanTSynGeneral/CanTSynSecurityEventRefs/SEV_T-SYN_CAN_FRESHNESS_NOT_AVAILABLE

Short Name	SEV_TSYN_CAN_FRESHNESS_NOT_AVAILABLE
Description	FV not available from FVM. Context data provides the respective domain ID. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

6.3.1.2 CanTSyn/CanTSynGeneral/CanTSynSecurityEventRefs/SEV_T-

SYN_CAN_ICV_GENERATION_FAILEDSEV_TSYN_CAN_ICV_GEN-ERATION_FAILED

Short Name	SEV_TSYN_CAN_ICV_GENERATION_FAILED
Description	ICV generation for Follow_Up message failed. Context data provides the respective domain ID. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

6.3.1.3 CanTSyn/CanTSynGeneral/CanTSynSecurityEventRefs/SEV_T-SYN_CAN_ICV_VERIFICATION_FAILED

Short Name	SEV_TSYN_CAN_ICV_VERIFICATION_FAILED
Description	ICV verification for Follow_Up message failed. Context data provides the respective domain ID. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

6.3.1.4 CanTSyn/CanTSynGeneral/CanTSynSecurityEventRefs/SEV_T-SYN_CAN_MSG_SEQUENCE_ERROR

Short Name	SEV_TSYN_CAN_MSG_SEQUENCE_ERROR
Description	Failed to receive correct sequence of SYNC and FUP or OFS and OFNS from the TimeMaster within (CanTSynGlobalTimeFollowUpTimeout). Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

6.3.1.5 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeMaster/CanTSynGlobalTimeTxIcvGeneration/CanTSynTxAuthentication-BuildAttempts

Short Name	CanTSynTxAuthenticationBuildAttempts

Description	This parameter specifies the number of authentication build attempts that are to be carried out when the generation of the ICV failed for a given FUP message. If zero is set, then only one ICV build attempt is done. Tags: atp.Status=draft
Multiplicity	1..1
Value Range	0..65535
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

6.3.1.6 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeMaster/CanTSynMasterConfirmationTimeout

Short Name	CanTSynMasterConfirmationTimeout
Description	This represents the confirmation timeout after transmission of each Timesync message.
Introduction	Unit: seconds.
Multiplicity	1..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

6.3.1.7 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeMaster/CanTSynTxTmacCalculated

Short Name	CanTSynTxTmacCalculated
Description	(Not Supported) This parameter controls whether or not TMAC calculation shall be supported. Tags: atp.Status=draft
Multiplicity	0..0
Default Value	-
Enumeration Literals	TMAC_CALCULATED, TMAC_NOT_CALCULATED
Type	ECUC-ENUMERATION-PARAM-DEF

6.3.1.8 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSecureTmacLength

Short Name	CanTSynGlobalTimeSecureTmacLength
Description	(Not Supported) Represents the number of bytes for the used Truncated Message Authentication Code (TMAC). If 0, no message authentication will be used. Tags: atp.Status=draft

Multiplicity	0..0
Value Range	0..16
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

6.3.1.9 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSlave/CanTSynGlobalTimeRxIcvVerification/CanTSynRxAuthentication-BuildAttempts

Short Name	CanTSynRxAuthenticationBuildAttempts
Description	This parameter specifies the number of authentication build attempts that are to be carried out when the verification of the ICV failed for a given FUP message. If zero is set, then only one ICV verification attempt is done. Tags: atp.Status=draft
Multiplicity	1..1
Value Range	0..65535
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

6.3.1.10 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSlave/CanTSynRxTmacValidated

Short Name	CanTSynRxTmacValidated
Description	(Not Supported) This parameter controls whether or not TMAC validation shall be supported. Tags: atp.Status=draft
Multiplicity	0..0
Default Value	-
Enumeration Literals	TMAC_NOT_VALIDATED, TMAC_VALIDATED
Type	ECUC-ENUMERATION-PARAM-DEF

6.3.2 Additional Services

The following API functions are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

6.4 Limitations

The following features are not supported for version v12.7.0:

- Post Build Selectable is not supported for the CanTSynGlobalTimeMaster-ConfirmationHandleId and CanTSynGlobalTimeSlaveHandleId parameters (bug in Autosar).

6.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `CanTSyn_IcvGenerationIndication`
- `CanTSyn_IcvVerificationIndication`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- CanTSyn/CanTSynGeneral/CanTSynSecurityEventRefs/CANTSYN_-SEV_FRESHNESS_NOT_AVAILABLE
- CanTSyn/CanTSynGeneral/CanTSynSecurityEventRefs/CANTSYN_-SEV_ICV_GENERATION_FAILED
- CanTSyn/CanTSynGeneral/CanTSynSecurityEventRefs/CANTSYN_-SEV_ICV_VERIFICATION_FAILED
- CanTSyn/CanTSynGeneral/CanTSynTimeValidationSupport
- CanTSyn/CanTSynGlobalTimeDomain/CanTSynEnableTimeValidation
- CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeNetworkSegmentId
- CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSlave/CanTSyn-GlobalTimeMinMsgGap

6.6 Deviations

6.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

6.6.1.1 CanTSyn/CanTSynGeneral/CanTSynHardwareTimestampSupport

[RTA-BSW] Description	Activate/Deactivate the hardware time stamping functionality of the CAN hardware. Note: HW time stamping feature is supported only as a prototype.
[AUTOSAR] Description	Activate/Deactivate the hardware time stamping functionality of the CAN hardware. Tags: atp.Status=draft
[RTA-BSW] Introduction	* true: Timestamp is retrieved from the CAN hardware. * false: Timestamp is retrieved from the StbM.

[AUTOSAR] Introduction	True: Timestamp is retrieved from the CAN hardware False: Timestamp is retrieved from the StbM

6.6.1.2 CanTSyn/CanTSynGeneral/CanTSynSecurityEventRefs

[RTA-BSW] Description	Container for the references to IdsMEvent elements representing the security events that the CanTSyn module shall report to the IdsM in case the coresponding security related event occurs (and if CanTSynEnable SecurityEventReporting is set to “true”). Tags: atp.Status=draft
[AUTOSAR] Description	Container for the references to IdsMEvent elements representing the security events that the CanTSyn module shall report to the IdsM in case the coresponding security related event occurs (and if Tags: atp.Status=draft
[RTA-BSW] Introduction	The standardized security events in this container can be extended by vendor-specific security events.
[AUTOSAR] Introduction	CanTSynEnableSecurityEventReporting is set to “true”). The standardized security events in this container can be extended by vendor-specific security events.

6.6.1.3 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeFupDataIDList

[RTA-BSW] Description	The DataIDList for FUP messages ensures the identification of data elements due to CRC calculation and message authentication process.(Message authentication process is not supported yet)
[AUTOSAR] Description	The DataIDList for FUP messages ensures the identification of data elements due to CRC calculation and message authentication process.

6.6.1.4 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeFupDataIDList/CanTSynGlobalTimeFupDataIDListElement

[RTA-BSW] Description	Element of the DataIDList for FUP messages ensures the identification of data elements due to CRC calculation and message authentication process.(Message authentication process is not supported yet)
[AUTOSAR] Description	Element of the DataIDList for FUP messages ensures the identification of data elements due to CRC calculation and message authentication process.

6.6.1.5 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeFupDataIDList/CanTSynGlobalTimeFupDataIDListElement/CanTSynGlobalTimeFupDataIDListIndex

[RTA-BSW] Description	Index of the DataIDList for FUP messages ensures the identification of data elements due to CRC calculation and message authentication process.(Message authentication process is not supported yet)
[AUTOSAR] Description	Index of the DataIDList for FUP messages ensures the identification of data elements due to CRC calculation and message authentication process.

6.6.1.6 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeFupDataIDList/CanTSynGlobalTimeFupDataIDListElement/CanTSynGlobalTimeFupDataIDListValue

[RTA-BSW] Description	Value of the DataIDList for FUP messages ensures the identification of data elements due to CRC calculation and message authentication process.(Message authentication process is not supported yet)
[AUTOSAR] Description	Value of the DataIDList for FUP messages ensures the identification of data elements due to CRC calculation and message authentication process.

6.6.1.7 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeMaster

[RTA-BSW] Description	Configuration of a Time Master for a Time Domain (refer to parent container). If CanTSynGlobalTimeMasters container exists, the local ECU acts as a Time Master for the Time Domain.
[AUTOSAR] Description	Configuration of a Time Master for a Time Domain (refer to parent container). If CanTSynGlobalTimeMaster container exists, the local ECU acts as a Time Master for the Time Domain.

6.6.1.8 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeMaster/CanTSynGlobalTimeTxIcvGeneration/CanTSynIcvGenerationTimeout

[RTA-BSW] Description	Timeout of ICV generation (respective CSM job completion in asynchronous behavior). A value of 0 disables the ICV timeout monitoring. Unit: Seconds Tags: atp.Status=draft
[AUTOSAR] Description	Timeout of ICV generation (respective CSM job completion in asynchronous behaviour). Unit: Seconds Tags: atp.Status=draft

6.6.1.9 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeMaster/CanTSynGlobalTimeTxIcvSecured

[RTA-BSW] Description	This parameter controls whether or not ICV generation shall be supported. Tags: Optional parameter since is Prototype feature.
[AUTOSAR] Description	This parameter controls whether or not ICV generation shall be supported. Tags: atp.Status=draft
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

6.6.1.10 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSlave/CanTSynGlobalTimeRxIcvVerification/CanTSynIcvVerificationAttempts

[RTA-BSW] Description	This parameter specifies the number of ICV verification attempts that are to be carried out when the verification of the ICV failed for a given FUP message. If zero is set, then only one ICV verification attempt is done. Tags: atp.Status=draft
[AUTOSAR] Description	This parameter specifies the number of ICV verification attempts that are to be carried out when the verification of the ICV failed for a given FUP message. If zero is set, then only one ICV verification attempt is Tags: atp.Status=draft
[REMOVED] Introduction	done.

6.6.1.11 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSlave/CanTSynGlobalTimeRxIcvVerification/CanTSynIcvVerificationBase

[RTA-BSW] Description	Symmetric or asymmetric cryptography selection for the ICV generation. Tags: atp.Status=draft
[AUTOSAR] Description	Symmetric or asymmetric cryptography selection for the ICV generation Tags: atp.Status=draft

6.6.1.12 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSlave/CanTSynGlobalTimeRxIcvVerification/CanTSynIcvVerificationTimeout

[RTA-BSW] Description	Timeout of ICV generation (respective CSM job completion in asynchronous behavior). A value of 0 disables the ICV timeout monitoring. Unit: Seconds Tags: atp.Status=draft
[AUTOSAR] Description	Timeout of ICV verification (respective CSM job completion in asynchronous behaviour). Unit: Seconds Tags: atp.Status=draft

6.6.1.13 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSlave/CanTSynRxIcvVerificationType

[RTA-BSW] Description	This parameter controls whether or not ICV verification shall be supported. Tags: Optional parameter since is Prototype feature.
[AUTOSAR] Description	This parameter controls whether or not ICV verification shall be supported. Tags: atp.Status=draft
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

6.6.1.14 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSyncDataIDList

[RTA-BSW] Description	The DataIDList for SYNC messages ensures the identification of data elements due to CRC calculation and message authentication process.(Message authentication process is not supported yet)
[AUTOSAR] Description	The DataIDList for SYNC messages ensures the identification of data elements due to CRC calculation and message authentication process.

6.6.1.15 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSyncDataIDList/CanTSynGlobalTimeSyncDataIDListElement

[RTA-BSW] Description	Element of the DataIDList for SYNC messages ensures the identification of data elements due to CRC calculation and message authentication process.(Message authentication process is not supported yet)

[AUTOSAR] Description	Element of the DataIDList for SYNC messages ensures the identification of data elements due to CRC calculation and message authentication process.

6.6.1.16 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSyncDataIDList/CanTSynGlobalTimeSyncDataIDListElement/CanTSynGlobalTimeSyncDataIDListIndex

[RTA-BSW] Description	Index for the DataIDList for SYNC messages ensures the identification of data elements due to CRC calculation and message authentication process.(Message authentication process is not supported yet)
[AUTOSAR] Description	Index for the DataIDList for SYNC messages ensures the identification of data elements due to CRC calculation and message authentication process.

6.6.1.17 CanTSyn/CanTSynGlobalTimeDomain/CanTSynGlobalTimeSyncDataIDList/CanTSynGlobalTimeSyncDataIDListElement/CanTSynGlobalTimeSyncDataIDListValue

[RTA-BSW] Description	Value of the DataIDList for SYNC messages ensures the identification of data elements due to CRC calculation and message authentication process.(Message authentication process is not supported yet)
[AUTOSAR] Description	Value of the DataIDList for SYNC messages ensures the identification of data elements due to CRC calculation and message authentication process.

6.7 API Definition

List of supported API functions

Table 6.29.CanTSyn Supported API Functions

`CanTSyn_Init`	Initialises the Time Synchronization over CAN
`CanTSyn_GetVersionInfo`	Returns the version information of this CanTSyn module
`CanTSyn_SetTransmissionMode`	Turns on and off the TX capabilities of the CanTSyn module
`CanTSyn_RxIndication`	Indication of a received I-PDU from a lower layer communication interface module
`CanTSyn_TxConfirmation`	The lower layer communication interface module confirms the transmission of an IPDU
`CanTSyn_MainFunction`	Main function for cyclic call/response SYNC and FUP transmission

For Further information please refer to AUTOSAR_SWS_TimeSyncOverCAN.pdf

6.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

6.8.1 Configuration of System Extract for ConfGen

In order to automatically generate the configuration for CanTSyn (Master/Slave), the following configuration must be provided in the System Description:

Step 1: Create a Global Time Domain element in System Info.

Step 2: To create the "CanTSynGlobalTimeFupDataIDList" and "CanTSynGlobalTimeSyncDataIDList" containers in GlobalTimeDomain you need to select "Global Time Domain Props".

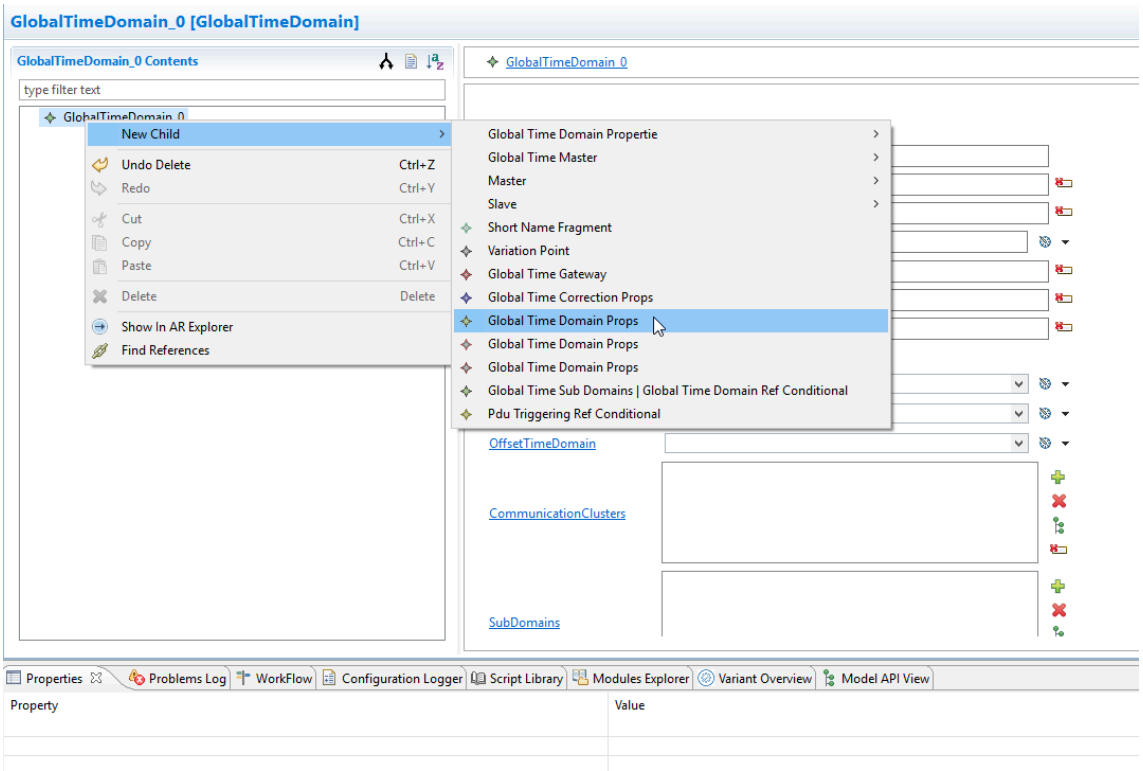


Figure 6.2.Create Global Time Domain element

Step 3: Enter all IDs in FupDataIDLists and SyncDataIDLists.

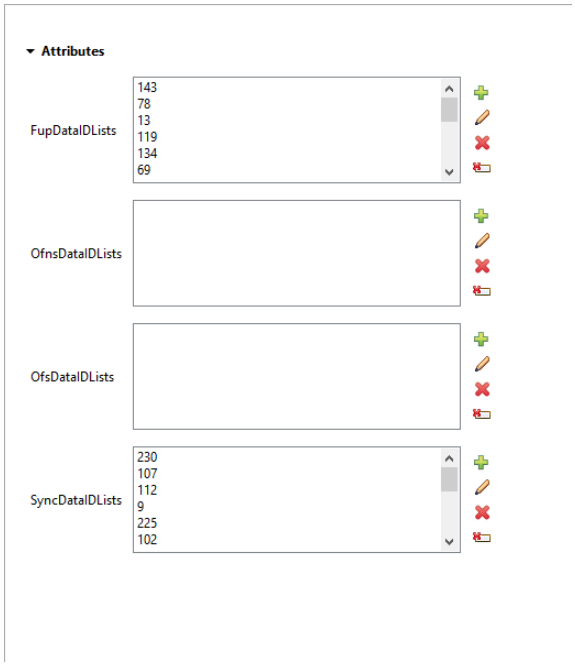


Figure 6.3.ID Lists

Step 4: To create the “CanTSynGlobalTimeMaster” container in GlobalTimeDomain you need to select “Master” and then “Global Time Can Master”

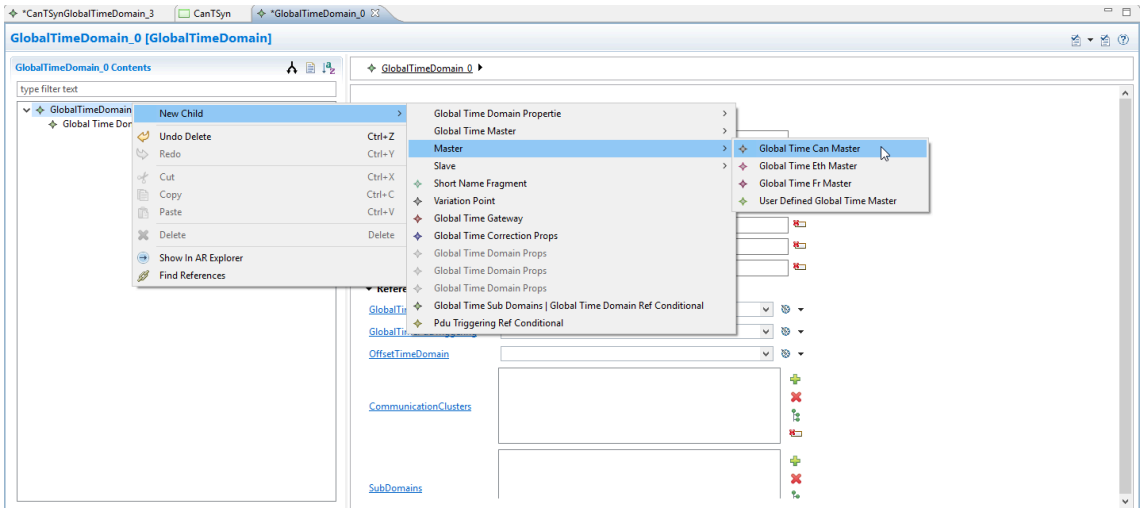


Figure 6.4.Create the CanTSynGlobalTimeMaster container

Step 5: To create the “CanTSynGlobalTimeSlave” container in CanTSynGlobalTimeDomain you need to select “Slave” and then “Global Time Can Slave”

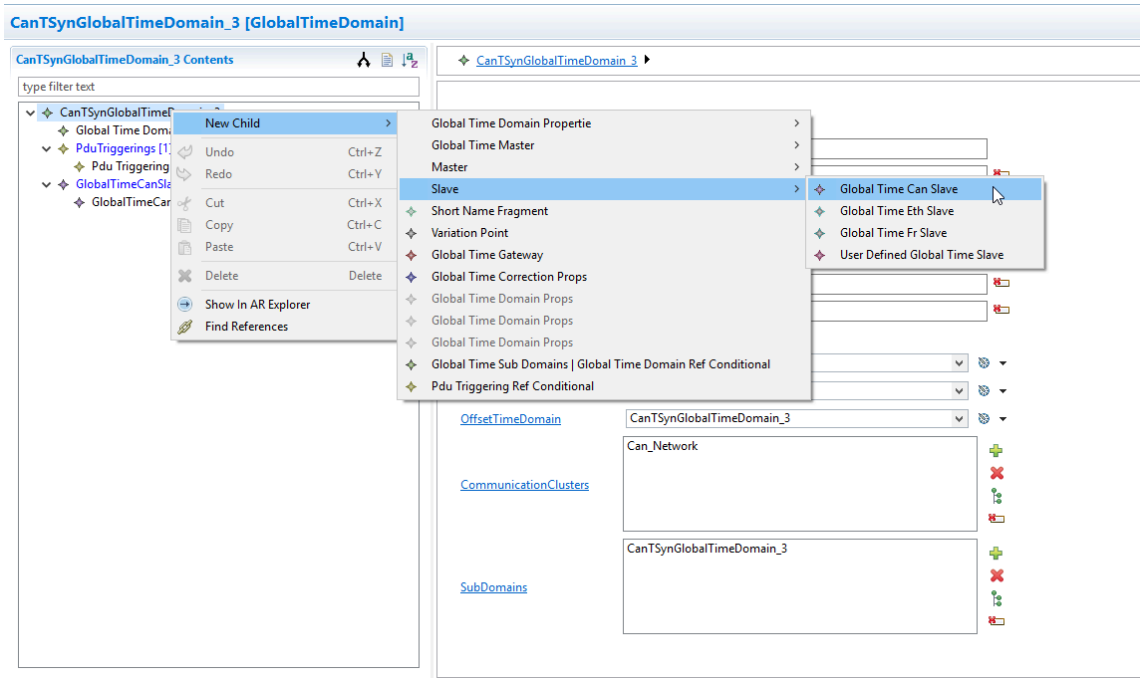


Figure 6.5.Create the CanTSynGlobalTimeSlave container

6.8.2 CRC Module

The CRC module, with XOR value = 0xFF and polynomial = 0x2F, must be available to support CRC in CanTSyn.

6.8.3 SYNC, FUP, OFS and OFNS messages

SYNC, FUP, OFS and OFNS messages are assigned to a dedicated message type "TimeSync", and they share the same CAN ID by using a multiplexed signal group. The Multiplexer is located at Byte 0, named as "Type".

6.8.4 RTE

If the project has RTE, then the `EcuCRbRteInUse` parameter (in `EcuCRbRteInUseEcuC->EcuCRbSettings`) in the EcuC module must be set as TRUE.

6.8.5 StbMSynchronizedTimeBase

For StbMSynchronizedTimeBase, which is referred by a Time master, the `CanTSynGlobalTimeDomain`, `StbMIsSystemWideGlobalTimeMaster` and `StbMAllowSystemWideGlobalTimeMaster` parameters should be configured to TRUE.

6.8.6 Security Event Reporting

If Time Domain Security Event reporting is enabled, then there is a dependency on IdsM (Intrusion Detection System Manager) and a Security Event can be configured.

The following table shows which security events are reported to IdsM:

Det error ID:	69
Error description:	Failed to receive correct sequence of SYNC and FUP, or OFS and OFNS, from the TimeMaster within `CanTSynGlobalTimeFollowUpTimeout`.

To enable this feature, set the parameter `CanTSynEnableSecurityEventReporting`.

6.9 Integration Advice

6.9.1 Init Functions

CanTSyn is initialized via CanTSyn_Init. A call to CanTSyn_Init initializes all internal variables and sets CanTSyn to the initialized state.

With the exception of CanTSyn_GetVersionInfo and CanTSyn_Init, CanTSyn's API functions may only be called when CanTSyn itself has been properly initialized.

If CanTSyn_Init is called while CanTSyn is in an initialized state, all internal variables will be re-initialized.

6.9.2 DeInit Functions

No DeInit is required for CanTSyn.

6.9.3 Scheduled Functions

CanTSyn_MainFunction is called at a cyclic interval, defined by the configuration parameter CanTSynMainFunctionPeriod (unit: second).

6.9.4 General

- CanTSyn_SetTransmissionMode and StbM_SetGlobalTime must be called before CanTSyn messages can be sent. This can be done during Initialization Phase by the BswM module.
- CanTSyn_SetTransmissionMode has to be called with the CanIf controller ID instead of the Can controller ID.

6.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual imple-

mentationof the corresponding BSW API calls to ensure exclusivity.

The CanTSyn module uses the following OS resources:

- CanTSyn_RxIndicationNoNest
- CanTSyn_TxConfirmationNoNest

6.10 Hardware Requirements

Table 6.31.CanTSyn Hardware Requirements

Support	Usage
FLASH	2458 bytes
RAM	41 bytes

Usage calculated based on a sample configuration.

7 Glossary

Below is a list of abbreviations and acronyms that appear in this Reference Guide.

For more information, please refer to the AUTOSAR Glossary (AUTOSAR_TR_-Glossary.pdf) and the "Acronyms and Abbreviations" section of the relevant AUTOSAR module specification (e.g. AUTOSAR_SWS_CANInterface.pdf for CanIf).

Can CC
CAN Communication Controller

CAN CF
CAN Consecutive Frame N-PDU

CAN FC
CAN Flow Control N-PDU

CAN FD
CAN flexible data rate

CAN FF
CAN First Frame N-PDU

CAN N-PDU
The PDU of the CAN Transport Layer

CAN N-SDU
The SDU of the CAN Transport Layer

CAN SF
CAN Single Frame N-PDU

CDD
Complex Device Driver

CRC
Cyclic Redundancy Checksum

DLC
Data Length Code. The portion of an L-PDU which describes the SDU length

ETK
Provides development and calibration tools with a powerful interface to the development ECU

FD Flexible Data

FUP message
Time adjustment message (Follow-Up)

GTD
Global Time Domain

GTM
Global Time Master

HOH
Hardware Object Handle - identifier for a specific CAN mailbox structure

HRH
Hardware Receive Handle - HOH for receive operations (defined by Can)

HTH
Hardware Transmit Handle - HOH for transmission operations (defined by Can)

L-PDU
Protocol Data Unit. Consists of an identifier, DLC and data portion (SDU). Visible to the Can module

L-SDU
Service Data Unit. Data transported inside a CAN L-PDU. Visible to the upper layer modules (e.g. PduR)

MSB
Most Significant Bit

N_Ar
Time from transmission of the CAN frame (any N_PDU) on the receiver side until function `CanTp_TxConfirmation()` is called

N_As
The time for transmission of a CAN frame (any N_PDU) on the part of the sender until function `CanTp_TxConfirmation()` is called

N_Br
Time from receiving the N_PDU CanFrame (FF,CF,FC) until transmission of the next FlowControl N_PDU

N_Bs
Time from transmission N_PDU CanFrame (FF, CF, FC) until receiving of next FlowControl N_PDU

N_Cr
Time from transmission of CAN frame (FC) until reception of the next Consecutive Frame N_PDU

N_Cs
Time from receiving N_PDU FlowControl until transmission of the next Consecutive Frame N_PDU in the sender side

N_PCI
Network protocol control information

N_TAtype
The N_TA parameter shall be used to encode one or multiple (depending on the N_TAtype: physical or functional) receiving network layer protocol entities

OFNS message
Offset adjustment message

OBD
On-Board Diagnosis

OFS message
Offset synchronization message

PDU
Protocol Data Unit

PN Partial Networking

PNI Partial Network Information

Rx Receipt/Reception

Rx L-PDU
Received L-PDU (lower to upper layer)

SDU
Service Data Unit

StbM
Synchronized Time-Base Manager

TD Time Domain

TG Time Gateway

TM Time Master

TS Time Slave

TSD
Time Sub-domain

Tx Transmission

Tx L-PDU
Transmitted L-PDU (upper to lower layer)

UDS
Unified Diagnostic Services

WUF
Wake up flag

XCP
A protocol description (ASAM standard) between a master (tool) and a slave (device)